

《计算机组成与设计:硬件/软件接口》0 基础教辅

基于经典黑皮书 **Patterson & Hennessy 《Computer Organization and Design: The Hardware/Software Interface》** (MIPS 版,第 4/5 版中文版)编写的完整自学教辅。

目标:让一个 **0 基础小白**,不依赖课堂、不需要先修课,也能全面掌握这本书的知识体系。

一、这套教辅是什么、怎么用

1.1 内容组织

文件	内容
README.md (本文件)	全书导览、学习路线、使用方法
01-第1章-计算机抽象与技术.md	计算机全貌、性能评价
02-第2章-指令-计算机的语言.md	MIPS 指令集(本书核心)
03-第3章-计算机的算术运算.md	加减乘除与浮点数
04-第4章-处理器.md	CPU 数据通路与流水线(本书核心)
05-第5章-存储器层次结构.md	cache 与虚拟存储器
06-第6章-并行处理器.md	多核、GPU、集群
07-附录-逻辑设计基础.md	附录 B 精讲(0 基础必读)
08-全书公式与名词速查卡.md	全部公式、MIPS 指令表、名词总索引

1.2 每章的统一结构

每章都按照同一个模板编写,方便你形成学习习惯:

- 本章思维导图** —— 先看骨架,再学血肉(提供树形图和 Mermaid 图,后者可在 Typora / VS Code / GitHub 中直接渲染)
- 学习目标** —— 学完这章你应该能回答哪些问题
- 核心名词先混个脸熟** —— 带着名词去读,不会迷路
- 分节精讲** —— 每个知识点都配:
 - 💡 **大白话注解**:用生活化比喻解释难点
 - ⚠️ **易错点**:初学者最常踩的坑
 - ❓ **常见疑问**:自问自答,深挖一层
 - 📌 **考点提示**:考试/面试常考的位置

5. **名词解释汇总表** —— 一章一张表,复习利器
6. **公式速查** —— 本章所有公式
7. **易混淆概念对比** —— 长得像但不一样的概念
8. **自测题(含答案)** —— 检验自己是否真的掌握
9. **本章小结** —— 两三句话收尾

1.3 符号约定

- 所有专业名词首次出现时附英文原文,如:流水线(pipelining)——本书是翻译版,记住英文对你读原版、查资料都大有好处
 - `$s0` 这类写法表示 MIPS 寄存器; `lw`、`add` 表示 MIPS 指令
-

二、全书总思维导图

树形图(任何地方都能看)

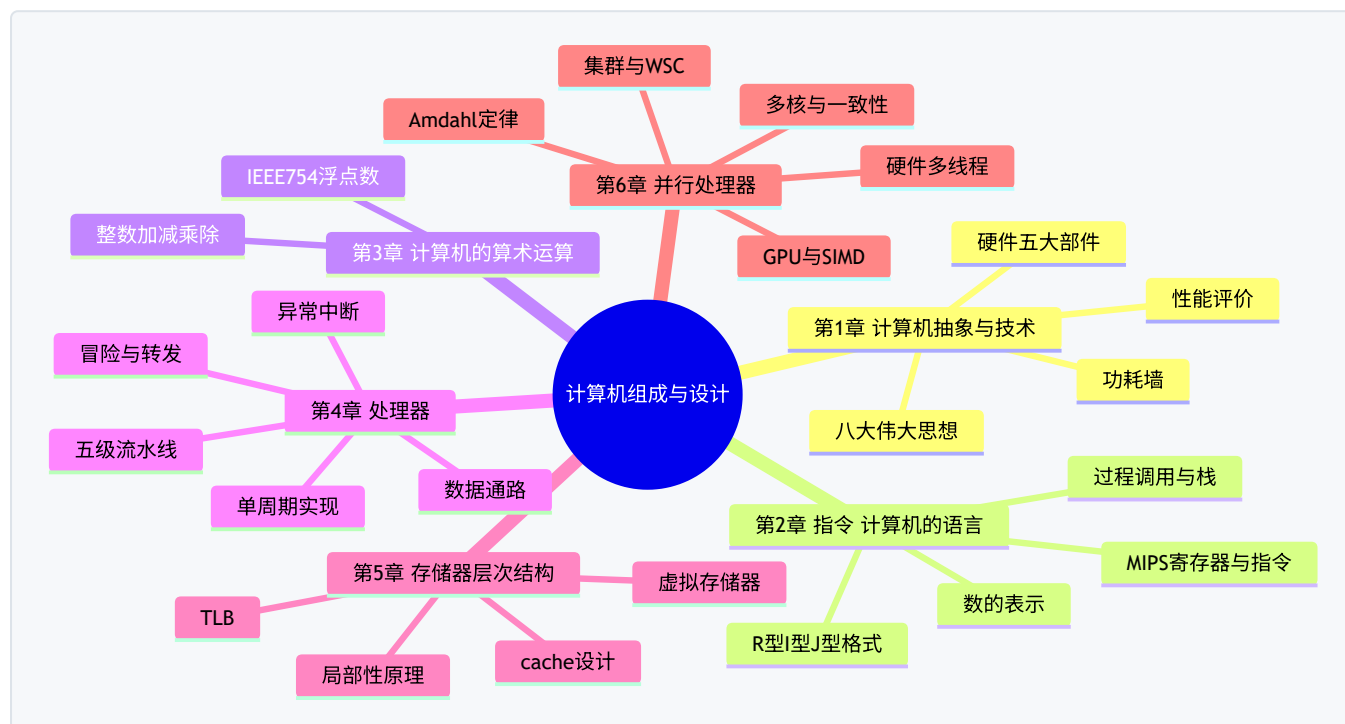
计算机组成与设计(硬件/软件接口)

- 第1章 计算机抽象与技术 — 「站在山顶看全貌」
 - 计算机系统的分层抽象(应用 → 软件 → 硬件)
 - 八大伟大思想(摩尔定律/抽象/加速大概率事件/并行/流水线/预测/存储器层次/冗余)
 - 硬件五大部件(输入/输出/存储器/运算器/控制器)
 - 性能评价:CPU时间 = 指令数 × CPI × 时钟周期
 - 功耗墙与向多处理器的历史转折
- 第2章 指令:计算机的语言 — 「学会说计算机的母语」
 - 指令集(ISA)与 MIPS 体系
 - 操作数:寄存器(\$s0-\$s7、\$t0-\$t9、\$sp、\$ra.....)
 - 算术指令 / 访存指令(lw、sw)/ 立即数
 - 数的表示:二进制、补码、符号扩展
 - 机器语言:R型 / I型 / J型 三种指令格式
 - 逻辑运算与移位、分支与跳转
 - 过程调用:栈、jal/jr、调用约定
 - 寻址方式、原子操作(同步)
 - 程序的一生:编译 → 汇编 → 链接 → 装载
- 第3章 计算机的算术运算 — 「计算机怎么做数学」
 - 整数加减法与溢出检测
 - 乘法与除法(硬件如何实现)
 - 浮点数:IEEE 754 标准(单精度/双精度/舍入)
- 第4章 处理器 — 「CPU 的内部解剖」
 - 数据通路:寄存器组、ALU、存储器如何协作
 - 单周期实现(最简单的 CPU 设计)
 - 流水线:五级(取指/译码/执行/访存/写回)
 - 三类冒险:结构冒险/数据冒险/控制冒险
 - 解决手段:转发(forwarding)、阻塞(stall)、预测
 - 异常与中断
 - 指令级并行:多发射、乱序执行
- 第5章 存储器层次结构 — 「快与大的平衡术」
 - 局部性原理(时间局部性/空间局部性)
 - 存储器技术:SRAM/DRAM/闪存/磁盘
 - cache:直接映射/全相联/组相联、写策略
 - cache 性能:AMAT 公式、十种优化
 - 虚拟存储器:页表、TLB、缺页
 - 通用框架:块放哪?如何找?替换谁?怎么写?
- 第6章 并行处理器 — 「多核时代的生存法则」
 - Amdahl 定律与并行加速的极限
 - SIMD / MIMD / SPMD 分类
 - 向量机与 GPU(SIMT 模型)
 - 硬件多线程(细粒度/粗粒度/同时多线程)
 - 多核处理器与缓存一致性(监听协议)
 - 集群与仓库级计算机(WSC)、MapReduce

附录(本书配套)

- 附录A 汇编器、链接器与 SPIM 模拟器
- 附录B 逻辑设计基础(0 基础必读,见 07 号文件)
- 附录C GPU 专题
- 附录D 映射控制到硬件(FSM 控制 cache)

Mermaid 思维导图(在 Typora / VS Code / GitHub 中可渲染成图)



三、0 基础学习路线(建议)

核心原则:先搭骨架,再填血肉;每章先看思维导图,再读精讲,最后用自测题检验。

阶段 0:热身(半天)

- 读 07-附录-逻辑设计基础.md 的第 1、2 节(什么是与或非门、什么是时钟)。
- 不要求全懂,只求知道"逻辑电路是搭积木"这件事。

阶段 1:建立全局观(约 2 天)——第 1 章

- 目标:能说出"计算机分哪几层、硬件有哪几大件、什么叫性能"。
- 检验:会算 CPU 执行时间 = 指令数 × CPI × 时钟周期。

阶段 2:学会计算机的语言(约 1~2 周)——第 2 章 ★最重要

- 目标:能手写 MIPS 汇编程序(循环、分支、过程调用),能读懂机器码格式。
- 建议:边学边在 **SPIM / MARS 模拟器**上跑书上的例子(免费软件,网上搜 MARS MIPS simulator)。

- 这章是全书的根基,第 3~5 章都建立在这章之上,值得花最多时间。

阶段 3:计算机怎么做数学(约 3~4 天)——第 3 章

- 目标:理解补码、溢出、IEEE 754 浮点格式,会手工做浮点加减。
- 这章相对独立,学累了第 2 章可以穿插学习换换脑子。

阶段 4:CPU 的内部解剖(约 1~2 周)——第 4 章 ★最重要

- 目标:理解五级流水线、三类冒险、转发和阻塞的原理,能画出流水线时空图。
- 建议:先复习 07-附录-逻辑设计基础.md 的寄存器和 FSM 部分,否则数据通路图会看不懂。

阶段 5:存储的魔法(约 1 周)——第 5 章

- 目标:理解局部性、cache 三种映射方式、写策略、TLB 和虚拟存储。
- 检验:会算 AMAT,会做 cache 地址划分(标记/索引/偏移)。

阶段 6:走向并行(约 3~4 天)——第 6 章

- 目标:理解 Amdahl 定律、SIMD/MIMD、多核一致性、GPU 与 CPU 的区别。
- 这章偏概念,压力比第 2、4 章小,适合收尾。

阶段 7:冲刺复习

- 用 08-全书公式与名词速查卡.md 过一遍全部公式和名词,再回看每章的自测题。

四、版本说明(第 4 版 vs 第 5 版)

本教辅以 第 5 版(中文版) 章节结构为蓝本,同时兼容第 4 版。两个版本主要差异:

差异点	第 4 版	第 5 版
GPU 内容	附录 C	并入第 6 章
第 6 章名称	并行与存储器层次?(以实际书为准)	从客户端到云的并行处理器
指令集	均为 MIPS	均为 MIPS
章节数	6 章 + 附录	6 章 + 附录

无论你手头是哪一版,按本教辅的六章顺序学习都成立:第 4/5 版在核心内容(第 1~5 章)上几乎一致。

五、给小白的三句忠告

1. **不要跳读第 2 章。** 这本书的一切(算术、CPU、存储)都建立在"指令"之上,第 2 章不扎实,后面全是空中楼阁。
2. **公式不要死背,要会"讲故事"。** 比如 CPI 是什么?是一条指令平均要花几个时钟周期——把这个故事讲出来,公式自然就记住了。
3. **动手胜过一切。** 装一个 MARS 模拟器跑 MIPS 代码;画一张流水线时空图;手算一次 cache 地址划分。做一遍,胜过读十遍。

现在,从第 1 章开始吧 → [01-第1章-计算机抽象与技术.md](#)

第 1 章 计算机抽象与技术

一句话导读:这一章带你站在山顶俯瞰整本书的地图——计算机是怎么分层的、硬件由哪些部件组成、以及"性能"到底怎么衡量。本章没有复杂电路,是全书中"性价比最高"的一章。

对应教材:第 1 章「计算机抽象与技术(Computer Abstractions and Technology)」

一、本章思维导图

第1章 计算机抽象与技术

1.1 八大伟大思想(贯穿全书的主线!)

- 面向摩尔定律的设计 → 设计要有前瞻性
- 抽象简化设计 → 分层!每层只需知道接口
- 加速大概率事件 → 把最常发生的事做到最快
- 通过并行提高性能 → 同时干多件事
- 通过流水线提高性能 → 像工厂流水线一样重叠执行
- 通过预测提高性能 → 猜对了就赚到
- 存储器层次结构 → 快的贵、慢的便宜,分层搭配
- 通过冗余提高可靠性 → 备胎思维

1.2 程序表象之下(软件分层)

- 高级语言 → 汇编语言 → 机器语言 → 硬件
- 系统软件:编译器、汇编器、操作系统
- 抽象:接口(interface)隐藏实现细节

1.3 硬件表象之下(硬件组成)

- 五大经典部件:输入/输出/存储器/运算器/控制器
- 显示器、触摸屏(输入+输出)
- 处理器:数据通路 + 控制器
- 存储器:DRAM(内存)、cache(缓存)
- 数据存储:磁盘、闪存
- 通信:局域网、广域网

1.4 集成电路与制造

- 晶体管、硅片、晶圆(wafer)、芯片(die)
- 芯片成本 = 晶圆成本 ÷ (每晶圆芯片数 × 成品率)
- 摩尔定律:晶体管数每 18~24 个月翻倍

1.5 性能(本章的计算核心)

- 响应时间(个人用户) vs 吞吐率(数据中心)
- CPU时间 = 指令数 × CPI × 时钟周期
- 相对性能:"A 是 B 的 n 倍快"
- 时钟频率 vs 时钟周期(互为倒数)
- MIPS 指标及其缺陷
- SPEC 基准测试

1.6 功耗墙

- 动态功耗 $\propto$ 容性负载 × 电压² × 频率
- Dennard 缩放定律的终结
- 暗硅:芯片上不能同时开机的部分

1.7 沧海巨变:从单处理器到多处理器

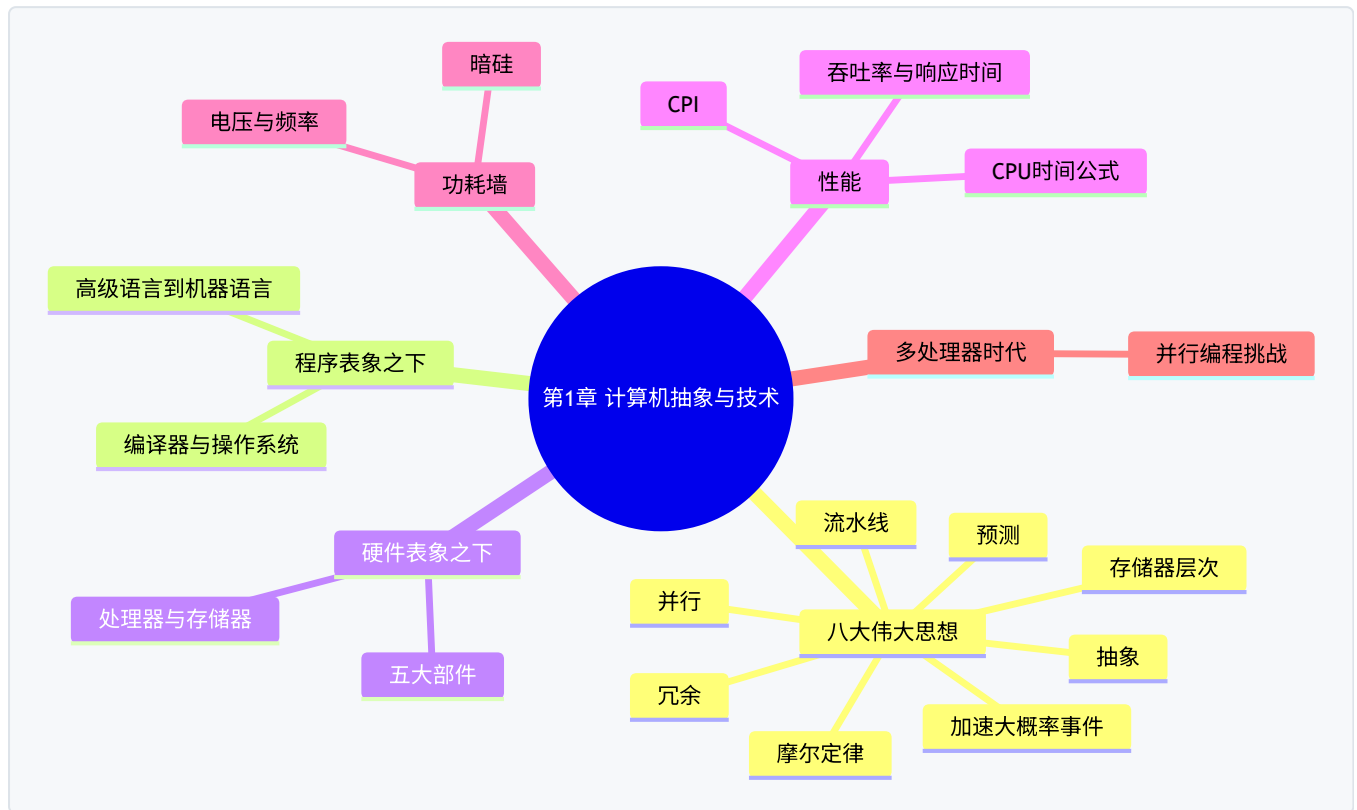
- 为什么:功耗墙 + 指令级并行挖尽
- 挑战:并行编程困难(预告 Amdahl 定律)
- 与八大伟大思想的呼应

1.8 谬误与陷阱(考试常客)

- 峰值性能 ≠ 实际性能

- └ 不能只比时钟频率(还要比 CPI)
- └ 只用执行时间的一部分作比较会失真

Mermaid 版(可渲染)



二、学习目标

学完本章,你应该能回答:

1. 计算机系统从高级语言到硬件分哪几层?每层的"接口"是什么?
2. 八大伟大思想分别是什么?各举一个生活化例子。
3. 硬件五大经典部件是哪五个?
4. 为什么"响应时间"和"吞吐率"是两回事?
5. 会算:**CPU 执行时间 = 指令数 × CPI × 时钟周期** 及其变式。
6. 为什么说 MIPS 指标有缺陷?为什么不能只比主频?
7. 什么是摩尔定律、功耗墙、暗硅?
8. 为什么处理器行业从"提高单核主频"转向"堆多核"?

三、核心名词先混个脸熟

名词	一句话
指令集架构(ISA)	软件与硬件之间的"合同",规定指令、寄存器等

名词	一句话
抽象(abstraction)	只暴露接口、隐藏内部细节的分层方法
编译器(compiler)	把高级语言翻译成汇编/机器语言的程序
汇编器(assembler)	把汇编语言翻译成机器码的程序
操作系统(OS)	管理硬件资源、给程序提供服务的软件
晶体管(transistor)	芯片上的电子开关,计算机的基本元件
晶圆(wafer)	制造芯片用的圆形硅片
响应时间(response time)	完成一个任务所需的时间
吞吐率(throughput)	单位时间内完成的任务数量
时钟周期(clock cycle)	CPU 心跳一次的时间
CPI	每条指令平均消耗的时钟周期数
MIPS	每秒百万条指令,一种性能指标
SPEC	权威的基准测试(benchmark)套件
摩尔定律(Moore's Law)	晶体管数量每 18~24 个月翻倍
暗硅(dark silicon)	因功耗限制无法同时工作的芯片区域

四、分节精讲

1.1 八大伟大思想 —— 全书的"暗线"

这八条思想不是背完就扔的考点,而是**贯穿全书每章的灵魂**。后面每学一个新概念,都可以回头问一句:"这是八大思想里的哪一条?"(如第 4 章流水线=第 5 条,第 5 章 cache=第 7 条。)

1. 面向摩尔定律的设计(Design for Moore's Law)

- 摩尔定律:芯片上的晶体管数量大约每 18~24 个月翻一番。
- 含义:设计硬件时要预见到几年后资源会翻好几倍,设计要有前瞻性。
- 💡 大白话:买电脑时按"两年后的需求"来配,而不是按今天的需求。

2. 抽象简化设计(Use Abstraction to Simplify Design)

- 抽象=分层,每层只和上下层通过"接口"打交道,内部细节被隐藏。
- 例子:你开车不用知道发动机怎么工作(接口:方向盘、油门);程序员不用知道 CPU 电路怎么布线(接口:指令集)。
- ? 为什么抽象能简化设计?因为人脑一次只能处理有限复杂度;分层后每层只需关注自己的事,出问题也好定位。

3. 加速大概率事件(Make the Common Case Fast)

- 把资源投在最常发生的操作上。
- 例子:加减法比乘法常用 → 早期 CPU 上加减法比乘法快得多;cache 就是"大概率事件=访问最近用过的数据"。

4. 通过并行提高性能(Performance via Parallelism)

- 并行=同时做多件事。大到多核处理器,小到一条指令内部的多位加法同时算。
- 例子:搬 100 箱货,10 个人一起搬比 1 个人快。

5. 通过流水线提高性能(Performance via Pipelining)

- 流水线把一件大事拆成多个小步骤,前后任务重叠执行。
- 💡 大白话:洗车流水线——车 A 在冲洗时,车 B 同时在擦干。每一辆车还是花同样时间,但"出车速度"大大加快。
- 第 4 章的主角。

6. 通过预测提高性能(Performance via Prediction)

- 提前猜测,猜对了就省时间,猜错了再纠正。
- 例子:分支预测(猜程序往哪走)、cache 预取(提前把可能要用的数据取来)。

7. 存储器层次结构(Hierarchy of Memories)

- 又快又大的存储器不存在(快=贵)。解决:金字塔式分层——最顶层小、快、贵(寄存器、cache),底层大、慢、便宜(内存、硬盘),让常用数据待在高层。
- 第 5 章的主角。

8. 通过冗余提高可靠性(Dependability via Redundancy)

- 关键部件做备份,坏了有替换。
- 例子:飞机多个发动机、RAID 磁盘阵列、服务器双电源。

1.2 程序表象之下:软件怎么分层

四层结构(自上而下):

```

高级语言(C、Java、Python)
  ↓ 编译器(compiler)
汇编语言(面向特定指令集的助记符)
  ↓ 汇编器(assembler)
机器语言(二进制 0/1,CPU 真正执行的东西)
  ↓ 硬件直接执行
硬件(处理器、存储器、I/O)
  
```

关键点:

- **指令集架构(ISA, Instruction Set Architecture):**汇编层与硬件层之间的"合同"。它规定:有哪些指令、哪些寄存器、内存如何编址。MIPS 就是一种 ISA(第 2 章全面学习)。
- 同一份高级语言代码,可以编译给不同 ISA 的机器(所以 C 程序既能跑在 MIPS 上也能跑在 x86 上);而汇编语言是"绑死"在某一种 ISA 上的。

- 💡 大白话:ISA 就像 USB 接口标准——只要双方都遵守标准,任何充电头配任何手机线都能用。

系统软件(system software):为用户程序提供服务的软件,核心是:

- 操作系统(OS):**管理硬件资源(CPU、内存、外设),提供文件、网络等抽象。
- 编译器(compiler):**高级语言 → 汇编语言(或直接到机器码)。
- 汇编器(assembler):**汇编语言 → 机器码(详见第 2 章 2.13 节)。
- 加载器/链接器(loader/linker):**把多个目标文件拼成可执行程序。

1.3 硬件表象之下:五大经典部件

任何计算机都逃不出这五件(称为计算机的经典组成):

部件	作用	你熟悉的对应物
输入设备(input)	把外界信息送进机器	键盘、鼠标、触摸屏
输出设备(output)	把结果呈现给外界	显示器、打印机
存储器(memory)	存数据和程序	内存条(DRAM)、硬盘、cache
运算器(datapath,数据通路)	执行算术/逻辑运算	CPU 内部的 ALU
控制器(control)	指挥其他部件按序工作	CPU 内部的控制器

⚠ 易错点:运算器在本书中常被称为**数据通路(datapath)**,它包含寄存器组和 ALU 等;控制器(control)则根据指令产生控制信号。**"处理器(CPU)= 数据通路 + 控制器"**,考试爱考这个等式。第 4 章会解剖这两部分。

几个硬件术语:

- CPU(中央处理器):**执行指令的部件 = 数据通路 + 控制器。
- 内存(main memory,主存):**通常指 DRAM,掉电丢失(易失);CPU 正在跑的程序和数据都在这里。
- 辅助存储器(secondary memory):**磁盘、闪存(SSD),掉电不丢(非易失)。
- cache(缓存):**位于 CPU 和主存之间的小而快的存储器(第 5 章)。
- I/O 设备:**输入输出设备的总称,通过总线/接口与 CPU、内存相连。

1.4 集成电路:芯片是怎么造出来的

制造链条:硅锭 → 切成薄片 → **晶圆(wafer)** → 光刻刻出电路 → 切成一个个 **芯片(die)** → 封装测试。

芯片成本公式(常考计算题):

每个芯片成本 = 晶圆成本 ÷ (每晶圆芯片数 × 成品率)

- 每晶圆芯片数 ≈ 晶圆面积 ÷ 芯片面积(忽略边缘损耗)
- 成品率(yield)= 合格芯片的比例。芯片越大,缺陷命中它的概率越高,成品率越低。
- 所以:芯片面积越大 → 每晶圆芯片数越少 + 成品率越低 → 成本暴涨。这是芯片不能无限做大的经济原因。

摩尔定律:1965 年戈登·摩尔提出,芯片上晶体管数量约每 18~24 个月翻倍。几十年里,它既是"预言"也是行业的目标:大家都按这个速度迭代,掉队就被淘汰。

✦ 考点提示:摩尔定律说的是**晶体管数量/集成度**翻倍,不是性能翻倍、不是主频翻倍——虽然历史上性能也随之增长,但概念要分清。

1.5 性能:本章的计算核心 ★

1.5.1 两个角度:响应时间 vs 吞吐率

- **响应时间(response time)/ 执行时间(execution time):**完成**一个**任务要多久。个人用户最关心——"我的游戏加载要几秒?"
- **吞吐率(throughput):**单位时间完成**多少**任务。数据中心最关心——"这个服务器每秒能处理多少请求?"

⚠ 易错点:两者可能背道而驰。例:给洗车店再加 10 个工位,单辆车洗得更慢(响应时间变差),但每天洗的车总数变多(吞吐率提升)。

1.5.2 相对性能:"n 倍快"

"A 是 B 的 n 倍快" $\Leftrightarrow$ 性能A $\div$ 性能B = n $\Leftrightarrow$ 执行时间B $\div$ 执行时间A = n

关键:**性能与执行时间互为倒数**。所以"快 n 倍"必须用**执行时间之比**来算,而不是直接比频率或 MIPS。

💡 大白话:甲跑 100 米用 10 秒,乙用 20 秒。甲的时间是乙的一半,所以甲快 2 倍——用时间比,别用别的数比。

1.5.3 时钟:CPU 的心跳

- **时钟周期(clock cycle time,记为 T):**CPU 每"心跳"一次的时间,单位秒。
- **时钟频率(clock rate)= 1 / T,**单位 Hz。3 GHz 主频 $\Leftrightarrow$ 时钟周期 ≈ 0.333 ns(纳秒)。
- 注意单位换算:1 GHz = 10^9 Hz, 1 ns = 10^{-9} s。 **频率与周期互为倒数,频率越大周期越小。**

1.5.4 核心公式:CPU 执行时间

CPU 执行时间 = CPU 时钟周期数 $\times$ 时钟周期时间
= CPU 时钟周期数 $\div$ 时钟频率

CPU 时钟周期数 = 指令数(IC) $\times$ CPI

$\therefore$ CPU 执行时间 = 指令数 $\times$ CPI $\times$ 时钟周期时间

- **指令数(IC, Instruction Count):**程序一共执行多少条指令。由**程序和编译器+ISA**决定。
- **CPI(Cycles Per Instruction):**平均每条指令消耗几个时钟周期。由**处理器设计和 ISA**决定。
- **时钟周期:**由**硬件工艺和设计**决定。

✦ 这是全书最重要的公式之一,考试几乎必考。记住"三个因子各由谁决定"这张表,选择题常考。

例题(必须会做):

程序 P 有 10^9 条指令,在 2 GHz 的 CPU 上运行,平均 CPI = 2。求执行时间。

解:执行时间 = $10^9 \times 2 \div (2 \times 10^9) = 1$ 秒。

变式:同样程序在 CPU A(4 GHz,CPI=1)和 CPU B(2 GHz,CPI=0.8)上,哪个快?

A:时间 = $IC \times 1 / 4 \times 10^9 = 0.25 \times IC / 10^9$ B:时间 = $IC \times 0.8 / 2 \times 10^9 = 0.4 \times IC / 10^9 \rightarrow$ A 更快。可见主频低但 CPI 更低也可能更快,**不能只看主频**。

1.5.5 MIPS 指标及其缺陷

$MIPS = \text{指令数} \div (\text{执行时间} \times 10^6)$ [百万条指令每秒]

它等效于 频率 $\div (CPI \times 10^6)$ 。缺陷:

1. **不同指令集之间不可比:**RISC 机器完成同一任务指令数多,CISC 指令数少,直接比 MIPS 不公平。
2. **同一台机器上不同程序 MIPS 不同:**MIPS 是"程序相关"的,不是机器的固有属性。
3. **MIPS 可能和执行时间相反:**CPI 高的机器配高主频可能 MIPS 更高,但实际执行时间更长。

✦ 考点提示:"为什么 MIPS 不是好的性能指标"是经典简答题,答上述三条即可。

1.5.6 SPEC 基准测试

- 用一组**真实程序**(编译器、数据库、游戏引擎等)跑分,取几何平均,给出**标准化分数(SPECratio)**:分数高者性能好。
- 分数是"相对一台参考机器"的比值,可以直接说"这台机器 SPEC 分数是那台的 2 倍"。

1.6 功耗墙:为什么 CPU 不能一直变快

动态功耗公式(常考):

动态功耗 $\propto 1/2 \times \text{容性负载} \times \text{电压}^2 \times \text{开关频率}$

三个参数都能优化,但**电压²是平方项**,最"致命":

- 历史上每代工艺把电压降低,从而在频率上升的同时控制功耗——这被称为 **Dennard 缩放定律**。
- 但电压已降到接近物理极限(约 1V),**Dennard 缩放定律失效** $\rightarrow$ 频率再往上加,功耗爆炸、散热跟不上 $\rightarrow$ **功耗墙(power wall)**。
- 结果:**暗硅(dark silicon)**——芯片上很大一块区域因功耗限制不能同时工作,只能轮流开机。

💡 大白话:CPU 像一间教室,以前每学期都能把"电压"这个空调温度调低一点,于是风扇(频率)可以越转越快。现在温度调到最低了,风扇再快教室就热炸——只能另想办法(多开几间教室=多核)。

1.7 沧海巨变:从单处理器到多处理器

- **转折原因:**① 功耗墙挡住主频路线;② 单核内部的指令级并行(ILP)已被挖尽,再堆硬件收益甚微。

- **新方向:**多核处理器(multicore)——把多个处理器核放进一个芯片,靠**显式并行**(程序自己拆分任务)提速。
- **代价:**并行编程远比串行编程难,不是所有程序都能并行。
- **✦ Amdahl 定律**(本章预告,第 6 章详解):若程序只有 P 部分可并行,则无论多少核,加速比上限 = $1/(1-P)$ 。比如一半代码串行,加速比最多 2 倍。

1.8 谬误与陷阱

1. **谬误:峰值性能(peak performance)就是实际性能。** 峰值是"所有部件满负荷"的理论值;真实程序受 cache 缺失、分支、依赖等影响,实际性能常只有峰值的一小部分。
2. **谬误:只看时钟频率选 CPU。** 主频高但 CPI 高的机器可能更慢(见 1.5.4 变式)——要综合"频率 × 每周期能干的活"。
3. **陷阱:只用执行时间的一部分做比较。** 比如只比"CPU 时间"而忽略 I/O 时间,或把开机加载时间摊薄到大量任务上,结论都会失真。
4. **陷阱:忽视 Amdahl 定律。** 并行部分再快,串行部分不解决,整体加速有天花板(第 6 章详解)。

五、名词解释汇总表

名词	英文	解释
指令集架构	Instruction Set Architecture (ISA)	软硬件之间的接口契约:规定指令、寄存器、内存编址
抽象	Abstraction	通过分层隐藏内部细节,只暴露接口
汇编器	Assembler	将汇编语言翻译为机器码的程序
编译器	Compiler	将高级语言翻译为汇编/机器语言的程序
操作系统	Operating System	管理硬件资源、提供程序接口的系统软件
数据通路	Datapath	处理器中执行运算的部分(ALU、寄存器等)
控制器	Control	产生控制信号、指挥数据通路工作的部件
晶体管	Transistor	芯片上的基本开关元件
晶圆	Wafer	制造芯片的圆形硅片,被切割成多块芯片(die)
成品率	Yield	晶圆上合格芯片的比例
响应时间	Response time	单个任务从开始到完成的时间
吞吐率	Throughput	单位时间内完成的任务总数
时钟周期	Clock cycle	CPU 基本节拍的时间长度
时钟频率	Clock rate	时钟周期的倒数,如 2 GHz
CPI	Cycles Per Instruction	平均每条指令所需的时钟周期数
指令数	Instruction Count	程序执行的指令总条数

名词	英文	解释
MIPS	Million Instructions Per Second	每秒百万条指令,一种有缺陷的性能指标
基准测试	Benchmark	用于评测性能的典型程序套件,如 SPEC
摩尔定律	Moore's Law	芯片晶体管数每 18~24 个月翻倍
功耗墙	Power Wall	因功耗/散热限制无法继续提高主频的障碍
暗硅	Dark Silicon	芯片上因功耗限制无法同时工作的区域
多核处理器	Multicore processor	集成多个处理器核的芯片
系统软件	System software	为程序提供服务、管理硬件的软件(OS 等)

六、公式速查

公式	说明
时钟频率 = $1 / \text{时钟周期}$	互为倒数
CPU执行时间 = CPU时钟周期数 $\times$ 时钟周期	基本式
CPU时钟周期数 = 指令数 $\times$ CPI	拆分式
CPU执行时间 = 指令数 $\times$ CPI $\times$ 时钟周期	核心公式
性能X $\div$ 性能Y = 执行时间Y $\div$ 执行时间X	"n 倍快"
MIPS = 指令数 $\div$ (执行时间 $\times 10^6$)	每秒百万指令
芯片成本 = 晶圆成本 $\div$ (每晶圆芯片数 $\times$ 成品率)	制造成本
动态功耗 $\propto$ 容性负载 $\times$ 电压 ² $\times$ 开关频率	功耗墙
加速比上限 = $1 / (1 - P)$ (Amdahl,预告)	P=可并行比例

七、易混淆概念对比

对比	区别一句话
响应时间 vs 吞吐率	一个任务多久(个人视角)vs 单位时间多少任务(系统视角)
时钟周期 vs 时钟频率	互为倒数:周期是时间(秒),频率是速率(Hz)
性能 vs 执行时间	互为倒数:执行时间越短,性能越高
编译 vs 汇编	编译器处理高级语言;汇编器处理汇编语言
CPU vs 数据通路	CPU = 数据通路 + 控制器,数据通路只是其中一部分

对比	区别一句话
主存(DRAM) vs 辅助存储(磁盘)	前者易失、快、贵;后者非易失、慢、便宜
MIPS vs CPI	MIPS 是速率指标(越大越好但有缺陷);CPI 是平均周期数(越小越好)
摩尔定律 vs Dennard 缩放	前者:晶体管数翻倍;后者:同功耗下频率可提升(已失效)

八、自测题

1. (简答)计算机系统从高级语言到硬件的四层分别是什么?每两层之间的"翻译者"是谁?
2. (填空)CPU = _____ + _____。
3. (计算)程序 P 指令数 2×10^9 条, CPI=1.5, 时钟频率 3 GHz, 求执行时间。
4. (计算)机器 A 运行程序 X 需 8 秒, 机器 B 需 10 秒。A 比 B 快多少倍?
5. (计算)CPI=2, 主频 3 GHz 的机器, MIPS 指标是多少?
6. (简答)为什么 MIPS 不是可靠的性能指标?至少说两条。
7. (简答)什么是功耗墙?它如何导致了多核时代的到来?
8. (判断)摩尔定律说 CPU 主频每 18 个月翻倍。(对/错, 并说明)
9. (简答)八大伟大思想中, 第 4 章"流水线"和第 5 章"cache"分别对应哪两条思想?
10. (计算)晶圆成本 5000 元, 每晶圆可切 400 个芯片, 成品率 0.8, 求单芯片成本。

参考答案

1. 高级语言→(编译器)→汇编语言→(汇编器)→机器语言→硬件。操作系统也属于横跨各层的系统软件。
2. 数据通路(datapath)+ 控制器(control)。
3. $T = 2 \times 10^9 \times 1.5 \div (3 \times 10^9) = 1$ 秒。
4. 性能A/性能B = 时间B/时间A = $10/8 = 1.25$, 即 A 比 B 快 1.25 倍。
5. $MIPS = \text{频率} / (CPI \times 10^6) = 3 \times 10^9 / (2 \times 10^6) = 1500$ MIPS。
6. ① 不同 ISA 间不可比(指令数口径不同);② 同一机器跑不同程序 MIPS 不同(程序相关);③ MIPS 高不代表执行时间短。
7. 动态功耗 $\propto$ 电压²×频率, Dennard 缩放失效后主频无法继续提高(散热跟不上), 于是行业转向在芯片内堆多个核心, 靠并行提升性能。
8. 错。摩尔定律说的是**晶体管数量**翻倍, 不是主频。
9. 流水线=第 5 条"通过流水线提高性能";cache=第 7 条"存储器层次结构"。
10. $5000 \div (400 \times 0.8) = 15.625$ 元。

九、本章小结

- 计算机 = 分层抽象的艺术: 上层只用接口, 不关心下层细节。
- 八大伟大思想是全书的灵魂, 后面每章都在践行它们。

- 性能评价记住一条主线:**CPU时间 = 指令数 × CPI × 时钟周期**,以及"性能与时间互为倒数".
- 历史转折:功耗墙终结了主频竞赛,计算机进入多核时代。

下一章 → [02-第2章-指令-计算机的语言.md](#):开始学习 MIPS 指令集,这是全书最重要的一章。

第 2 章 指令:计算机的语言

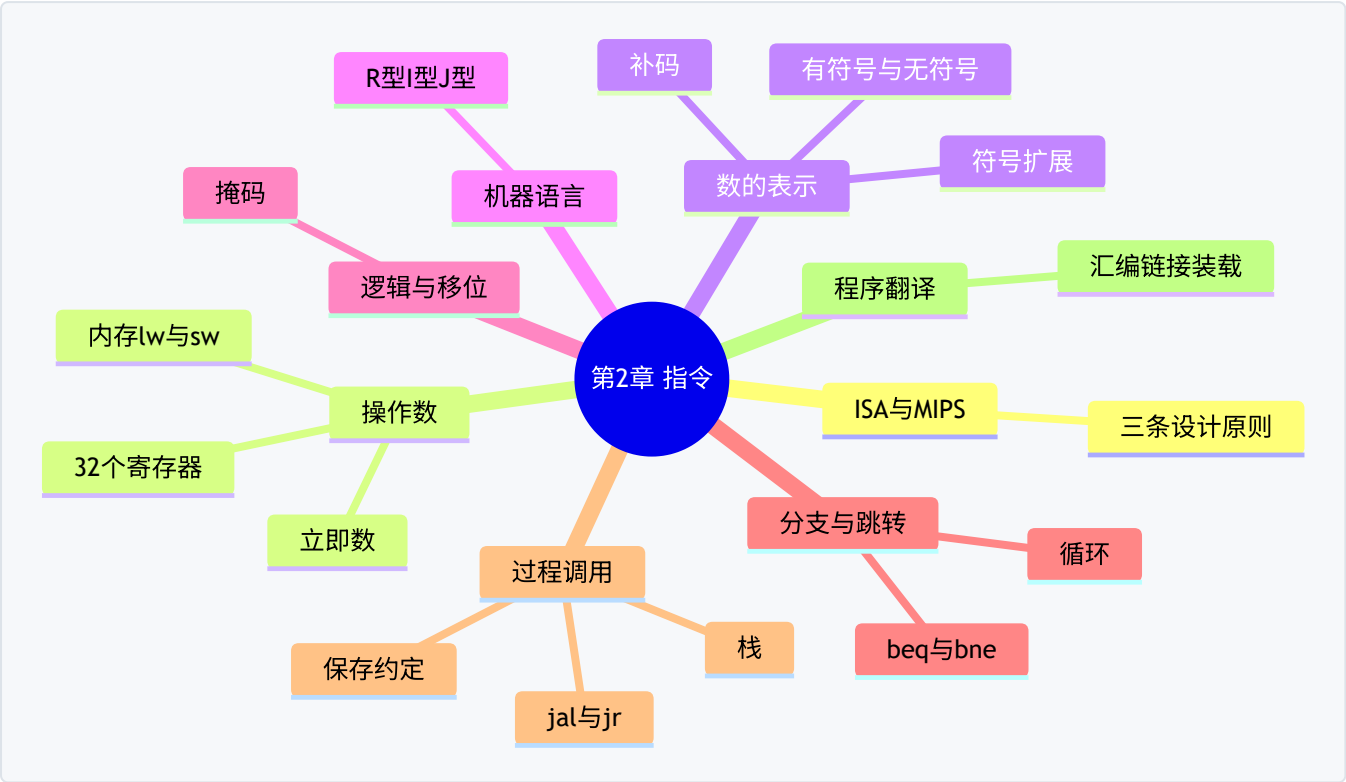
一句话导读:这一章教你"说计算机的母语"——MIPS 汇编。学完你会明白:高级语言里的一行代码,在 CPU 眼里是怎样一堆 0 和 1。这是**全书最重要的一章**,第 3、4、5 章全都建立在它之上。

对应教材:第 2 章「指令:计算机的语言(Instructions: Language of the Computer)」

一、本章思维导图

第2章 指令:计算机的语言

- 2.1 指令集架构(ISA)与 MIPS
 - ISA = 软硬件之间的"合同"
 - MIPS:经典 RISC 架构
 - 三条设计原则(贯穿全章!)
 - 原则1:简单源于规整
 - 原则2:越小越快
 - 原则3:加速大概率事件
- 2.2 算术运算:add / sub(三操作数,全是寄存器)
- 2.3 操作数
 - 寄存器:32 个 32 位寄存器(\$zero、\$s0-\$s7、\$t0-\$t9.....)
 - 寄存器溢出:变量太多 → 存回内存
 - 对齐限制:字必须存在 4 的倍数地址
- 2.4 内存操作数:lw / sw(基址寻址:寄存器 + 偏移量)
 - 字节寻址、字地址相差 4
 - 大端 / 小端
- 2.5 立即数:addi(16 位,要符号扩展)
- 2.6 有符号与无符号数
 - 原码/反码/补码(补码 = 反码 + 1)
 - 符号扩展(sign extension)
 - slt / slti(有符号) vs sltu / sltiu(无符号)
- 2.7 机器语言:三种指令格式
 - R 型:op rs rt rd shamt funct(寄存器运算)
 - I 型:op rs rt immediate(访存/立即数/分支)
 - J 型:op address(跳转)
- 2.8 逻辑运算:and / or / nor / sll / srl / sra(掩码!)
- 2.9 分支与循环:beq / bne(PC 相对寻址)、case 跳转表
- 2.10 过程调用 ★
 - jal(跳转并保存返回地址)/ jr \$ra
 - 栈与栈帧(\$sp)
 - 寄存器约定:调用者保存 vs 被调用者保存
 - 叶子过程 vs 非叶子过程
- 2.11 五种寻址方式总结
- 2.12 并行与同步:ll / sc 原子交换
- 2.13 程序的一生:编译 → 汇编(符号表/目标文件)
 - 链接(重定位)→ 装载 → 动态链接库
- 2.14 32 位立即数:lui + ori;伪指令(move、li、la)
- 2.15 实战:C 排序程序 + 数组与指针
- 2.16 其他架构:ARMv7、x86(对比理解)
- 2.17 谬误与陷阱



二、学习目标

学完本章,你应该能:

1. 说出 MIPS 三条设计原则,并各举一个指令设计上的体现。
2. 默写 32 个寄存器的用途(\$t、\$s、\$a、\$v、\$sp、\$ra……)。
3. 会写:算术、访存、分支、循环、过程调用的 MIPS 汇编程序。
4. 会做:十进制 ↔ 二进制 ↔ 补码 ↔ 十六进制转换;符号扩展。
5. 会拆:给定机器码(十六进制)反汇编出汇编指令;反之把汇编翻译成机器码。
6. 理解 R/I/J 三种指令格式各字段含义,记住常用 opcode/funct 值。
7. 能解释:调用者保存 vs 被调用者保存寄存器;为什么需要栈。
8. 能描述:一个 C 程序从编译到装载的全过程。

三、核心名词先混个脸熟

名词	一句话
指令集架构(ISA)	软件与硬件之间的"合同"(指令、寄存器、编址)
助记符(mnemonic)	指令的可读名字,如 add、lw
寄存器(register)	CPU 内部的小而快的存储单元

名词	一句话
字(word)	MIPS 中为 32 位(4 字节)的数据单位
立即数(immediate)	直接写在指令里的常数
符号扩展(sign extension)	把短的有符号数补长,补符号位
补码(two's complement)	计算机中表示负数的标准方法
大小端(endianness)	多字节数据在内存中的字节排列顺序
寻址方式(addressing mode)	指令计算操作数地址的方式
程序计数器(PC)	指向下一条待执行指令的寄存器
栈(stack)	后进先出的内存区域,过程调用用它
栈指针(\$sp)	指向栈顶的寄存器
调用者保存(caller-saved)	\$t 寄存器:子程序可随意用,用了不负责恢复
被调用者保存(callee-saved)	\$s 寄存器:子程序用了必须恢复原值
伪指令(pseudoinstruction)	汇编器"扩展"的指令,硬件并不存在
目标文件(object file)	汇编/编译产物,含机器码+符号表
链接器(linker)	把多个目标文件拼成可执行文件
动态链接库(DLL)	运行时才链接进来的共享代码库

四、分节精讲

2.1 指令集架构与 MIPS

指令集架构(ISA, Instruction Set Architecture) 是硬件与底层软件之间的接口契约,规定了:哪些指令、哪些寄存器、内存如何编址、数据如何表示。MIPS 与 x86、ARM 一样,都是一种 ISA。

- MIPS 是 **RISC(精简指令集)** 的典范:指令数量少、格式规整、每条指令做一件小事。
- 对比 x86 是 **CISC(复杂指令集)**:指令多而复杂、长度不固定。
- 💡 大白话:ISA 是"合同",微架构(第 4 章)是"合同的具体履行方式"。同一份 MIPS 程序可以在任何遵守 MIPS 合同的 CPU 上跑,不管内部实现如何。

三条设计原则(全章乃至全书反复出现,必须背熟):

1. **简单源于规整(Simplicity favors regularity)** —— 所有算术指令都是"三操作数、全是寄存器"的固定格式 → 硬件实现简单。
2. **越小越快(Smaller is faster)** —— 寄存器只有 32 个,比几百万个内存单元少得多 → 访问寄存器比访问内存快得多(呼应八大思想第 2、3 条)。

3. **加速大概率事件(Make the common case fast)** —— 常数 0 太常用 → 专门设 \$zero;立即数常用 → 设 addi。

2.2 硬件操作:加法和减法

MIPS 算术指令的固定套路:**一条指令,一个运算,三个操作数,全是寄存器。**

```
add $t0, $s1, $s2    # $t0 = $s1 + $s2
sub $t0, $s1, $s2    # $t0 = $s1 - $s2
```

对应 C 语句 `a = b + c + d;` 需要两条指令(先加两个,再加第三个)。

💡 大白话:MIPS 的算术指令像"只有三个槽的机器":两个原料入口、一个成品出口。所以 `a = b + c + d` 这种三个数相加,必须分两次做: `a = (b + c) + d`。

⚠️ 易错点:MIPS 算术指令**不能直接操作内存里的数**,操作数必须先在寄存器里。要想对内存变量做加法,必须先用 lw 取进来(见 2.4)。

2.3 操作数:寄存器

32 个 32 位寄存器,每个都是 4 字节大小。寄存器比内存快得多,所以尽量让数据待在寄存器里(原则 2)。

MIPS 寄存器约定(必须背,考试和写代码都要用):

名字	编号	用途	保存责任
\$zero	0	常数 0(读它永远得 0)	无
\$at	1	汇编器保留(伪指令用)	汇编器
\$v0-\$v1	2-3	函数返回值	调用者保存
\$a0-\$a3	4-7	函数参数(第 1~4 个)	调用者保存
\$t0-\$t7	8-15	临时变量	调用者保存
\$s0-\$s7	16-23	保存变量(跨调用存活)	被调用者保存
\$t8-\$t9	24-25	临时变量	调用者保存
\$k0-\$k1	26-27	操作系统保留	OS
\$gp	28	全局指针	被调用者保存
\$sp	29	栈指针	被调用者保存
\$fp	30	帧指针	被调用者保存
\$ra	31	返回地址(jal 写入)	被调用者保存

记忆口诀:**s = saved(存钱要自己负责),t = temporary(用完就扔)。**

- **寄存器溢出(register spilling):**变量比寄存器还多时,编译器把"不常用"的变量暂存回内存,用到时再取。

- **对齐限制(alignment restriction):**MIPS 要求 `lw/sw` 的地址必须是 **4 的倍数**(字对齐)。地址不整除 4 → 运行报错。

2.4 内存操作数:lw 和 sw

寄存器只有 32 个,放不下大量数据 → 数据住内存,用时搬运到寄存器。MIPS 是"存取型"(load-store)架构:**内存只能通过 lw/sw 访问,算术指令不碰内存。**

```
lw $t0, 32($s3)    # $t0 = Memory[$s3 + 32](取字)
sw $t0, 48($s3)    # Memory[$s3 + 48] = $t0(存字)
```

- 这种"**寄存器 + 偏移量(常数)**"的形式叫**基址寻址(base addressing)**。`$s3` 是基地址,32 是偏移量(字节数)。
- 内存**按字节编址**,但每个字 4 字节 → 相邻字地址差 4。
- 对应 C 代码:设 `$s3` 是数组 A 的基址,则 `A[8]` 就是 `lw $t0, 32($s3)` (第 8 个元素,偏移 $8 \times 4 = 32$)。

💡 大白话:内存像一个大公寓楼,每个字节是一个房间,每 4 个房间一套公寓(一个字)。lw 是"从公寓取货",sw 是"把货放进公寓"。房间号(地址)必须从每套公寓的"开头"开始(对齐)。

大小端(endianness):一个字 4 字节,在内存里按什么顺序排?

- **大端(big-endian):**高位字节放低地址(人读数字的习惯:"高位数写左边")。
- **小端(little-endian):**低位字节放低地址(x86 就是这个)。
- ⚠ 易错点:两个字节序的机器之间传数据,字节序不同会读出"倒过来"的数。MIPS 可配置,教材默认大端。

2.5 立即数:addi

常数太常用,不可能每次都先 lw 到寄存器(原则 3)→ 直接把常数写进指令:

```
addi $s3, $s3, 4    # $s3 = $s3 + 4
```

- 立即数只有 **16 位**(且是**有符号数**,范围 $-32768 \sim +32767$),用时要**符号扩展到 32 位**。
- `$zero` 配合 addi 可以"造"常数: `addi $t0, $zero, 5` 就是 `$t0 = 5`。

2.6 有符号与无符号数 ★

2.6.1 二进制与十六进制

- 二进制每 4 位对应 1 个十六进制位: `0x8E6 = 1000 1110 0110`。
- 十六进制是机器码的"速记符号",考试必考转换。

2.6.2 负数的表示:原码 → 反码 → 补码

- **原码:**最高位是符号位(0 正 1 负),其余位是绝对值。
- **反码:**负数把绝对值按位取反。
- **补码(two's complement):**负数 = 反码 + 1(即:按位取反再加 1)。

- 计算机**只用补码**表示有符号数。为什么?补码让"加法和减法共用一套电路": $a - b = a + (-b)$, 符号位可以直接参与运算。
- 公式: 一个 n 位补码 x , 最高位权值是 $-2^{(n-1)}$ 。如 8 位 1111 1101 = $-128 + 64 + 32 + 16 + 8 + 4 + 1 = -3$ 。
- 8 位有符号范围: **-128 ~ +127**; 无符号范围: **0 ~ 255**。 n 位即 $-2^{(n-1)} \sim 2^{(n-1)} - 1$ 。
- 💡 大白话: 补码就像里程表倒拨: 从 0 往回拨 1 格, 显示的是 999...9。最高位的"1"既是符号又是数值, 天生就能用加法器做减法。

2.6.3 符号扩展(sign extension)

把短的有符号数变长(如 16 位立即数 $\rightarrow$ 32 位): **把符号位复制到所有新增的高位**。

- 例: 16 位的 $-3 = 0xFFFD$, 扩展成 32 位 = $0xFFFFFD$ (复制了 16 个 1)。数值不变!
- 正数扩展: 补 0。例: $0x0005 \rightarrow 0x00000005$ 。
- ⚠ 易错点: `lbu` (无符号取字节) 用**零扩展** (补 0); `lb` (有符号取字节) 用**符号扩展**。用错指令, 128~255 的字节会被读成负数。

2.6.4 有符号 vs 无符号指令

指令	含义	比较方式
<code>slt \$t0, \$s1, \$s2</code>	$\$s1 < \$s2 ? 1 : 0$	有符号 (补码比较)
<code>sltu \$t0, \$s1, \$s2</code>	同上	无符号 (按位值比较)
<code>slti / sltiu</code>	与立即数比较	有符号 / 无符号

⚠ 易错点(全书最经典陷阱之一): `1111 1111` 有符号读作 -1 , 无符号读作 255。所以**同一组位, 有符号比较和无符号比较结果可能相反**。数组下标这种"非负数"要用 `sltu` (见 2.9.3 边界检查)。

2.7 机器语言: 指令的表示 ★

汇编是给人看的, 机器只认 0/1。MIPS 全部指令统一 **32 位长**, 分三种格式(**原则 1: 简单源于规整**的极致体现):

R 型(寄存器型, 算术/逻辑运算用)

```
opcode(6)  rs(5)  rt(5)  rd(5)  shamt(5)  funct(6)
31-26      25-21  20-16  15-11  10-6      5-0
```

- `opcode = 0`, 具体运算由 `funct` 决定(所以 R 型有两个"操作码字段")。
- `rs` = 第一源寄存器, `rt` = 第二源寄存器, `rd` = 目的寄存器, `shamt` = 移位数。

I 型(立即数型, 访存/立即数/分支用)

```
opcode(6)  rs(5)  rt(5)  constant/address(16)
```

- `lw/sw/addi` 用 `rs+rt+立即数`; `beq/bne` 用 `rs+rt+偏移量`。

J 型(跳转型)

```
opcode(6)  address(26)
```

常用 opcode / funct 值(必须背):

指令	类型	opcode	funct
add	R	0	32
sub	R	0	34
and / or / nor	R	0	36 / 37 / 39
slt	R	0	42
sll / srl / sra	R	0	0 / 2 / 3
jr	R	0	8
j	J	2	—
jal	J	3	—
beq	I	4	—
bne	I	5	—
addi	I	8	—
andi / ori	I	12 / 13	—
slti	I	10	—
lw	I	35	—
sw	I	43	—

手工汇编示例(必会):

把 `add $t0, $s1, $s2` 翻译成机器码。

op=0, rs=\$s1=17, rt=\$s2=18, rd=\$t0=8, shamt=0, funct=32 二进制:000000 10001 10010 01000 00000 100000 十六进制:**0x02324020**

把 `lw $t0, 32($s3)` 翻译成机器码。

op=35, rs=\$s3=19, rt=\$t0=8, imm=32 二进制:100011 10011 01000 0000000000100000 十六进制:**0x8E680020**

反汇编 `0x01A95020` 。

二进制:000000 01101 01001 01010 00000 100000 op=0、funct=32 → add;rs=\$t5, rt=\$t1, rd=\$t2 结果:**add \$t2, \$t5, \$t1**

2.8 逻辑运算与移位

```
and $t0, $t1, $t2    # 按位与
or  $t0, $t1, $t2    # 按位或
nor $t0, $t1, $t2    # 按位或非(或之后取反)
sll $t0, $t1, 4      # 逻辑左移 4 位(低位补 0)
srl $t0, $t1, 4      # 逻辑右移 4 位(高位补 0)
sra $t0, $t1, 4      # 算术右移 4 位(高位补符号位)
andi/ori             # 立即数版本(注意:立即数按"零扩展"处理)
```

- **掩码(mask)**:用 and 把不关心的位清零。例:取最低字节: `andi $t0, $t1, 0xFF`。
- 用 ori 把某位置 1;用 nor + \$zero 可实现"非"。
- ⚠ 易错点: `sll` 左移 1 位 = $\times 2$ (无符号); `sra` 右移 1 位 = $\div 2$ (有符号,向负无穷取整); `srl` 右移对负数会变成正数(因为补 0)。**逻辑右移 vs 算术右移**是常考对比。

2.9 分支:程序的"岔路口"

2.9.1 beq / bne

```
beq $s1, $s2, L      # 若 $s1 == $s2,跳到 L
bne $s1, $s2, L      # 若 $s1 != $s2,跳到 L
```

- 只有"相等/不等"两种比较;其他比较(<、>)用 `slt + beq/bne` 组合实现(汇编器提供伪指令 `blt` 等)。
- **PC 相对寻址**:目标地址 = **(PC + 4) + 偏移量 $\times 4$** 。因为所有 MIPS 指令都是 4 字节,偏移量按"指令条数"存,可跳 $\pm 2^{15}$ 条指令范围。
- 💡 大白话:分支像"如果...就往前/往后翻几页书"。偏移量存的是"页数差",不是绝对页码,这样代码搬到任何位置都能跑。

2.9.2 循环:while / for

C 的 `while (A[i] == k) i++;` 翻译骨架:

```
Loop:  sll  $t1, $s3, 2      # $t1 = i × 4(字节偏移)
       add  $t1, $t1, $s6    # $t1 = A 的基址 + 偏移
       lw   $t0, 0($t1)     # 取 A[i]
       bne  $t0, $s5, Exit   # A[i] != k → 退出
       addi $s3, $s3, 1      # i++
       j    Loop            # 回去
Exit:  ...
```

⚠ 易错点:数组元素字节偏移 = 下标 $\times 4$,忘了乘 4 是最常见的新手错误。

2.9.3 边界检查(经典考点)

C 代码 `if (a < 0 || a >= length) error();` ——数组下标 `a` 是**无符号数**,必须用 `sltu`,否则负数(如 `0xFFFFFFFF`)会被当成"巨大正数"而漏检:

```
sltu $t0, $a0, $s1    # $t0 = (a < length) ? 1 : 0    ← 无符号!
beq  $t0, $zero, Error # 若 a >= length(无符号)→ 报错
```

✦ 考点提示:"为什么这里用 `sltu` 而不是 `slt`"几乎是必考题,答案:下标按无符号数对待,负数会变成极大正数,`slt` 会误判。

2.9.4 case/switch:跳转表

`case k` 语句:内存里放一张**跳转表(jump address table)**,每一项存一个分支代码的地址。程序用 `k` 算偏移 → `lw` 取出目标地址 → `jr` 跳过去。用一次 `lw + jr` 代替一长串 `beq` 判断(原则 3:加速大概率事件)。

2.10 过程调用 ★(本章最难点)

2.10.1 核心思想

过程(函数)调用要做六件事(书的顺序):

1. 调用者把**参数**放进 `$a0-$a3`;
2. 调用者执行 `jal`:把 **PC+4 存入 \$ra**,然后跳到被调过程;
3. 被调过程**开栈帧**: `addi $sp, $sp, -n`;
4. 被调过程执行时,若用到 `$s` 寄存器(或还要再调用别人),先把旧值**压栈保存**;
5. 被调过程把**返回值**放进 `$v0`;
6. 被调过程恢复现场、弹栈, `jr $ra` 返回。

2.10.2 栈(stack)与栈帧(stack frame)

- 栈:内存里一段**后进先出**的区域,**从高地址向低地址生长**。`$sp` 总指向栈顶。
- 压栈 = `$sp` 减,再 `sw`;弹栈 = `lw`,再 `$sp` 加。
- 栈帧:一次过程调用在栈上占的一小块空间,用来放返回地址、局部变量、被保存的寄存器。

💡 大白话:栈像一摞盘子。函数调用=放一个新盘子上去(`$sp` 往下移),返回=把盘子拿走(`$sp` 往上移)。最新放的盘子永远在最上面——所以嵌套调用和递归天然能用栈处理。

2.10.3 调用者保存 vs 被调用者保存 ★

- **调用者保存(caller-saved)**: `$t0-$t9`、`$a0-$a3`、`$v0-$v1`、`$ra`。规则:**调用者**若在乎它们的值,调用前必须自己保存。
- **被调用者保存(callee-saved)**: `$s0-$s7`、`$sp`、`$fp`、`$gp`。规则:**被调用者**若要用它们,必须先保存旧值,返回前恢复。
- 为什么分两类?如果所有寄存器都要保存,每次调用都太浪费;约定好"谁负责",就能省去不必要的保存(原则 3)。

- **?** 为什么 \$ra 算调用者保存?因为 jal 会覆盖 \$ra。调用者若之后还要用当前 \$ra(比如还会继续往下走),就得先存好。对 \$t 同理:调用者无法保证子程序不动 \$t,只能自己保护。

2.10.4 叶子过程 vs 非叶子过程

- **叶子过程(leaf procedure):**不调用任何其他过程 → 不用保存 \$ra 和 \$s,最省事。
- **非叶子过程:**会调用别人 → **必须**保存 \$ra(否则 jal 会覆盖它)以及用到的 \$s。

非叶子过程标准模板(务必背下来,考试默写):

```
proc:
    addi $sp, $sp, -8      # 开 8 字节栈帧
    sw   $ra, 4($sp)      # 保存返回地址
    sw   $s0, 0($sp)      # 保存将用到的 $s0
    ...                  # 过程体(可随意调用其他过程)
    lw   $s0, 0($sp)      # 恢复 $s0
    lw   $ra, 4($sp)      # 恢复返回地址
    addi $sp, $sp, 8      # 弹栈
    jr   $ra              # 返回
```

完整例子——阶乘递归过程(理解递归+栈的绝佳案例):

```
fact:                                # int fact(int n) 参数在 $a0
    addi $sp, $sp, -8
    sw   $ra, 4($sp)
    sw   $a0, 0($sp)
    slti $t0, $a0, 1          # n < 1 ?
    beq  $t0, $zero, L1       # 否 → 递归
    addi $v0, $zero, 1        # 是 → 返回 1
    addi $sp, $sp, 8
    jr   $ra
L1:
    addi $a0, $a0, -1         # n-1
    jal  fact                 # 递归调用 fact(n-1)
    lw   $a0, 0($sp)          # 取回 n
    lw   $ra, 4($sp)
    addi $sp, $sp, 8
    mul  $v0, $a0, $v0        # $v0 = n × fact(n-1)
    jr   $ra
```

2.11 五种寻址方式总结 ★(常考列表题)

寻址方式	格式	使用指令
1. 立即数寻址(immediate)	操作数就在指令里	addi、andi、ori
2. 寄存器寻址(register)	操作数在寄存器	add、sub、and 等 R 型
3. 基址寻址(base)	内存地址 = 寄存器 + 偏移量	lw、sw
4. PC 相对寻址(PC-relative)	地址 = PC+4 + 偏移×4	beq、bne

寻址方式	格式	使用指令
5. 伪直接寻址(pseudodirect)	地址 = (PC+4 高 4 位) (26 位地址×4)	j、jal

💡 大白话:指令里放不下 32 位完整地址(指令总共才 32 位),所以:分支用"相对跳"(离我多远),跳转用"拼接"(高 4 位借用 PC,低 26 位×4)。

2.12 并行与同步:原子交换

多处理器时代(第 6 章),两个核同时改一个内存变量会冲突。解决:硬件提供**原子操作**——ll (load linked,关联加载)+ sc (store conditional,条件存储):

```
try: add $t0, $zero, $s4    # $t0 = 新值
      ll  $t1, 0($s1)        # 关联读取旧值
      sc  $t0, 0($s1)        # 尝试存回;成功 $t0=1,失败 $t0=0
      beq $t0, $zero, try    # 失败(有人抢先改过)→ 重试
```

- sc 的语义:自 ll 之后若该地址没被别人改过,写入成功并返回 1;否则不写,返回 0。
- 这是"**原子交换(atomic exchange)**"的教科书实现,也是第 6 章锁(lock)的硬件基础。

2.13 程序的一生:编译 → 汇编 → 链接 → 装载 ★

C源文件 → [编译器] → 汇编文件 → [汇编器] → 目标文件 → [链接器] → 可执行文件 → [装载器] → 内存运行

1. **编译(compile)**:C → 汇编(中间也可能先优化)。
2. **汇编(assembly)**:
 - 把助记符翻译成机器码;把**伪指令**展开成真实指令;
 - 产生**目标文件(object file)**:机器码 + 数据 + **符号表(symbol table,记录"名字→地址"的标签表)** + **重定位信息**。
3. **链接(link)**:把多个目标文件(你的代码 + 库函数代码)拼成一个可执行文件,解决跨文件的符号引用(**重定位 relocation**:给标签填上最终地址)。
4. **装载(load)**:操作系统把可执行文件读进内存,设置 PC,开跑。

内存布局(可执行文件装载后):

```
高地址 0x7FFF FFFC: 栈(向下生长)
      ↓
      (动态数据/堆,向上生长)
      静态数据(全局变量)
      正文(text,机器指令,从 0x0040 0000 起)
低地址 0x0000 0000: 保留区
```

- **动态链接库(DLL)**:不在链接时并入,而在**程序运行时才链接**进来的库(如 Windows 的 .dll)。优点:多个程序共享一份代码,节省内存;库更新不用重链接程序。

2.14 32 位立即数与伪指令

- 16 位立即数不够用(如把 32 位地址放进寄存器):**lui(load upper immediate)+ ori** 两步:

```
lui $t0, 0x1001      # $t0 高 16 位 = 0x1001, 低 16 位清零
ori $t0, $t0, 0x0008 # $t0 低 16 位 = 0x0008 → $t0 = 0x10010008
```

- **伪指令(pseudoinstruction)**:汇编器提供的"方便指令",硬件并不存在,汇编时被展开。常见: `move $t0,$t1` → `add $t0,$zero,$t1`; `li $t0,5` → `addi`; `la $t0,label` → `lui+ori`; `blt` → `slt+bne`。
- ⚠ 易错点:伪指令会**占用 \$at** 寄存器(所以 \$at 是"汇编器保留"),考试常问"move 是真指令还是伪指令"。

2.15 实战:C 排序程序与数组指针

用"数组下标"写(把 i 存在 \$s0,\$s1 这类寄存器,偏移用 i×4):

```

# void sort(int v[], int n); $a0=v 基址, $a1=n
sort:
    addi $sp, $sp, -20      # 非叶子:保存现场
    sw   $ra, 16($sp)
    sw   $s3, 12($sp)      # i
    sw   $s2, 8($sp)       # j
    sw   $s1, 4($sp)       # v(基址)
    sw   $s0, 0($sp)       # n
    add  $s1, $a0, $zero    # 存参数
    add  $s0, $a1, $zero

for1:
    slt  $t0, $s3, $s0     # i < n ?
    beq  $t0, $zero, exit1
    addi $s2, $s3, -1      # j = i - 1
for2:
    slti $t0, $s2, 0       # j >= 0 ?
    bne  $t0, $zero, exit2
    sll  $t1, $s2, 2       # j×4
    add  $t2, $s1, $t1     # &v[j]
    lw   $t3, 0($t2)       # v[j]
    lw   $t4, 4($t2)       # v[j+1]
    slt  $t0, $t4, $t3     # v[j+1] < v[j] ?
    beq  $t0, $zero, exit2
    sw   $t4, 0($t2)       # 交换
    sw   $t3, 4($t2)
    addi $s2, $s2, -1      # j--
    j    for2
exit2:
    addi $s3, $s3, 1       # i++
    j    for1
exit1:
    lw   $s0, 0($sp)       # 恢复现场
    lw   $s1, 4($sp)
    lw   $s2, 8($sp)
    lw   $s3, 12($sp)
    lw   $ra, 16($sp)
    addi $sp, $sp, 20
    jr   $ra

```

"数组 vs 指针"对比:用指针版本可以省掉每次 `sll; add` 算地址(直接对地址自增 4),指令数更少。结论:现代编译器会做这类优化,所以**用高级语言写清晰代码,让编译器去优化**才是正道。

✦ 考点提示:会默写"保存/恢复现场"的完整骨架 + 能解释为什么 sort 必须保存 \$s0-\$s3 和 \$ra。

2.16 其他架构:ARMv7 与 x86(了解即可)

- **ARMv7:**与 MIPS 同为 RISC,差异举例:① 支持**条件执行**(几乎每条指令都能带"如果...才执行"的条件,省掉小分支);② 没有专门的除法指令(软件实现)。手机/嵌入式霸主。
- **x86:**CISC 代表。指令长度 1~17 字节可变;允许"内存到内存"运算(不需要 load-store);能兼容 40 年前的 8086 代码(这是历史包袱也是生态优势)。x86 内部把复杂指令拆成类 RISC 微操作(第 4 章),所以性能上两种哲学殊途同归。

2.17 谬误与陷阱(考试简答题素材)

- 1. **谬误:指令越强大性能越高。** x86 的"强大"指令并不多见于真实程序,还拖累流水线;RISC 简单规整反而高效。
- 2. **谬误:用汇编语言编程能获得最高性能。** 现代编译器生成代码常优于手写汇编,且可移植、可维护。真正的性能在于算法和数据结构。
- 3. **陷阱:字节序不同导致数据交换出错**(网络传数据、读二进制文件)。
- 4. **陷阱:伪指令不是硬件指令**,真正的指令集里没有 move、li、blt。
- 5. **陷阱:立即数有符号:** addi \$t0, \$t1, -1 合法;但 ori \$t0, \$t1, 0xFFFF 里的立即数是按**零扩展**的,别指望它是"负一"。
- 6. **陷阱:忘了 lw/sw 的偏移量是字节数**(数组下标要 ×4)。

五、MIPS 指令速查表(全书核心,建议打印)

类别	指令	功能	格式
算术	add rd, rs, rt	rd = rs + rt(有符号,溢出报错)	R
算术	sub rd, rs, rt	rd = rs - rt	R
算术	addi rt, rs, imm	rt = rs + imm(符号扩展)	I
算术	addiu rt, rs, imm	无符号版,不报溢出	I
算术	addu / subu	不检测溢出版	R
算术	mul rd, rs, rt	低 32 位乘法	R
算术	mult rs, rt	积存到 Hi/Lo 寄存器	R
算术	div rs, rt	商入 Lo,余数入 Hi	R
算术	mfhi / mflo	从 Hi / Lo 取数到寄存器	R
逻辑	and / or / nor	按位与/或/或非	R
逻辑	andi rt, rs, imm	按位与立即数(零扩展)	I
逻辑	ori rt, rs, imm	按位或立即数(零扩展)	I
移位	sll / srl / sra	左移 / 逻辑右移 / 算术右移	R
访存	lw rt, off(rs)	取字	I
访存	sw rt, off(rs)	存字	I
访存	lb / lbu / sb	字节存取(有符号/无符号取)	I
访存	lh / lhu / sh	半字存取	I

类别	指令	功能	格式
访存	lui rt, imm	高 16 位装载,低 16 位清零	I
比较	slt / sltu / slti / sltiu	小于则置 1(有符号/无符号)	R/I
分支	beq rs, rt, label	相等则跳(PC 相对)	I
分支	bne rs, rt, label	不等则跳	I
跳转	j label	无条件跳(伪直接)	J
跳转	jal label	跳转并把 PC+4 存入 \$ra	J
跳转	jr rs	跳到寄存器地址(返回用 jr \$ra)	R
同步	ll / sc	关联加载 / 条件存储(原子操作)	I
常用伪指令	move、li、la、blt、bgt、ble、bge	汇编器展开,硬件不存在	—

六、名词解释汇总表

名词	英文	解释
指令集架构	ISA	软硬件接口契约:指令、寄存器、编址、数据表示
助记符	mnemonic	指令的可读名字,如 add、lw
寄存器	register	CPU 内 32 个 32 位存储单元
寄存器溢出	register spilling	变量多于寄存器时暂存回内存
字	word	32 位(4 字节)数据单位
对齐限制	alignment restriction	字地址必须是 4 的倍数
基址寻址	base addressing	内存地址 = 寄存器 + 偏移量
立即数	immediate	嵌在指令里的 16 位常数
补码	two's complement	负数表示法:反码 + 1
符号扩展	sign extension	补符号位把短数变长
大端/小端	big/little endian	字节在内存中的排列顺序
掩码	mask	用与运算选择性清零的位模式
程序计数器	PC	存放下一条指令地址
分支延迟槽	branch delay slot	分支指令后紧跟的指令(老 MIPS 特性,了解)
跳转表	jump address table	存分支目标地址的表,实现 switch

名词	英文	解释
栈	stack	后进先出内存区,过程调用用
栈帧	stack frame	一次调用在栈上占用的区域
调用者保存	caller-saved	\$t、\$a、\$v、\$ra:调用者负责保护
被调用者保存	callee-saved	\$s、\$sp、\$fp:被调用者负责保护
叶子过程	leaf procedure	不调用其他过程的过程
伪指令	pseudoinstruction	汇编器扩展指令,硬件不存在
符号表	symbol table	目标文件中"标签→地址"表
重定位	relocation	链接时为符号填入最终地址
动态链接库	DLL	运行时才链接的共享库

七、易混淆概念对比

对比	区别一句话
add vs addu	前者溢出即报错,后者不检查
addi vs addiu	立即数都符号扩展;区别在溢出检测
slt vs sltu	有符号比较 vs 无符号比较
lb vs lbu	符号扩展 vs 零扩展
srl vs sra	逻辑右移补 0 vs 算术右移补符号位(负数÷2 用 sra)
j vs jal	后者会保存返回地址到 \$ra
jr \$ra vs j	前者目标在寄存器(动态返回),后者目标固定
beq 偏移量 vs lw 偏移量	前者单位是"指令条数"×4;后者单位是字节
\$t vs \$s	临时(调用者保存)vs 长期保存(被调用者保存)
汇编器 vs 编译器	处理汇编语言 vs 处理高级语言
链接器 vs 装载器	生成可执行文件 vs 把程序读入内存运行
真指令 vs 伪指令	硬件实现 vs 汇编器展开
大端 vs 小端	高位字节在低地址 vs 低位字节在低地址

八、自测题

1. (写代码)用 MIPS 实现 C 语句 $f = (g + h) - (i + j)$; ($f \rightarrow \$s0, g \rightarrow \$s1, h \rightarrow \$s2, i \rightarrow \$s3, j \rightarrow \$s4$).
2. (写代码)设数组 A 基址在 $\$s3$, 变量 h 在 $\$s2$, 实现 $A[12] = h + A[8]$;
3. (机器码)把 `add $t0, $s1, $s2` 翻译成十六进制机器码。
4. (机器码)把 `lw $t0, 32($s3)` 翻译成十六进制机器码。
5. (反汇编)机器码 `0x01A95020` 是什么指令?
6. (数制)-13 的 8 位补码是多少?16 位补码 `0xFFFF` 符号扩展成 32 位是多少?
7. (分支)beq 指令在地址 `0x00400010`, 目标是 `0x00400020`, 求 16 位偏移量字段的值。
8. (循环)写出"计算数组前 n 项和"的 MIPS 循环(基址 $\$a0$, n 在 $\$a1$, 结果放 $\$v0$)。
9. (简答)为什么 sort 过程必须保存 $\$s0-\$s3$? 它要不要保存 $\$t0$? 为什么?
10. (简答)什么是伪指令? 举两个例子。伪指令由谁展开?
11. (简答)调用者保存与被调用者保存寄存器各有哪些? 为什么要这样分工?
12. (简答)一个 C 程序从源码到运行, 经过哪些步骤? 各产生什么?

参考答案

1. `add $t0, $s1, $s2 / add $t1, $s3, $s4 / sub $s0, $t0, $t1`
2. `lw $t0, 32($s3) / add $t0, $s2, $t0 / sw $t0, 48($s3)`
3. `0x02324020`(拆法见 2.7 节)
4. `0x8E680020`
5. $op=0$ 、 $funct=32 \rightarrow add; rs=13(\$t5), rt=9(\$t1), rd=10(\$t2) \rightarrow add \$t2, \$t5, \$t1$
6. $-13 = 1111\ 0011$ ($13 = 0000\ 1101$, 取反 $1111\ 0010$, 加 1); `0xFFFF` $\rightarrow$ `0xFFFFFFFF`
7. 目标 - (PC+4) = `0x20 - 0x14 = 0xC = 12` 字节 = 3 条指令 $\rightarrow$ 偏移量 = 3
- 8.

```
add $v0, $zero, $zero    # sum = 0
add $t0, $zero, $zero    # i = 0
Loop:
    slt $t1, $t0, $a1      # i < n ?
    beq $t1, $zero, Exit
    sll $t2, $t0, 2        # i*4
    add $t2, $a0, $t2      # &A[i]
    lw  $t3, 0($t2)
    add $v0, $v0, $t3
    addi $t0, $t0, 1
    j   Loop
Exit:
```

9. $\$s$ 是被调用者保存寄存器: sort 用了 $\$s0-\$s3$, 按约定必须先保存旧值、返回前恢复, 否则调用者的数据被破坏。 $\$t0$ 是临时寄存器, 调用者保存——sort 不需要保存它(反正调用者不指望 $\$t0$ 跨调用存活)。
10. 伪指令是汇编器提供的"方便指令", 硬件并不直接实现, 由**汇编器**在汇编时展开成一条或多条真实指令。
例: `move $t0, $t1` $\rightarrow$ `add $t0, $zero, $t1; li $t0, 5` $\rightarrow$ `addi $t0, $zero, 5`。

11. 调用者保存:\$t0-\$t9、\$a0-\$a3、\$v0-\$v1、\$ra;被调用者保存:\$s0-\$s7、\$sp、\$fp、\$gp。分工原因:避免每次调用都保存全部寄存器(加速大概率事件)——临时值由"在乎它的人"(调用者)保护,长期值由"使用它的人"(被调用者)保护。
12. 编译(C→汇编)→ 汇编(→目标文件,含机器码+符号表,展开伪指令)→ 链接(多目标文件+库 → 可执行文件,完成重定位)→ 装载(OS 读入内存,设 PC,开始执行)。运行期还可能有动态链接。
-

九、本章小结

- MIPS 的哲学:**简单规整**(固定 32 位、三种格式)、**越小越快**(寄存器)、**加速大概率事件**(立即数、\$zero)。
- 一条主线:**C 语句** → **寄存器/内存操作** → **汇编指令** → **32 位机器码**,来回都要会。
- 一块硬骨头:**过程调用**(栈 + jal/jr + 保存约定)——把模板默写 10 遍就通了。
- 一个习惯:装 **MARS/SPIM 模拟器**把每道题的代码跑一遍,这章就活了。

下一章 → [03-第3章-计算机的算术运算.md](#):计算机怎么实现乘除法和浮点数。

第 3 章 计算机的算术运算

一句话导读:计算机里没有"手算",只有加法器。这一章告诉你:减法、乘法、除法、小数(浮点数)如何全部归结为"移位 + 加法",以及浮点误差从哪来。

对应教材:第 3 章「计算机的算术运算(Arithmetic for Computers)」

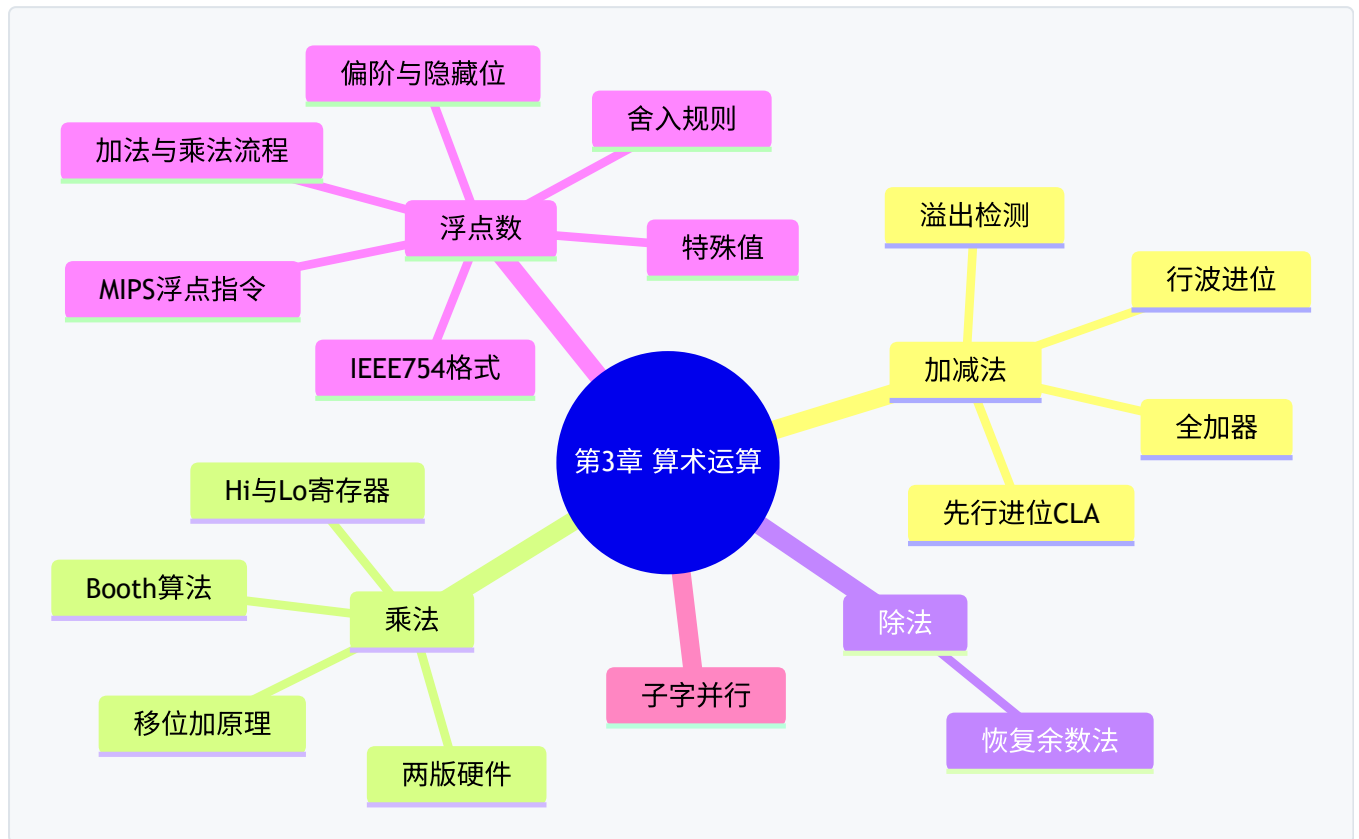
一、本章思维导图

第3章 计算机的算术运算

- 3.1 加法
 - 半加器(两位相加,不管进位)/ 全加器(带上进位)
 - 行波进位加法器(ripple carry):慢在进位"串行传递"
 - 先行进位(CLA):并行算进位,快
 - 减法 = 加补码($a - b = a + (-b)$)
- 3.2 溢出(overflow)
 - 定义:结果超出表示范围
 - 判定:正+正→负,负+负→正(看符号!)
 - addu/subu 不检测;add/sub 检测
- 3.3 乘法
 - 手算原理:逐位"移位+加"
 - 第一版硬件:64位ALU、被乘数左移
 - 优化版:32位ALU、积与乘数右移(省一半硬件)
 - 有符号乘法:Booth 算法(把连续 1 打包)
 - MIPS:mult → Hi/Lo;mfhi/mflo;mul(取低32位)
- 3.4 除法
 - 恢复余数法:减了不行再加回来
 - 硬件:除数右移、余数左移
 - 有符号:商符号 = 两符号异或;余数随被除数
 - MIPS:div → Lo=商、Hi=余数;除以零是陷阱
- 3.5 浮点数(IEEE 754)★本章核心
 - 格式: $(-1)^S \times (1.Frac) \times 2^{(Exp-Bias)}$
 - 单精度:S(1)Exp(8)Frac(23),Bias=127
 - 双精度:S(1)Exp(11)Frac(52),Bias=1023
 - 隐藏位:小数点前的 1 不存,白赚一位精度
 - 特殊值:±0、±∞、NaN、非规格化数
 - 溢出/下溢
- 3.6 浮点运算
 - 加法四步:对齐指数→有效数相加→规格化→舍入
 - 乘法三步:指数相加→有效数相乘→规格化+舍入
 - 舍入:保护位/舍入位/粘贴位;就近舍入到偶数
 - 精度陷阱:加法不满足结合律!
- 3.7 MIPS 浮点指令
 - 寄存器:\$f0-\$f31(双精度用偶数对)
 - lwc1/swc1、add.s/add.d、c.eq.s/bc1t
 - 例子:浮点矩阵乘法
- 3.8 子字并行(SIMD within a register)
 - 一个宽寄存器装多个小数,同时算
- 3.9 谬误与陷阱
 - 左移 ≠ 有符号乘(溢出时)

- └ 浮点结合律不成立
- └ 用浮点数做精确计算(如钱)会出错

Mermaid 版(可渲染)



二、学习目标

1. 理解全加器、行波进位、先行进位,会算加法器的"门延迟"。
2. 会判定补码加减是否溢出(三种方法至少会一种)。
3. 能手算 MIPS 乘法器(优化版)的逐步过程(移位+加)。
4. 理解恢复余数除法。
5. **会做十进制 ↔ IEEE 754 单/双精度互转(必考)。**
6. 会说清偏阶、隐藏位、非规格化数、NaN 的含义。
7. 能手算浮点加法(对齐、相加、规格化、舍入)。
8. 理解舍入规则和"浮点加法不满足结合律"。
9. 会写简单的 MIPS 浮点程序。
10. 知道什么是子字并行。

三、核心名词先混个脸熟

名词	一句话
全加器(full adder)	三位相加(两个输入+进位入),输出和+进位出
行波进位(ripple carry)	进位像波浪一样逐位传,慢
先行进位(CLA)	进位并行计算,快
溢出(overflow)	结果超出表示范围,符号错乱
被乘数/乘数/积	乘法三要素(multiplicand/multiplier/product)
Hi / Lo	存放乘除法结果的专用寄存器
Booth 算法	把连续 1 打包,减少加法次数的乘法优化
恢复余数法(restoring)	除法:先减,不够再加回来
IEEE 754	浮点数的国际标准格式
偏阶(bias)	指数存的是"指数+偏移量",保证全正
隐藏位(hidden bit)	规格化数小数点前的 1,不存储
规格化数(normalized)	形式为 $1.xxx \times 2^e$ 的数
非规格化数(denormalized)	指数全 0 时表示的超小数
NaN	Not a Number,如 0/0 的结果
保护位/舍入位/粘贴位	舍入用的三个额外精度位
就近舍入到偶数	round to nearest even,IEEE 默认舍入方式
子字并行	一个宽寄存器同时算多个窄数据

四、分节精讲

3.1 加法:一切运算的基石

半加器(half adder):两个 1 位数相加,输出"和"与"进位",不考虑来自低位的进位。 **全加器(full adder)**:输入两个数位 + 低位进位(CarryIn),输出和 + 向高位的进位(CarryOut)。 **n 位加法 = n 个全加器串联**。

行波进位加法器(ripple carry adder):第 i 位的 CarryOut 作为第 i+1 位的 CarryIn,进位一位一位"波"过去。每位 2 级门延迟 $\rightarrow$ n 位要 **2n 级延迟**,n 越大越慢。

先行进位加法器(CLA, carry lookahead):把进位展开成公式,所有位的进位**并行**算出 $\rightarrow$ 延迟约 **$O(\log n)$** (或用 2 级逻辑的"两级逻辑"看:延迟与位数几乎无关)。代价:电路更复杂。

💡 大白话:行波进位像"击鼓传花"——第 100 位的进位要等前 99 位传完;先行进位像"群发消息"——每个进位同时算出来。快,但要用更多电路(原则:用空间换时间)。

3.2 减法与溢出 ★

减法 = 加补码: $a - b = a + (\sim b + 1)$, 硬件只要加法器 + 求反电路, 不用单独减法器。

溢出(overflow): 结果超出 n 位补码的表示范围(如 8 位: $-128 \sim +127$)。判定方法(会一种即可):

- 1. **符号法(最直观):** 正 + 正 = 负, 或 负 + 负 = 正 → 溢出。(一正一负相加**永不会溢出**)
- 2. **进位法:** 最高数值位的进位 $\neq$ 符号位的进位 → 溢出。
- 3. 公式: $\text{溢出} = (\sim a.s \ \& \ \sim b.s \ \& \ r.s) \mid (a.s \ \& \ b.s \ \& \ \sim r.s)$ (a.s、b.s、r.s 为符号位)。

例(4 位): $0111 + 0001 = 1000 \rightarrow 7+1 = -8?$ 错! 正+正得负 → **溢出**。 $1111 + 1111 = 1110 \rightarrow -1 + -1 = -2 \rightarrow$ 负+负得负, 正常, **无溢出**。

- MIPS: add / sub / addi 溢出即报错; addu / subu / addiu **不检测**(编译器常用 addu 处理 C 的无符号数)。
- **?** 为什么需要"不检测溢出"的版本? 因为有些语言的语义规定无符号运算"回绕"(溢出取模), 如 C 的 unsigned 加法——这时溢出是特性不是错误。

3.3 乘法: 移位 + 加

手算原理: 十进制里 $123 \times 456 = 123 \times 6 + 123 \times 50 + 123 \times 400$; 二进制更简单——乘数每位只有 0 或 1, 所以 **乘法 = 一连串"移位 + 条件加法"**。

第一版硬件(直观但浪费): 64 位被乘数寄存器(左移) + 64 位 ALU + 32 位乘数寄存器 + 64 位积寄存器。每步: 看乘数最低位, 是 1 就"积 += 被乘数"; 然后被乘数左移、乘数右移。

优化版(教材例子, 考试手算用这个):

- 用 32 位 ALU; **被乘数不动; 积和乘数共用 64 位寄存器向右移**(低 32 位放乘数, 高 32 位做累加)。
- 每一步: 看乘数最低位(积寄存器的第 0 位) → 是 1 就"积的高 32 位 += 被乘数" → 整个 64 位积寄存器**右移一位**。
- 乘法结果 32×32 位最多 64 位, 所以需要 Hi/Lo 一对寄存器存。

教科书标准手算示例(0010 × 0011, 被乘数 0010, 乘数 0011):

迭代	步骤	积寄存器(高32位+乘数)
0	初值	0000 0011
1	最低位=1 → 加被乘数	0010 0011
1	右移	0001 0001
2	最低位=1 → 加被乘数	0011 0001
2	右移	0001 1000
3	最低位=0 → 不加	0001 1000
3	右移	0000 1100
4	最低位=0 → 不加	0000 1100
4	右移	0000 0110 = 6 ✓

有符号乘法:Booth 算法(旁注,知道思想即可):把乘数里"连续的 1"(如 $011110 = 100000 - 000010$)打包成"一次加、一次减",减少加法次数,速度与乘数中 1 的个数无关。

MIPS 乘法指令:

```
mult $s2, $s3    # Hi/Lo = $s2 × $s3(64位,有符号)
mfhi $t0         # $t0 = 积的高32位
mflo $t1         # $t1 = 积的低32位
mul  $t2, $s2, $s3 # $t2 = 低32位(编译器常用,不管高32位)
```

⚠ 易错点:乘法结果**可能超 32 位**,只取 mflo 会得到"截断"的结果。C 里两个 int 相乘也是取低 32 位。

3.4 除法:恢复余数法

术语:被除数(dividend)÷ 除数(divisor)= 商(quotient)...余数(remainder)。

恢复余数法(restoring division)——人类竖式的硬件版:

1. 余数寄存器左移,把被除数"喂"进来;
2. 余数 -= 除数;
3. 若余数为负:说明减多了 → **加回去(恢复,restore)**,商位记 0;
4. 若余数为正(或零):商位记 1;
5. 除数右移(或余数左移),重复 n 次。

有符号除法规则:商的符号 = 被除数符号 ⊕ 除数符号(异或);**余数的符号与被除数相同。**

MIPS 除法指令:

```
div $s2, $s3    # Lo = 商, Hi = 余数
mflo $t0        # 取商
mfhi $t1        # 取余数
```

⚠ 陷阱:**除以零**——有的机器会报错,有的(如部分 MIPS 实现)静默返回 0。总之别在程序里除以零。

💡 大白话:除法的本质是"反复试探性减法,减多了就反悔加回来"。所以除法是四种运算里最慢的——这也是编译器见到"除以 2 的幂"就用右移(sra)替换的原因(快得多)。

3.5 浮点数:IEEE 754 ★本章核心

3.5.1 科学计数法思想

十进制科学计数法: $-1.5 \times 10^{-3} = \text{符号} \times \text{有效数} \times 10^{\text{指数}}$ 。浮点数就是**二进制的科学计数法**。

3.5.2 格式

```
单精度(32位):S(1) | Exp(8) | Frac(23),Bias = 127
双精度(64位):S(1) | Exp(11) | Frac(52),Bias = 1023

值 =  $(-1)^S \times (1.\text{Frac}) \times 2^{(\text{Exp} - \text{Bias})}$     ← 规格化数
```

- **偏阶(bias,移码)**:指数存 $\text{Exp} = \text{真指数} + \text{Bias}$,好处:① 存的全是正数,好排序;② 用无符号比较就能比浮点数大小。
- **隐藏位(hidden bit)**:规格化数的有效数一定是 1.xxx,那个"1."人人都有 → **干脆不存**,白赚 1 位精度(单精度实际精度 24 位,双精度 53 位)。
- **范围(单精度)**:Exp 取 1~254,最大约 $\pm 3.4 \times 10^{38}$,最小规格化数约 $\pm 1.18 \times 10^{-38}$ 。
- **溢出(overflow)**:指数太大放不下(如 $1e38 \times 1e38$);**下溢(underflow)**:指数太小,落入非规格化数或 0。

3.5.3 特殊值(必考表)

Exp	Frac	含义
全 0	0	± 0 (注意 0 有正负!)
全 0	$\neq 0$	非规格化数(denormalized) :值 = $(-1)^S \times (0.\text{Frac}) \times 2^{(1-\text{Bias})}$,填在 0 和最小规格化数之间,"渐进下溢"
全 1	0	$\pm \infty$ (如 1.0/0.0)
全 1	$\neq 0$	NaN (Not a Number,如 0.0/0.0、 $\infty - \infty$)
其他	任意	规格化数(正常数)

💡 大白话:非规格化数是 IEEE 754 的"慈悲设计"——没有它,0 和最小规格化数(1.18×10^{-38})之间就是一片空白,更小的数直接被"四舍五入"成 0,误差巨大。有它,小数值"慢慢衰减"到 0,精度平缓。

3.5.4 十进制 ↔ 单精度互转(必会)

例:把 -0.75 转成单精度。

1. 符号: $S = 1$ 。
2. $0.75 = 0.11_2 = 1.1 \times 2^{-1} \rightarrow$ 真指数 = -1。
3. $\text{Exp} = -1 + 127 = 126 = 0111\ 1110$ 。
4. Frac = 隐藏位后面的部分:1.1 的"1"被隐藏,剩 .1 $\rightarrow \text{Frac} = 1000\dots 0$ (23 位)。
5. 拼起来:**1 01111110 1000000000000000000000 = 0xBF400000**。

例:反方向,0x3F800000 是什么?

1. 拆: $S=0, \text{Exp}=0111\ 1111=127, \text{Frac}=0$ 。
2. 真指数 = $127 - 127 = 0$;有效数 = 1.0。
3. 值 = $+1.0 \times 2^0 = 1.0$ 。

3.6 浮点运算

3.6.1 浮点加法四步(手算流程,必考)

1. **对齐指数**:把指数小的数对齐到大的(小数的有效数**右移**,差几位移几位)。
2. **有效数相加**。

3. **规格化**:结果左移/右移使形式回到 1.xxx(可能溢出→特殊处理)。
4. **舍入**:多余位按规则舍去(舍入后可能需要再规格化一次)。

手算例:0.5 + (-0.4375)

1. $0.5 = 1.0 \times 2^{-1}$; $-0.4375 = -(7/16) = -0.0111_2 = -1.11 \times 2^{-2}$ 。
2. 对齐: $-1.11 \times 2^{-2} = -0.111 \times 2^{-1}$ (右移一位)。
3. 相加: $1.000 \times 2^{-1} + (-0.111 \times 2^{-1}) = 0.001 \times 2^{-1}$ 。
4. 规格化: $0.001 \times 2^{-1} = 1.0 \times 2^{-4} = 1/16 = 0.0625 \checkmark$ 。

3.6.2 浮点乘法

1. 指数**相加**(注意:两个 Exp 都含 Bias,所以 $\text{Exp} = \text{Exp}_1 + \text{Exp}_2 - \text{Bias}$,要减掉一份);
2. 有效数相乘;
3. 规格化 + 舍入;符号 = 两符号异或。

3.6.3 舍入:保护位、舍入位、粘贴位 ★

运算中间结果比最终格式多出几位,最低的几位依次叫:

- **保护位(guard)**:第一位附加位;
- **舍入位(round)**:第二位附加位;
- **粘贴位(sticky)**:其余所有附加位的"或"(只要后面有任何一个 1, sticky=1)。

IEEE 默认**就近舍入到偶数(round to nearest even)**:超过一半则进,不到一半则舍,恰好一半时向最低位为偶数的方向舍(避免统计上的累积偏差)。

💡 大白话:为什么"一半取偶"?因为每次都"四舍五入"(一半总是进位)会让大量计算的结果系统性地偏大;"取偶"让一半的情况进、一半的情况舍,平均误差趋近于零。

3.6.4 精度陷阱

- **浮点加法不满足结合律**: $(a+b)+c \neq a+(b+c)$ 。例:一个大数加很多极小数,小数可能被"吞掉"。
- 浮点表示是**近似的**:0.1 在二进制里是无限循环小数,存的是近似值 $\rightarrow 0.1 + 0.2 \neq 0.3$ 。
- **?** 为什么金融系统不用浮点数算钱?因为十进制的小数(0.1 元)在二进制中无法精确表示,多次运算误差累积,钱会"凭空消失"。银行用定点十进制或专用库。

3.7 MIPS 浮点指令

- 寄存器:**\$f0-\$f31**(32 个 32 位);双精度需**偶数编号对**(\$f0/\$f1 一起用,\$f1 不能单独用)。
- 访存: `lwc1 $f0, 0($s1)` (取单精度)、`swc1` (存);双精度用 `ldc1/sdc1`。
- 运算: `add.s` / `sub.s` / `mul.s` / `div.s` (单精度)、`add.d` 等(双精度,后缀 .d)。
- 比较与分支:浮点比较不能直接用 `beq`!要 `c.eq.s` (置条件标志) $\rightarrow$ `bc1t` (标志为真则跳)/ `bc1f`。

```
lwc1 $f4, 0($s1)    # F = A
lwc1 $f6, 0($s2)    # 取 B
add.s $f4, $f4, $f6  # F = A + B
swc1 $f4, 0($s3)    # 存回
```

⚠ 易错点:浮点比较→分支是"两步走"(c.eq.s + bc1t),不能写成 `beq $f4, $f6, L`;且 `beq/bne` 的寄存器是**整数寄存器**,浮点寄存器另有一套。

3.8 子字并行(subword parallelism)

- 思想(八大思想第 4 条"并行"):一个 128 位寄存器,同时装 **4 个 32 位**或 **8 个 16 位**数据,一条指令同时算完 → 多媒体、图像处理的利器(x86 的 SSE/AVX 就是干这个的)。
- 为什么划算:加法器被切成几段,硬件几乎没增加,吞吐翻几倍。

3.9 谬误与陷阱

- 陷阱:左移代替乘 2 的幂,仅对无符号数恒对**;对有符号数,溢出时结果是错的(正确做法:sll 用于无符号/确认不溢出;sra 用于有符号除 2 的幂,注意负数向负无穷取整与 C 语言语义的关系)。
- 陷阱:浮点加法不满足结合律**(见 3.6.4)。
- 陷阱:浮点数是近似表示**,不能做精确比较(==),要用误差范围比较。
- 谬误:只有理论数学家才关心浮点精度**。1994 年 Intel Pentium 浮点除法错误直接造成数亿美元召回。
- 陷阱:整数与浮点的换算有代价**(lwc1 取进浮点寄存器后,整数运算碰不到它;两者寄存器组是分开的)。

五、名词解释汇总表

名词	英文	解释
全加器	full adder	1 位加法电路:两输入 + 进位入 → 和 + 进位出
行波进位	ripple carry	进位逐位串行传递的加法器结构,2n 级延迟
先行进位	carry lookahead (CLA)	进位并行计算的快速加法器
溢出	overflow	补码运算结果超出表示范围
Hi / Lo	—	存乘法积/除法结果的专用寄存器对
Booth 算法	Booth's algorithm	压缩乘数中连续 1 的乘法优化
恢复余数法	restoring division	先试减、为负则恢复的除法算法
IEEE 754	—	浮点数表示与运算的国际标准
偏阶	bias	存储指数加的偏移量(127/1023)
隐藏位	hidden bit	规格化数 1.xxx 中不存储的"1"
规格化数	normalized number	形式为 $1.xxx \times 2^e$ 的浮点数
非规格化数	denormalized number	Exp 全 0、Frac≠0 的超小浮点数
NaN	Not a Number	非法运算(0/0)的结果
无穷大	infinity	1.0/0.0 的结果,Exp 全 1、Frac 为 0

名词	英文	解释
保护位	guard bit	舍入用的第一个附加位
舍入位	round bit	舍入用的第二个附加位
粘贴位	sticky bit	其余附加位的逻辑或
就近舍入到偶数	round to nearest even	IEEE 默认舍入:恰好一半时取偶
子字并行	subword parallelism	宽寄存器同时算多个窄操作数

六、公式速查

公式	说明
$a - b = a + (\sim b + 1)$	减法用补码加法实现
溢出 = 正+正→负 或 负+负→正	符号判定法
值 = $(-1)^S \times (1.\text{Frac}) \times 2^{(\text{Exp}-\text{Bias})}$	规格化浮点数
值 = $(-1)^S \times (0.\text{Frac}) \times 2^{(1-\text{Bias})}$	非规格化数
$\text{Exp} = \text{真指数} + \text{Bias}(127 \text{ 或 } 1023)$	编码
浮点乘法: $\text{Exp} = \text{Exp}_1 + \text{Exp}_2 - \text{Bias}$	指数相加(去掉重复偏阶)
行波进位延迟 $\approx 2n$ 级门延迟	加法器性能
有效精度:单 24 位 / 双 53 位	含隐藏位

七、易混淆概念对比

对比	区别一句话
溢出 vs 进位	溢出=补码范围装不下(符号错);进位=无符号装不下(丢最高位)
add vs addu	前者溢出报错,后者不检测
乘法器第一版 vs 优化版	前者被乘数左移(64位ALU);后者积+乘数右移(32位ALU)
商的符号 vs 余数的符号	商:两符号异或;余数:跟随被除数
规格化数 vs 非规格化数	Exp 正常 vs Exp 全 0;后者填 0 附近的空隙
NaN vs ∞	0/0 之类非法 $\rightarrow$ NaN;1/0 $\rightarrow \infty$
单精度 vs 双精度	8 位指数/23 位尾数 vs 11/52;精度 24 vs 53 位
sra vs div	有符号除以 2 的幂可用 sra(快);一般除法用 div

对比	区别一句话
整数寄存器 vs 浮点寄存器	\$0-\$31 整数; \$f0-\$f31 浮点,两套互不通用
保护位 vs 粘贴位	单一附加位 vs 其余附加位的"或"

八、自测题

- (溢出)4 位补码运算,判断是否溢出: $0111+0001$; $1111+1111$; $1000+0111$; $0101+0011$ 。
- (乘法)用优化版乘法器手算 0110×0101 ,写出每步积寄存器的值。
- (数制)把 -0.75 转成 IEEE 754 单精度(十六进制)。
- (数制) $0x3F800000$ 表示的十进制数是多少?
- (数制)把 6.25 转成单精度。
- (浮点加)手算 $0.5 + 0.4375$,写出对齐/相加/规格化过程。
- (概念)什么是偏阶?为什么指数要加偏阶?
- (概念)什么是隐藏位?它带来什么好处?
- (概念)非规格化数解决什么问题?
- (概念)为什么浮点加法不满足结合律?举例说明。
- (MIPS)写代码实现单精度 $F = A * B + C$ (A 、 B 、 C 、 F 的地址分别在 $\$s0$ - $\$s3$ 指向处)。
- (概念)保护位、舍入位、粘贴位分别是什么?"就近舍入到偶数"为什么优于"四舍五入"?

参考答案

- $0111+0001=1000:7+1=-8 \rightarrow$ **溢出**(正+正=负); $1111+1111=1110:-1-1=-2 \rightarrow$ 无溢出; $1000+0111=1111:-8+7=-1 \rightarrow$ 无溢出(异号相加); $0101+0011=1000:5+3=-8 \rightarrow$ **溢出**。
- 0110×0101 (被乘数 0110 ,乘数 0101):
 - 初值:0000 0101
 - 步1:最低位 1 $\rightarrow$ 加:0110 0101 $\rightarrow$ 右移:0011 0010
 - 步2:最低位 0 $\rightarrow$ 右移:0001 1001
 - 步3:最低位 1 $\rightarrow$ 加:0111 1001 $\rightarrow$ 右移:0011 1100
 - 步4:最低位 0 $\rightarrow$ 右移:0001 1110 = 30 $\checkmark$
- $0xBF400000$ (过程见 3.5.4)。
- 1.0。
- $6.25 = 110.01_2 = 1.1001 \times 2^2 \rightarrow S=0, \text{Exp}=2+127=129=1000\ 0001, \text{Frac}=100100\dots 0 \rightarrow$ **$0x40C80000$** 。
- $0.5=1.0 \times 2^{-1}; 0.4375=0.0111_2=1.11 \times 2^{-2} \rightarrow$ 对齐: 0.111×2^{-1} ;相加: $1.0 \times 2^{-1}+0.111 \times 2^{-1}=1.111 \times 2^{-1}$;已是规格化 $\rightarrow$ 结果 $1.111 \times 2^{-1} = 0.9375 \checkmark$ 。
- 偏阶是加到真指数上的常数(单 127/双 1023)。作用:① 让存储的指数恒为非负,方便按无符号数比较大;② 排序时浮点数可直接当整数比较。

8. 隐藏位是规格化数"1.xxx"里不存储的"1"。好处:同样位数多 1 位有效精度(23 位 Frac 实际 24 位精度)。
9. 填满 0 与最小规格化数之间的空隙,让小数值渐进衰减到 0(渐进下溢),避免骤然变 0 的巨大误差。
10. 因为对齐和舍入:大数加极小数时,小数对齐后可能被完全舍掉。例:单精度下 $(1e20 + 1e-20) + (-1e20) = 0$,而 $1e20 + (1e-20 - 1e20) = 1e-20$ (数量级)——括号不同结果不同。
- 11.

```
lwc1 $f4, 0($s0)    # A
lwc1 $f6, 0($s1)    # B
lwc1 $f8, 0($s2)    # C
mul.s $f4, $f4, $f6  # A*B
add.s $f4, $f4, $f8  # +C
swc1 $f4, 0($s3)    # F
```

12. 保护位=舍入时保留的第一位附加位;舍入位=第二位;粘贴位=其余附加位的或(记住"后面是否还有非零")。就近舍入到偶数在"恰好一半"时一半进一半舍,避免"逢五全进"导致的系统性偏差累积。

九、本章小结

- 一切运算归于**加法与移位**:减法=补码加法,乘法=移位+加,除法=试减+恢复。
- IEEE 754 浮点 = 二进制科学计数法:**符号、带偏阶的指数、隐藏位的尾数**,外加一整套特殊值(0/∞/NaN/非规格化数)。
- 浮点有精度代价:会舍入、不满足结合律——这是工程权衡,不是 bug。

下一章 → [04-第4章-处理器.md](#):进入 CPU 内部,看数据通路与流水线。

第 4 章 处理器

一句话导读:这一章拆开 CPU 的外壳:指令在 CPU 里是怎么被"取、译、算、存"的?为什么 CPU 要像工厂流水线一样工作?冒险(hazard)是什么、怎么解决?这是**全书最难也最核心**的一章,拿下它,你就真正"懂计算机"了。

对应教材:第 4 章「处理器(The Processor)」

一、本章思维导图

第4章 处理器

4.1 数据通路(datapath)

- 部件:PC、指令存储器、加法器、寄存器组、ALU、数据存储器
符号扩展器、移位器
- 通用五步:取指 → 译码 → 执行 → 访存 → 写回
- 各指令的路径:add(R型)、lw、sw、beq、j

4.2 单周期实现(最简单的CPU)

- 特点:一条指令一个时钟周期
- 控制信号:RegDst/RegWrite/ALUSrc/ALUOp/MemRead/MemWrite
/MemtoReg/Branch/Jump
- ALU控制:ALUOp + funct → 具体运算
- 主控单元:纯组合逻辑(真值表)
- 致命缺点:周期 = 最慢指令(lw),浪费

4.3 流水线(pipelining)★

- 思想:像洗车流水线,指令重叠执行
- 五级:IF → ID → EX → MEM → WB
- 流水线寄存器:IF/ID、ID/EX、EX/MEM、MEM/WB
- 加速的是吞吐率(每条指令的时间不变!)
- 加速比 ≈ 级数(理想),实际打折

4.4 冒险(hazards)★★本章最难点

- 结构冒险:部件不够用(如单存储器)
- 数据冒险:后面指令要用前面的结果
 - 解决1:转发(forwarding)—把结果"抄近路"
 - 场景:EX/MEM 转发(ALU结果直接送)
 - 场景:MEM/WB 转发(访存结果)
 - 解决2:阻塞(stall)+ 气泡(bubble)
 - load-use 冒险:转发也不够,必须停1拍
- 控制冒险:分支方向未定
 - 解决:尽早判断分支(ID段)+ 预测不跳
 - 猜错代价:冲刷(flush)误取的指令

4.5 异常与中断

- 异常(内部)/ 中断(外部)统称 exception
- EPC:保存出错指令地址
- Cause:记录异常原因
- 向量中断:固定入口的处理程序
- 处理流程:保存 → 跳转处理 → 恢复(eret)

4.6 指令级并行(ILP)

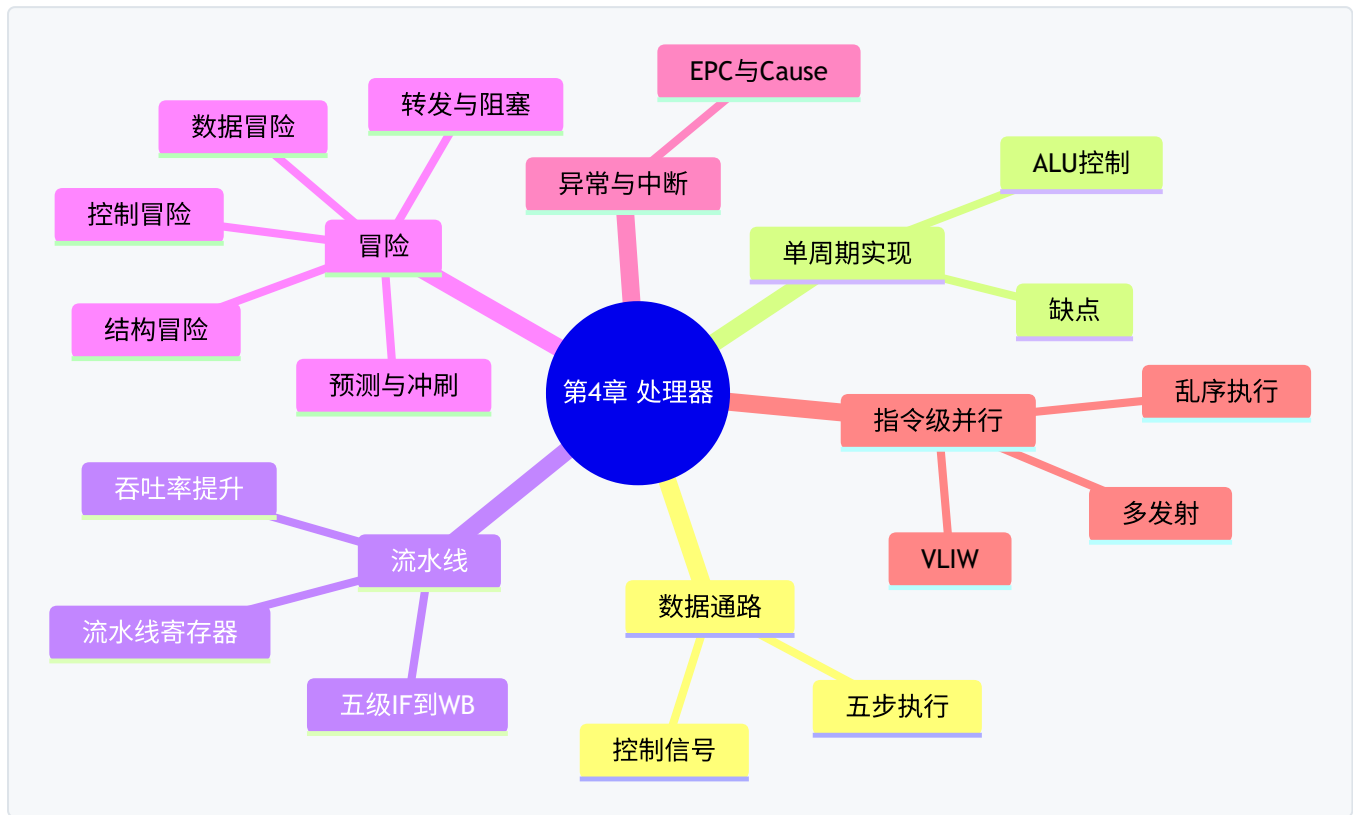
- 多发射(multiple issue):每周发射多条指令
- 静态多发射:VLIW(编译器排好)
- 动态多发射:乱序执行、寄存器重命名、推测执行
- CPI = 1/发射宽度(理想)

4.7 谬误与陷阱

- 流水线不是越深越好(收益递减、功耗大)

- └ 纯汇编未必最快(编译器+乱序执行很强)
- └ 分支预测失败的代价巨大

Mermaid 版(可渲染)



二、学习目标

1. 说出指令执行的通用五步,以及各步用到的数据通路部件。
2. 能画出(或描述)lw、sw、beq 在单周期数据通路中的完整路径。
3. 能默写控制信号表(RegDst/ALUSrc/MemtoReg/RegWrite/MemRead/MemWrite/Branch/ALUOp)。
4. 理解 ALU 控制:ALUOp + funct 如何决定 ALU 做什么。
5. 能画出流水线的多时钟周期图(时空图)。
6. **会找数据冒险**,并判断:哪种用转发解决、哪种必须阻塞。
7. 理解转发单元和冒险检测单元的工作原理。
8. 理解控制冒险与"预测不跳 + 冲刷"。
9. 能区分异常与中断,说出 EPC、Cause 的作用。
10. 知道多发射、VLIW、乱序执行、寄存器重命名的基本思想。

三、核心名词先混个脸熟

名词	一句话
数据通路(datapath)	CPU 里"干活"的部件:寄存器组、ALU、存储器等
控制器(control)	产生控制信号、指挥数据通路的部件
单周期(single-cycle)	每条指令固定用 1 个时钟周期的实现
流水线(pipelining)	多条指令重叠执行的实现技术
流水线寄存器	各级之间的"隔板",保存级间传递的数据
冒险(hazard)	流水线中导致下条指令无法正确执行的情况
转发(forwarding)	把结果抄近路直接送给需要的级
气泡(bubble)	插入流水线的"空指令"(什么都不做)
阻塞(stall)	让流水线停一拍
load-use 冒险	lw 后紧跟使用该值的指令,必须阻塞
控制冒险	分支结果未知导致后续取指不确定
冲刷(flush)	丢弃误取的指令,换成气泡
异常(exception)	指令执行中的意外事件(内部)
中断(interrupt)	外部设备发来的事件
EPC	保存异常指令地址的寄存器
Cause	记录异常原因的寄存器
多发射(multiple issue)	一个时钟周期发射多条指令
VLIW	超长指令字,编译器静态排好并行
乱序执行(out-of-order)	硬件动态重排指令执行顺序
寄存器重命名	消除假依赖(WAW/WAR)的技术
推测(speculation)	提前执行"猜"的指令,猜错回滚

四、分节精讲

4.1 数据通路:CPU 的"器官"

通用五步(任何指令都是这五步的子集):

1. **取指(IF, Instruction Fetch)**:PC → 指令存储器,取出指令;PC 自增 4。
2. **译码(ID, Instruction Decode)**:读寄存器组,识别指令。

- 3. **执行(EX, Execute)**:ALU 运算(算数、算地址、比分支)。
- 4. **访存(MEM, Memory Access)**:只有 lw/sw 用:读写数据存储器。
- 5. **写回(WB, Write Back)**:结果写回寄存器。

数据通路部件清单(与五步对应):

部件	用途
PC(程序计数器)	存下一条指令地址
指令存储器	存程序(机器码)
加法器(+4)	PC = PC + 4
寄存器组(register file)	32 个寄存器,同时支持"读 2 个 + 写 1 个"
符号扩展器(sign extend)	16 位立即数 → 32 位
ALU	算术/逻辑运算
数据存储器	存数据(lw/sw 用)
移位器(shift left 2)	分支偏移量 ×4、j 地址 ×2

各指令走哪条路(考试画图题):

- **R 型(add)**:取指 → 读 rs、rt → ALU 算 → 写回 rd。不访存。
- **lw**:取指 → 读基址寄存器 → ALU 算地址(基址+符号扩展的偏移)→ 数据存储器读出 → 写回 rt。
- **sw**:取指 → 读基址 + 读数据(rt)→ ALU 算地址 → 写入数据存储器。 **不写回寄存器。**
- **beq**:取指 → 读 rs、rt → ALU 做减法比相等 → 结果控制 PC。
- **j**:取指 → 用指令的 26 位地址拼接出目标地址 → 更新 PC。

4.2 单周期实现:第一版 CPU

思想:整个 CPU 用一个超长时钟周期,每条指令**一个周期**内完成。周期长度 = 最慢指令(lw:取指+读寄存器+ALU+访存+写回)的时间。

控制信号表(必背,考试大题):

信号	作用	R型	lw	sw	beq
RegDst	写 rd(1)还是 rt(0)	1	0	×	×
ALUSrc	ALU 第二操作数:寄存器(0)还是立即数(1)	0	1	1	0
MemtoReg	写回数据来自 ALU(0)还是存储器(1)	0	1	×	×
RegWrite	是否写寄存器	1	1	0	0
MemRead	是否读数据存储器	0	1	0	0
MemWrite	是否写数据存储器	0	0	1	0

信号	作用	R型	lw	sw	beq
Branch	是否分支	0	0	0	1
ALUOp(2位)	ALU 做什么	10	00	00	01

ALU 控制(ALUOp + funct 两级译码):

ALUOp	含义	结合 funct 的结果
00	加法(lw/sw 算地址)	加
01	减法(beq 比较)	减
10	由 funct 决定	100000→加,100010→减,100100→与,100101→或,101010→小于置位

- 主控单元 = **纯组合逻辑**(真值表直接实现,不需要状态)。
- **?** 为什么 ALU 要两级译码?因为 R 型指令有 6 种运算,1 位 ALUOp 不够表达;分成"大类(ALUOp)+ 细分(funct)"让主控单元简单规整(原则 1)。

单周期的致命缺点:时钟周期被最慢指令拖死(其他指令都在"等"lw 的时间);且加法器、存储器等部件每条指令只用一小段时间,其余时间闲着 → 违反"加速大概率事件"。

4.3 流水线:像洗车场一样工作 ★

4.3.1 思想

把指令执行分成 5 级,像**洗车流水线**:第 i 条指令在"冲洗"时,第 i-1 条在"擦干".....**同一时刻 5 条指令处于不同阶段。**

五级:IF(取指)→ ID(译码)→ EX(执行)→ MEM(访存)→ WB(写回)

- **流水线寄存器(IF/ID、ID/EX、EX/MEM、MEM/WB):**级与级之间的"隔板",把本级的结果(指令、数据、控制信号)锁存起来传给下一级。它们的存在让各级能"各干各的",互不干扰。
- **💡 大白话:**流水线寄存器像传送带上的托盘——每道工序完成后,把半成品放托盘上传给下一道工序,自己立刻接下一个活儿。

4.3.2 性能:加速的是"吞吐率"

- 单周期:5 条指令要 5 个长周期;流水线:5 条指令约 9 个短周期(5 级),之后**每周期流出一条指令**。
- 理想加速比 ≈ **级数(5 倍)**。为什么不是精确 5 倍:① 各级耗时不平衡(周期=最慢级);② 流水线寄存器有开销;③ 冒险会打乱节奏。
- **⚠ 易错点:**流水线**不缩短单条指令的时间**(反而略增),它提高的是**单位时间完成的指令数**(吞吐率)。

4.3.3 图形化表示(两种图都要会画)

单时钟周期图:画出某一时刻每级在做什么。

多时钟周期图(时空图):横轴时间(周期),纵轴指令,每行一条指令的推进:

周期:	1	2	3	4	5	6	7	8	9
lw \$t0,20(\$t1)	IF	ID	EX	MEM	WB				
sub \$t1,\$t2,\$t3		IF	ID	EX	MEM	WB			

4.4 冒险:流水线的三只拦路虎 ★★

4.4.1 结构冒险(structural hazard)

原因:硬件部件不够用,两条指令同时抢一个部件。 **经典例子:**指令和数据共用**同一个存储器** → 第 i 条取指和 lw 的访存撞车。 **解决:**指令和数据分开存储(哈佛结构),或把部件"复制"多份。现在处理器普遍通过独立的一级指令/数据 cache 规避。

4.4.2 数据冒险(data hazard)★

原因:后面的指令要用前面指令**还没写回**的结果。

经典序列(教材原例):

```
sub  $2, $1, $3    # ①
and  $12, $2, $5    # ② 需要 $2 的新值
or   $13, $6, $2    # ③ 需要 $2 的新值
add  $14, $2, $2    # ④ 需要 $2 的新值
sw   $15, 100($2)   # ⑤ 需要 $2 的新值
```

- ②紧跟①:①的 ALU 结果在**EX 结束后**就有,但还要走 MEM、WB 才写寄存器,②等不及 → **转发**解决。
- ③④⑤隔得远:①的结果此时已在 **MEM/WB 段**,同样转发。
- **转发(forwarding / bypassing):**把 ALU 结果或访存结果**抄近路**直接送进 ALU 的输入,不等写回寄存器。加一个**转发单元** + 两个多路选择器(ForwardA/ForwardB)。
 - **EX/MEM 转发:**结果来自前一条指令的 EX 段(解决场景一)。
 - **MEM/WB 转发:**结果来自更早指令的 MEM/WB 段(解决场景二)。
- 💡 大白话:转发像"厨房直接递菜"——菜炒好了直接从灶台递到出餐口,不用先端回仓库(寄存器)再取出来。

load-use 冒险:转发救不了,必须阻塞!

```
lw   $s0, 20($t1)   # ① 数据要到 MEM 结束后才有
and  $t2, $s0, $t3   # ② 在 EX 段就要用 $s0 → 差一拍!
```

- lw 的结果最快要到 **MEM 结束**才出现,而②在**自己的 EX 段**就要用 → 时间上差一个周期,**转发也够不着**。
- 解决:**冒险检测单元(hazard detection unit)** 检测到这种情况 → 让②(以及后随指令)**阻塞(stall)**一个周期,插入一个**气泡(bubble,即空操作)**;同时配合 MEM/WB 转发。
- 检测条件: ID/EX.MemRead 为真 且 (ID/EX.rt == IF/ID.rs 或 IF/ID.rt) → stall。
- 时空图(阻塞 + 转发):

周期:	1	2	3	4	5	6	7	8
lw \$s0,20(\$t1)	IF	ID	EX	MEM	WB			
and \$t2,\$s0,\$t3		IF	ID	泡	EX	MEM	WB	← 在ID多停1拍

- **⚠ 易错点:**考试常问"哪些冒险转发解决、哪些必须阻塞"。口诀:**ALU 结果滞后 1~2 条指令 → 转发;load 结果紧跟使用 → 阻塞 + 转发。**

4.4.3 控制冒险(control hazard)★

原因:分支指令(beq/bne)的**跳不跳还不知道**,后面的取指怎么办?

解决三步走:

1. **提前判断:**把分支判断从 MEM 段**提前到 ID 段**(加一个小比较器),代价减到只猜错 1 条指令;
 2. **预测:**默认**预测分支不跳**(大概率事件:多数分支不跳),按 PC+4 顺序取指;
 3. **猜错了就冲刷(flush):**把误取的指令在 IF/ID 寄存器里**改成气泡**(把控制信号清零),代价 = 浪费 1 个周期。
- **💡 大白话:**像堵车时的导航——先按"直行"往前开(预测不跳),到路口发现该转弯(猜错),只好倒回一个身位(冲刷)。更高级的处理器有"历史记录"帮助预测(2 位预测器,见第 6 章硬件多线程前后文)。

4.4.4 指令集设计必须配合流水线(呼应第 2 章)

- 所有指令**等长**(32 位)→ 取指和译码才能流水;
- 指令格式**规整**(三种)→ 译码简单;
- 只有 lw/sw **访存** → MEM 级统一;
- **分支延迟槽(branch delay slot):**老 MIPS 让分支后紧跟的那条指令"无论如何都执行",抵消冲刷代价——历史设计,如今大多已不用,了解即可。

4.5 异常与中断

术语(本书统称 exception):

- **异常(exception):**CPU **内部**的意外——算术溢出、未定义指令、系统调用(syscall)。
- **中断(interrupt):**来自 CPU **外部**的事件——I/O 设备请求、键盘、网络。

处理机制(MIPS 风格):

1. 保存现场:出错的指令地址存入 **EPC**(Exception Program Counter,异常程序计数器);
 2. 记录原因:原因码写入 **Cause** 寄存器;
 3. 跳转到**固定入口**(向量中断,如 MIPS 的 0x8000 0180)执行异常处理程序;
 4. 处理完用 `eret / mfc0` 从 EPC 恢复,继续执行。
- **?** 什么是**向量中断(vectored interrupt)**?异常类型直接决定跳转地址(每种异常一个固定入口)。好处:不用先判断类型再跳,响应快。
 - **⚠ 易错点:**EPC 存的是**出错指令的地址**。如果是分支延迟槽里的指令出错,EPC 要存分支指令的地址(为了能正确返回)——细节考点。

4.6 指令级并行(ILP):更狠的提速

单流水线到头后,进一步**每个周期发射(issue)多条指令**:

- **多发射(multiple issue)**:理想 $CPI = 1/\text{发射宽度}$ (每周期发 2 条 $\rightarrow CPI\ 0.5$)。
- **静态多发射 / VLIW(超长指令字)**:由**编译器**把能并行的指令打包成一条长指令,硬件简单但编译器难(要分析依赖、排指令)。
- **动态多发射 / 乱序执行(out-of-order execution)**:硬件在运行中自己找"能并行执行"的指令,**打乱顺序执行**(但提交按序,保证结果正确)。配套技术:
 - **寄存器重命名(register renaming)**:消除"假依赖"(两条指令用同一个寄存器但数据无关),多准备一批物理寄存器给它们"换名字"。
 - **推测执行(speculation)**:分支没定之前就"猜着执行",猜错回滚。
 - (旁注:Tomasulo 算法——历史上第一台乱序执行的 IBM 360/91 的经典方案,考试了解思想即可。)
- 💡 大白话:动态多发射像**餐厅后厨的经理**——单子上 5 道菜,经理发现两道菜都用"3 号锅"但互不影响,就另拿一口锅(重命名)同时炒(乱序),上菜顺序仍按单子(按序提交)。

4.7 谬误与陷阱

1. **陷阱:流水线级数不是越深越好**。 每级拆分产生额外的流水线寄存器开销;级数深了,一次预测失败的代价更大,功耗也更高(Pentium 4 的 31 级超深流水线是著名反例)。
2. **谬误:手写汇编一定比编译器快**。 现代编译器 + 乱序执行已经非常强,人肉排指令往往不如编译器;真正的性能在算法。
3. **陷阱:用峰值吞吐率评价处理器**。 冒险、cache 缺失会让实际吞吐远低于"每周期 N 条"的理论值。
4. **陷阱:忽略分支预测失败的代价**。 深流水线 + 高发射宽度下,一次猜错可能浪费几十条指令的"坑位"。

五、名词解释汇总表

名词	英文	解释
数据通路	datapath	执行指令的运算部件(寄存器组、ALU、存储器)
控制器	control unit	产生控制信号的部件
单周期实现	single-cycle implementation	每条指令一个长时钟周期
流水线	pipelining	多条指令重叠执行
流水线寄存器	pipeline register	级间锁存器,传递中间结果
结构冒险	structural hazard	硬件部件被两条指令同时争用
数据冒险	data hazard	后续指令等待前序指令的数据
转发	forwarding / bypassing	结果抄近路直送 ALU 输入
load-use 冒险	load-use hazard	lw 结果紧跟使用,必须阻塞

名词	英文	解释
阻塞	stall	流水线停一个或多个周期
气泡	bubble	插入流水线的空操作
控制冒险	control hazard	分支方向未定引发的取指问题
冲刷	flush	丢弃误取的指令
分支预测	branch prediction	猜测分支方向,错了再纠正
异常	exception	CPU 内部意外事件
中断	interrupt	外部设备事件
EPC	Exception PC	保存异常指令地址
Cause	—	保存异常原因码
向量中断	vectored interrupt	异常类型直接映射到处理入口
多发射	multiple issue	每周期发射多条指令
VLIW	Very Long Instruction Word	编译器静态打包的超长指令字
乱序执行	out-of-order execution	硬件动态重排执行顺序
寄存器重命名	register renaming	用物理寄存器消除假依赖
推测执行	speculation	提前执行猜测路径的指令
分支延迟槽	branch delay slot	分支后恒执行的指令位(老 MIPS)

六、公式速查

公式	说明
单周期执行时间 = $IC \times \text{最长指令时间}$	单周期性能
流水线理想加速比 $\approx$ 级数	五级 ≈ 5 倍
流水线执行时间 $\approx (IC + \text{级数} - 1) \times \text{周期}$	首条"灌满"后每周期一条
CPI 理想 = $1 / \text{发射宽度}$	多发射
异常代价 = 预测失败率 $\times$ 冲刷代价	控制冒险成本

七、易混淆概念对比

对比	区别一句话
数据通路 vs 控制器	干活的 vs 指挥的(合起来是 CPU)
单周期 vs 多周期 vs 流水线	每条指令 1 个长周期 / 1 条指令分多个周期执行 / 多条指令重叠执行
结构冒险 vs 数据冒险 vs 控制冒险	抢部件 / 等数据 / 等分支方向
转发 vs 阻塞	抄近路(不损失时间)vs 停下来(损失周期)
气泡 vs 冲刷	插入空操作(阻塞)vs 把已取指令变成空操作(丢弃)
异常 vs 中断	内部事件 vs 外部设备事件
EPC vs Cause	存"在哪出错" vs 存"为什么出错"
静态多发射 vs 动态多发射	编译器排指令(VLIW)vs 硬件乱序执行
真依赖 vs 假依赖(WAW/WAR)	数据真相关(必须等)vs 寄存器冲突(重命名可消)
延迟槽 vs 冲刷	编译器填指令抵消代价 vs 硬件丢弃误取指令

八、自测题

- (简答)指令执行的通用五步是什么?哪一步只有 lw/sw 才做?
- (填表)填出 lw、beq 的控制信号(RegDst、ALUSrc、MemtoReg、RegWrite、MemRead、MemWrite、Branch、ALUOp)。
- (简答)ALUOp=10 时,ALU 控制如何决定具体运算?举 funct=101010 的例子。
- (画图)画出流水线的多时钟周期图(前 5 周期):lw、add、beq、sw、sub 依次进入流水线。
- (冒险)找出下面代码的所有数据冒险,并说明解决方式:

```
sub  $2, $1, $3
and  $12, $2, $5
or   $13, $6, $2
add  $14, $2, $2
sw   $15, 100($2)
```

- (冒险)为什么 lw \$s0, 20(\$t1) 后紧跟 and \$t2, \$s0, \$t3 必须阻塞一拍?冒险检测单元如何检测?
- (简答)转发解决的是什么?转发不了的是什么?各举一个例子。
- (简答)控制冒险怎么解决?为什么默认预测"不跳"?
- (简答)异常和中断的区别?EPC 和 Cause 各存什么?
- (简答)什么是寄存器重命名?它消除什么依赖?
- (简答)为什么流水线不是越深越好?

12. (计算)五级流水线,周期 2ns;程序有 1000 条指令,无冒险。求总执行时间与理想加速比(对比单周期:每指令 10ns)。

参考答案

1. 取指(IF)、译码/读寄存器(ID)、执行(EX)、访存(MEM)、写回(WB)。MEM 只有 lw/sw 才做。
2. lw:RegDst=0、ALUSrc=1、MemtoReg=1、RegWrite=1、MemRead=1、MemWrite=0、Branch=0、ALUOp=00;beq:RegDst=x、ALUSrc=0、MemtoReg=x、RegWrite=0、MemRead=0、MemWrite=0、Branch=1、ALUOp=01。
3. ALUOp=10 表示"看 funct":funct=100000→加、100010→减、100100→与、100101→或、101010→小于置位(slt)。两级译码让主控简单规整。
- 4.

周期:	1	2	3	4	5
lw	IF	ID	EX	MEM	WB
add		IF	ID	EX	MEM
beq			IF	ID	EX
sw				IF	ID
sub					IF

5. ②对①:EX/MEM 转发;③对①:MEM/WB 转发;④对①:MEM/WB 转发(两个源都来自 \$2);⑤对①:MEM/WB 转发(sw 的写数据端口)。
6. lw 的数据要到 MEM 结束才有,而 and 在自己的 EX 段就要用,时间上差一拍,转发够不着 → 必须 stall 一拍 + 转发。检测:冒险检测单元发现 ID/EX.MemRead=1 且 ID/EX.rt 等于 IF/ID.rs 或 IF/ID.rt,就插入气泡、阻塞一拍。
7. 转发解决"结果已算出但还没写回寄存器"的滞后(sub→and);转发不了的是 load-use(数据还没算出来,如 lw→and),必须阻塞。转发像抄近路,但近路上也得有"货"才行。
8. ① 分支判断提前到 ID 段;② 预测不跳(多数分支确实不跳,加速大概率事件);③ 猜错则冲刷误取指令(变气泡),代价 1 周期。
9. 异常来自 CPU 内部(溢出、未定义指令、syscall),中断来自外部设备(I/O)。EPC 存出错指令的地址,Cause 存异常原因码。
10. 寄存器重命名:给指令分配额外的物理寄存器"换名",消除 WAW(写后写)、WAR(读后写)这类"假依赖"(只是撞了寄存器名字,数据并无先后关系),让更多指令能并行/乱序执行。
11. ① 级数越多,流水线寄存器开销越大、时钟周期被寄存器延迟拖累;② 预测失败/冒险的代价变大;③ 功耗上升。收益递减(Pentium 4 的 31 级是反例)。
12. 流水线: $(1000 + 5 - 1) \times 2\text{ns} = 2008\text{ ns}$;单周期: $1000 \times 10\text{ns} = 10000\text{ ns}$;加速比 = $10000/2008 \approx 4.98 \approx 5$ 倍。

九、本章小结

- **数据通路是骨架**: 五步(IF/ID/EX/MEM/WB)贯穿所有指令,控制信号表是"大脑"。

- **流水线是灵魂**:重叠执行,加速吞吐率;五级是理解一切现代 CPU 的起点。
- **冒险是必修课**:数据冒险 → 转发/阻塞;控制冒险 → 提前判断 + 预测 + 冲刷。能把任意一段代码的冒险分析清楚,这章就通了。
- **再往上走**:多发射 + 乱序 + 推测 = 现代高性能 CPU 的配方。

下一章 → [05-第5章-存储器层次结构.md](#):CPU 那么快,内存那么慢,中间靠什么救场?

第 5 章 大容量和高速度:开发存储器层次结构

一句话导读: CPU 很快,内存很慢(差两个数量级!)——计算机靠"又快又小 + 又慢又大"的金字塔结构让程序感觉"内存既快又大"。本章讲清楚 cache(缓存)和虚拟存储器这两大魔法。

对应教材: 第 5 章「大容量和高速度:开发存储器层次结构(Large and Fast: Exploiting Memory Hierarchy)」

一、本章思维导图

第5章 存储器层次结构

5.1 局部性原理(整个层次结构的基石!)

- 时间局部性:刚访问的,很可能再访问
- 空间局部性:隔壁的,很可能也要访问

5.2 存储器技术四兄弟

- SRAM:6个晶体管/位,快,贵 → cache
- DRAM:1晶体管+电容/位,慢,需刷新 → 主存
- Flash:非易失,擦写寿命有限 → SSD/U盘
- 磁盘:机械转动,最慢最便宜 → 大容量存储

5.3 cache 基础

- 结构:块(block)= 标记(tag)+ 数据 + 有效位
- 地址划分:标记 | 索引 | 块内偏移
- 命中(hit)/ 缺失(miss)
- 三种映射方式:
 - 直接映射:每块只有一个位置
 - 全相联:任意位置(要全比一遍)
 - 组相联:N路,折中(最常用)
- 写策略:写直达(+写缓冲)/ 写回(脏位)
 - 写分配 / 非写分配
- $AMAT = \text{命中时间} + \text{缺失率} \times \text{缺失代价}$
- 6 种基本优化(块大/cache大/相联度高/多级/读优先/索引免翻译)

5.4 cache 的高级优化(10 种,按目标分类)

- 降命中时间:小而简单、路预测
- 增带宽:流水线化、多bank、关键字优先
- 降缺失代价:合并写缓冲、多级cache
- 降缺失率:编译器优化(循环交换、分块)
- 并行:硬件预取、编译器预取

5.5 虚拟存储器 ★

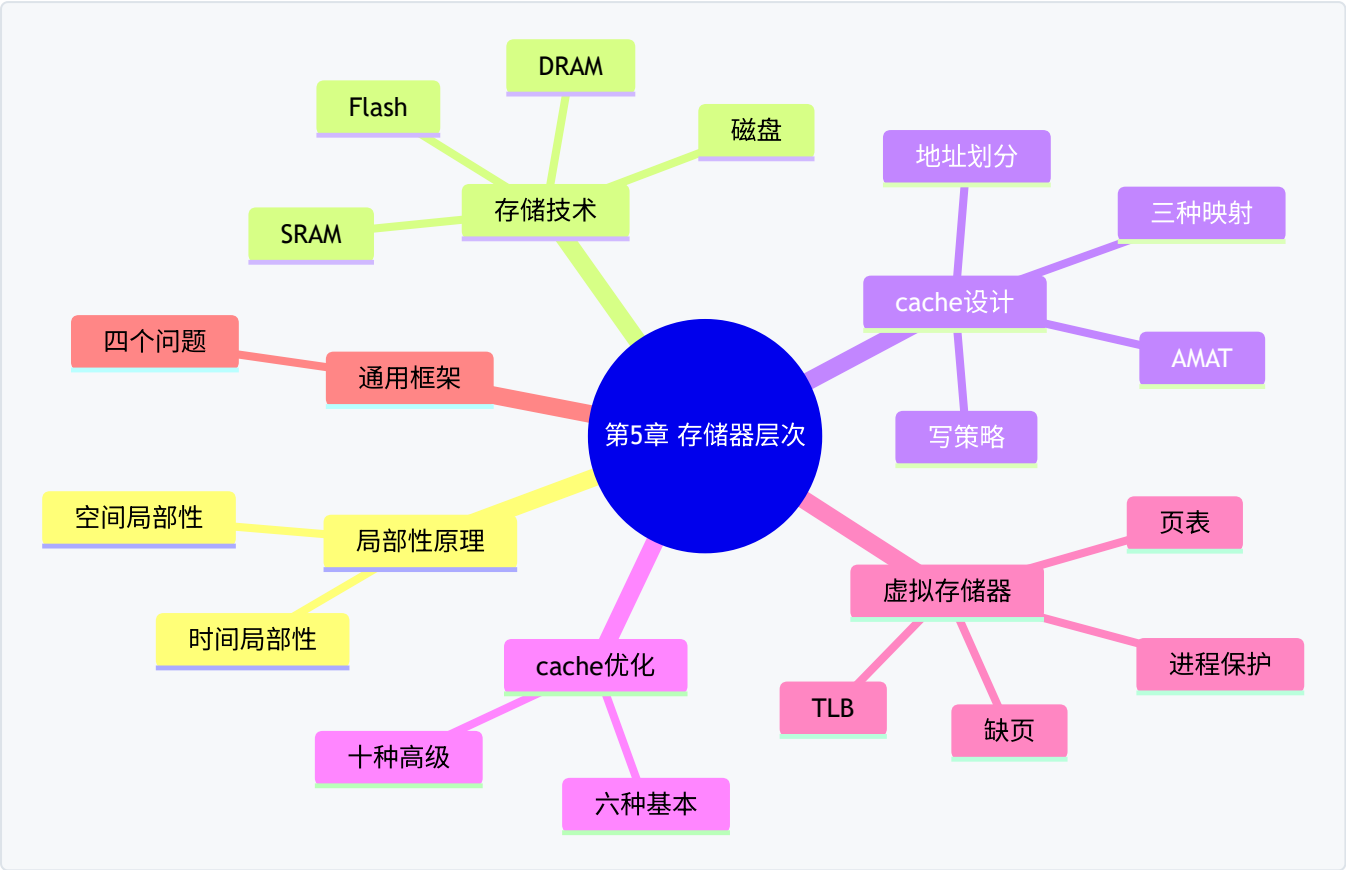
- 动机:让程序觉得"独享大内存" + 隔离进程
- 页(page)与页表(page table)
- 地址转换:虚拟地址 → 物理地址
- TLB(快表):页表的 cache
- 缺页(page fault):到磁盘取页,代价巨大
- 保护:每个进程独立页表,非法访问报错

5.6 存储器层次的通用框架(四个问题)

- Q1 块放哪?(映射策略)
- Q2 如何找到块?(标记+索引)
- Q3 缺失替换谁?(LRU/随机)
- Q4 写怎么办?(写直达/写回)

5.7 谬误与陷阱

- 缺失率低 ≠ 性能好(还要看缺失代价/命中时间)
- cache 越大不一定越快(命中时间变长)
- 峰值带宽 ≠ 实际带宽(现实受多因素限制)



二、学习目标

- 1. 用例子解释时间局部性和空间局部性。
- 2. 说出 SRAM/DRAM/Flash/磁盘 的特点与用途分工。
- 3. **会做 cache 地址划分**:给定容量、块大小、相联度,算出标记/索引/偏移位数(必考)。
- 4. 对比三种映射方式(直接/全相联/组相联)的优缺点。
- 5. 解释写直达、写回、写分配的含义与搭配。
- 6. **会算 AMAT**(必考)。
- 7. 说出 6 种基本优化与 10 种高级优化的分类。
- 8. 描述虚拟地址到物理地址的转换流程(TLB 命中与缺失)。
- 9. 解释缺页、TLB、页表的作用。
- 10. 用"四个问题"框架描述任意一级存储层次。

三、核心名词先混个脸熟

名词	一句话
局部性(locality)	程序访问数据的规律性(时间上/空间上)

名词	一句话
块(block)	cache 与主存之间传输的最小单位
标记(tag)	记录"这块装的是主存哪一块"的信息
索引(index)	地址中决定"去哪个 cache 位置找"的字段
有效位(valid bit)	标记该 cache 块是否有有效数据
命中/缺失(hit/miss)	要的数据在/不在 cache 里
缺失率(miss rate)	缺失次数 ÷ 访问次数
直接映射(direct mapped)	每块主存块只有一个 cache 位置可放
全相联(fully associative)	主存块可放任意 cache 位置
组相联(set associative)	折中:N 路,每组 N 个位置
写直达(write-through)	写 cache 时同时写主存
写回(write-back)	只写 cache,替换出去时才写主存(脏位)
写分配(write allocate)	写缺失时先把块取进 cache
AMAT	平均访存时间 = 命中时间 + 缺失率 × 缺失代价
LRU	最近最少使用替换策略
页(page)	虚拟存储中固定大小的块(典型 4KB)
页表(page table)	虚拟页号 → 物理页号 的映射表
TLB	页表的 cache(快表)
缺页(page fault)	页不在主存,要去磁盘取
脏位(dirty bit)	标记块是否被改过(写回用)
预取(prefetching)	提前把可能要用的块取来

四、分节精讲

5.1 局部性原理:一切的前提

- **时间局部性(temporal locality)**:刚访问过的数据,很快很可能再次访问(循环里的变量、函数调用)。
- **空间局部性(spatial locality)**:访问一个数据后,它**附近**的数据很可能也要访问(数组顺序遍历、顺序执行的指令)。
- 层次结构能成功,正是因为程序有局部性:把"最近常用的一小片"放在高层,大部分访问就命中了(八大思想第 7 条)。

- 💡 大白话:像学生写作业——桌面(cache)上只放当前要用的几本书,大部分书在书架(内存),更不常用的在图书馆(磁盘)。写作业时 90% 的时间都在桌面拿书。

5.2 存储器技术:四种材料,四种分工

技术	每 bit 电路	速度	价格	易失性	用途
SRAM	6 个晶体管	最快	最贵	易失	cache
DRAM	1 晶体管+1 电容	慢	便宜	易失(需刷新)	主存
Flash	浮栅管	较慢	较便宜	非易失	SSD、U 盘
磁盘	机械盘片	最慢	最便宜	非易失	大容量备份

- ❓ 为什么 cache 用 SRAM 而主存用 DRAM?SRAM 快但太贵,只能小容量;DRAM 密度高、便宜,可做大容量。**快和便宜不可兼得 → 分层。**
- ⚠️ 易错点:DRAM 的电容会漏电,**必须周期性刷新(refresh)**;Flash 擦写次数有限(约 10 万次),会"磨损"。

5.3 cache 基础

5.3.1 结构与地址划分

cache 按**块(block,也叫行 line)**组织,一块通常 16~64 字节。主存也按同样大小的块划分。要访问某地址:

```
32位地址 = [标记 tag] [索引 index] [块内偏移 offset]
```

- 索引**决定"去 cache 的哪个位置找";
- 标记**验证"这个位置放的是不是我要的那一块";
- 偏移**定位块内第几个字节。

计算示例(必会):

```
32 位地址,64KB 直接映射 cache,块大小 16 字节。块数 = 64KB ÷ 16B = 4096 = 212 → 索引 12 位;  
偏移 = log216 = 4 位; 标记 = 32 - 12 - 4 = 16 位。
```

5.3.2 三种映射方式(必考对比)

方式	规则	硬件	缺失率	查找
直接映射	主存块 i 只能放 cache 的 (i mod 块数) 号位	最简单	最高(冲突多)	只查 1 个位置
全相联	任意位置	最复杂(全比较)	最低	同时比所有标记
N 路组相联	每组 N 个位置,块 i 放 (i mod 组数) 组内任一路	折中	折中	组内比 N 个

- 💡 大白话:直接映射=每本书有**固定书架格**;全相联=书能放**任何一格**(找起来要把所有格看一遍);组相联=每本书固定到某一**排**,排内 N 格随便放——现实中主流的折中方案。

- 替换策略:直接映射没得选;组相联/全相联用 **LRU(最近最少使用)** 或随机。

5.3.3 写策略(常考)

- **写直达(write-through)**:每次写 cache **同时写主存**。优点:简单、数据一致;缺点:写很慢 → 配一个**写缓冲(write buffer)**:先丢进缓冲,CPU 继续跑,缓冲慢慢写主存。
- **写回(write-back)**:只写 cache,给块标**脏位(dirty bit)**;该块被替换出去时才写回主存。优点:减少写主存次数;缺点:实现复杂、断电丢数据。
- **写缺失时**:把块取进 cache 再写 = **写分配(write allocate)**;直接写主存不取块 = **非写分配**。常见搭配:**写回 + 写分配**(利用局部性),写直达 + 非写分配(简单)。

5.3.4 性能:AMAT 公式(必考)

$$AMAT = \text{命中时间} + \text{缺失率} \times \text{缺失代价}$$

(Average Memory Access Time, 平均访存时间)

- 例:命中时间 1ns,缺失率 5%,缺失代价 100ns → $AMAT = 1 + 0.05 \times 100 = \mathbf{6ns}$ 。
- 减小缺失率的收益 = 减少的缺失次数 × 缺失代价;优化命中时间则**每次访问**都受益——两条路径要权衡。
- 带多级 cache 的扩展(了解): $AMAT = T_1 + \text{缺失率}_1 \times (T_2 + \text{缺失率}_2 \times \text{主存代价})$ 。

5.3.5 六种基本优化(记住"动什么参数、换什么代价")

1. **增大块** → 降缺失率(空间局部性);块过大 → 缺失代价上升、块数变少冲突变多。
2. **增大 cache** → 降缺失率;但命中时间变长、成本变高。
3. **提高相联度** → 降缺失率;命中时间变长、硬件变贵。
4. **多级 cache** → 降低"缺失代价"(一级小快,二级大兜底)。
5. **读缺失优先于写** → 写缓冲满时,读别等写。
6. **索引 cache 时避免地址翻译**(如用虚拟地址的低位直接索引,高位做标记)→ 降命中时间。

5.4 cache 的十种高级优化(按目标分类记忆)

目标	优化	说明
缩短命中时间	① 小而简单的 cache	一级 cache 要小,否则比二级还慢
	② 路预测(way prediction)	猜是组内哪一路,先读出来再说
增加带宽	③ 流水线化 cache 访问	访问拆多级,提高吞吐
	④ 多 bank cache	组/块分散到多个存储体,并行读
	⑤ 关键字优先	先送出要的那个字,其余后补
降低缺失代价	⑥ 合并写缓冲	缓冲里相邻的写合并成一次
	⑦ 多级 cache	见上
降低缺失率	⑧ 编译器优化	循环交换(按行访问)、分块(blocking)

目标	优化	说明
并行	⑨ 硬件预取	猜测访问模式提前取块
	⑩ 编译器预取	编译器插入预取指令

💡 大白话:**分块(blocking)**优化——矩阵乘法按"一小块一小块"算,让小块数据常驻 cache,是编译器优化的经典例子,理解它就知道"为什么写代码的顺序影响速度"。

5.5 虚拟存储器 ★

5.5.1 动机

1. 让程序以为**独享一个巨大的地址空间**(虚拟地址空间);
2. **保护**:每个进程有自己的地址空间,互相看不到、踩不到;
3. 让主存成为磁盘的"cache"(程序可以比内存大)。

5.5.2 机制:页与页表

- 内存按固定大小的**页(page,典型 4KB)**划分,磁盘上的程序也按页存放。
- **页表(page table)** 记录:虚拟页号 → 物理页号(或"不在主存")。页表存在**主存**里,每个进程一份。
- **地址转换**:

```

虚拟地址 = [虚拟页号] [页内偏移]
      ↓ 查页表
物理地址 = [物理页号] [页内偏移](偏移不变!)

```

- ⚠ 易错点:页内偏移在转换中**保持不变**,只有页号被替换。
- **缺页(page fault)**:要访问的页不在主存(要去磁盘取)→ 触发异常,OS 从磁盘换入该页(代价 = 数百万周期! 所以缺页要极少发生,靠大页 + 好的替换策略)。
- **保护**:页表项带访问权限位(读/写/执行),非法访问(如写只读页)触发异常;每个进程独立页表,天然隔离。
- 虚拟存储的写策略用**写回**(脏位标记改过的页,换出时写回磁盘);另加**引用位(use bit)** 帮助 LRU 替换。

5.5.3 TLB(快表):页表的 cache

- 每次访存都查页表 = 每次访存多一次主存访问,太慢!→ 给页表配一个小 cache,叫 **TLB(Translation Lookaside Buffer)**。
- TLB 里存"最近用过的"页表项(虚拟页号 → 物理页号),典型 16~512 项,全相联或组相联。
- **访存完整流程(必背)**:

```

CPU 给出虚拟地址
      ↓
① 查 TLB(同时用索引字段查 cache)
   └─ TLB 命中 → 得到物理地址 → 访问 cache
   └─ TLB 缺失 → 查内存页表(可能缺页 → OS 处理)
                → 换入 TLB,再重试

```

- **?** TLB 缺失和缺页有什么区别?TLB 缺失只是"页表项不在快表里"(去内存查页表,几~几十周期);**缺页**是"页根本不在内存里"(去磁盘取,几百万周期)。层级不同,别混。
- **⚠** 易错点:TLB 是"**页表条目的 cache**",cache 是"**数据的 cache**",两者作用对象不同,可以同时存在、同时工作。

5.6 通用框架:存储器层次的四个问题(简答万能框架)

任何一级层次结构(寄存器↔cache、cache↔主存、主存↔磁盘)都回答四个问题:

问题	常见答案
Q1 块放在哪?	直接映射 / 全相联 / 组相联
Q2 如何找到块?	索引定位 + 标记比对(虚拟存储:页表;TLB:专用硬件)
Q3 缺失时替换哪块?	LRU / 随机(直接映射没得选)
Q4 写如何处理?	写直达 / 写回(主存↔磁盘几乎总用写回)

★ 考点提示:用这四问描述"cache"或"虚拟存储器"是经典简答题,套框架即可拿满分。

5.7 谬误与陷阱

1. **陷阱:缺失率低 ≠ 性能好。** 还要看缺失代价和命中时间——AMAT 才是综合指标;小 cache 缺失率虽高但命中时间短,可能比大 cache 更快。
2. **陷阱:cache 越大越好。** 大 cache 命中时间变长、功耗变大,一级 cache 尤其要小。
3. **陷阱:用峰值带宽宣传存储器性能。** 实际带宽受访问模式、bank 冲突、刷新等影响,远低于峰值。
4. **陷阱:磁盘"寻道时间"被忽略。** 顺序访问与随机访问磁盘速度差两个数量级,评估磁盘性能必须算上寻道。

五、名词解释汇总表

名词	英文	解释
时间局部性	temporal locality	访问过的数据近期会再访问
空间局部性	spatial locality	数据附近的数据近期会访问
SRAM	Static RAM	6 管/位,快而贵,用作 cache
DRAM	Dynamic RAM	1 管 1 容/位,慢而便宜,需刷新,用作主存
块/行	block / line	cache 与主存交换的最小单位
标记	tag	标识 cache 块对应哪个主存块
索引	index	定位 cache 位置的地址字段
有效位	valid bit	该块数据是否有效
命中率/缺失率	hit/miss rate	命中(缺失)访问占全部访问的比例

名词	英文	解释
命中时间	hit time	命中时访问 cache 的时间
缺失代价	miss penalty	缺失后从下一级取块的时间
直接映射	direct mapped	每主存块只有一个 cache 位置
全相联	fully associative	主存块可放任意位置
组相联	set associative	每组 N 路,折中方案
写直达	write-through	写 cache 同步写主存
写缓冲	write buffer	暂存待写主存数据的队列
写回	write-back	仅写 cache,替换时写回
脏位	dirty bit	标记块被修改过
LRU	Least Recently Used	替换最久未用的块
AMAT	Average Memory Access Time	平均访存时间指标
页	page	虚拟存储的固定大小块(4KB)
页表	page table	虚拟页号→物理页号映射
缺页	page fault	页不在主存,需从磁盘取
TLB	Translation Lookaside Buffer	页表项的 cache(快表)
引用位	use/reference bit	帮助 LRU 替换的标记
预取	prefetching	提前取入可能用到的块
分块	blocking	编译器/程序员的 cache 优化技术

六、公式速查

公式	说明
块数 = cache 容量 ÷ 块大小	直接映射设计
索引位数 = $\log_2(\text{块数} / \text{组数})$	组相联除以路数
偏移位数 = $\log_2(\text{块大小})$	字节单位
标记位数 = 地址位 - 索引位 - 偏移位	地址划分
AMAT = 命中时间 + 缺失率 × 缺失代价	核心公式
cache 总存储 = 块数 × (数据 + 标记 + 有效位 + 脏位)	求 cache 实际面积
多级: $AMAT = T_1 + \text{缺}_1 \times (T_2 + \text{缺}_2 \times T_3)$	两级扩展

七、易混淆概念对比

对比	区别一句话
时间局部性 vs 空间局部性	同一数据再次访问 vs 邻近数据随后访问
SRAM vs DRAM	6 管快而贵 vs 1 管 1 容慢而便宜(要刷新)
cache vs 主存	小而快(SRAM)vs 大而慢(DRAM),cache 是主存的缓存
直接映射 vs 组相联	1 路 vs N 路;冲突多 vs 折中
写直达 vs 写回	每次写都更新主存 vs 替换时才写回(配脏位)
写分配 vs 非写分配	写缺失先取块 vs 直接写主存
TLB vs 页表	页表条目的 cache vs 完整映射表(在主存)
TLB 缺失 vs 缺页	快表没条目(查页表,快)vs 页不在主存(读磁盘,极慢)
虚拟地址 vs 物理地址	程序眼中的地址 vs 真实内存地址
页 vs 块	虚拟存储的传输单位 vs cache 的传输单位
命中时间 vs 缺失代价	每次命中都付 vs 缺失才付
缺失率 vs 缺失次数	比例 vs 绝对次数(优化收益看后者)

八、自测题

1. (概念)举例说明时间局部性和空间局部性。
2. (计算)32 位地址,直接映射 cache 容量 32KB,块大小 16 字节。求:块数、索引位数、偏移位数、标记位数。
3. (计算)64KB 直接映射 cache、块 16B 的 cache,除数据外还要存多少标记+有效位?(给出总 bit 数)
4. (计算)命中时间 2ns,缺失率 10%,缺失代价 200ns,求 AMAT。若把缺失率降到 5%,收益多少?
5. (简答)三种映射方式各有什么优缺点?现实中为什么常用组相联?
6. (简答)写直达和写回的区别?各配什么机制?各有什么利弊?
7. (简答)描述一次访存的完整流程:虚拟地址 $\rightarrow$... $\rightarrow$ 数据(考虑 TLB 和 cache)。
8. (简答)TLB 缺失和缺页有什么区别?
9. (简答)什么是脏位?它在写回策略中的作用?
10. (简答)用"四个问题"框架描述 cache 这一级。
11. (计算)程序访问 1000 次,960 次命中,求缺失率;若每次缺失代价 50ns,总缺失代价占多少时间?

参考答案

1. 时间局部性:循环变量 i 每轮都要读;空间局部性:顺序遍历数组 $A[0]$ 、 $A[1]$ 、 $A[2]$...,访问 $A[i]$ 后接着访问 $A[i+1]$ 。
2. 块数 = $32\text{KB}/16\text{B} = 2048 = 2^{11} \rightarrow$ 索引 11 位;偏移 = $\log_2 16 = 4$ 位;标记 = $32-11-4 = 17$ 位。
3. 块数 = $64\text{KB}/16\text{B} = 4096$;每块:标记 16 位 + 有效位 1 位 = 17 位 $\rightarrow$ 总 = $4096 \times 17 = 69632 \text{ bit} \approx 8.5\text{KB}$ (数据本身 64KB)。
4. $\text{AMAT} = 2 + 0.10 \times 200 = 22\text{ns}$;降到 5%: $\text{AMAT} = 2 + 0.05 \times 200 = 12\text{ns}$,收益 10ns(收益 = 减少的缺失率 $\times$ 缺失代价)。
5. 直接映射:硬件最简单、命中时间短,但冲突缺失多;全相联:缺失率最低,但要同时比所有标记,硬件最贵;组相联折中:缺失率接近全相联、硬件接近直接映射,是性价比之王。
6. 写直达:每次写都更新主存,简单一致,但写流量大 $\rightarrow$ 用写缓冲吸收;写回:只写 cache 并标脏位,替换时才写主存,写次数少但实现复杂、需处理一致性。
7. CPU 给虚拟地址 $\rightarrow$ 查 TLB(命中得物理页号;缺失查页表,若缺页由 OS 从磁盘调入并更新页表/TLB) $\rightarrow$ 物理地址 $\rightarrow$ 查 cache(命中取数;缺失从主存取块换入) $\rightarrow$ 数据送达。
8. TLB 缺失:快表里没有该页表项 $\rightarrow$ 去主存查页表,几到几十周期;缺页:页根本不在主存 $\rightarrow$ 去磁盘读,数百万周期,由 OS 处理。
9. 脏位标记"该块自装入后被写过"。写回策略中:替换时若脏位=1,必须先写回下一级;若干净(没改过)直接丢弃即可,省一次写。
10. Q1:直接映射/组相联/全相联;Q2:索引定位 + 标记比对;Q3:LRU 或随机(直接映射无需选择);Q4:写直达或写回。
11. 缺失率 = $40/1000 = 4\%$;总缺失代价 = $40 \times 50\text{ns} = 2000\text{ns}$ 。

九、本章小结

- **局部性是因,层次结构是果**:时间+空间局部性让"小 cache + 大主存"的组合接近"又快又大"的理想。
- **cache 三件套**:地址划分(标记/索引/偏移)、三种映射、两种写策略——合起来就能设计一个 cache。
- **AMAT 是评价一切 cache 优化的尺子**。
- **虚拟存储器 = 把主存当磁盘的 cache**:页表做映射、TLB 加速、缺页兜底、顺带实现进程隔离。

下一章 $\rightarrow$ [06-第6章-并行处理器.md](#):多核时代,性能从哪里来?

第 6 章 从客户端到云的并行处理器

一句话导读:单核已经到极限(功耗墙),计算机进入"人多力量大"时代。这一章讲清楚:为什么并行能加速、为什么并行有天花板(Amdahl 定律)、多核/GPU/集群各自怎么干活。

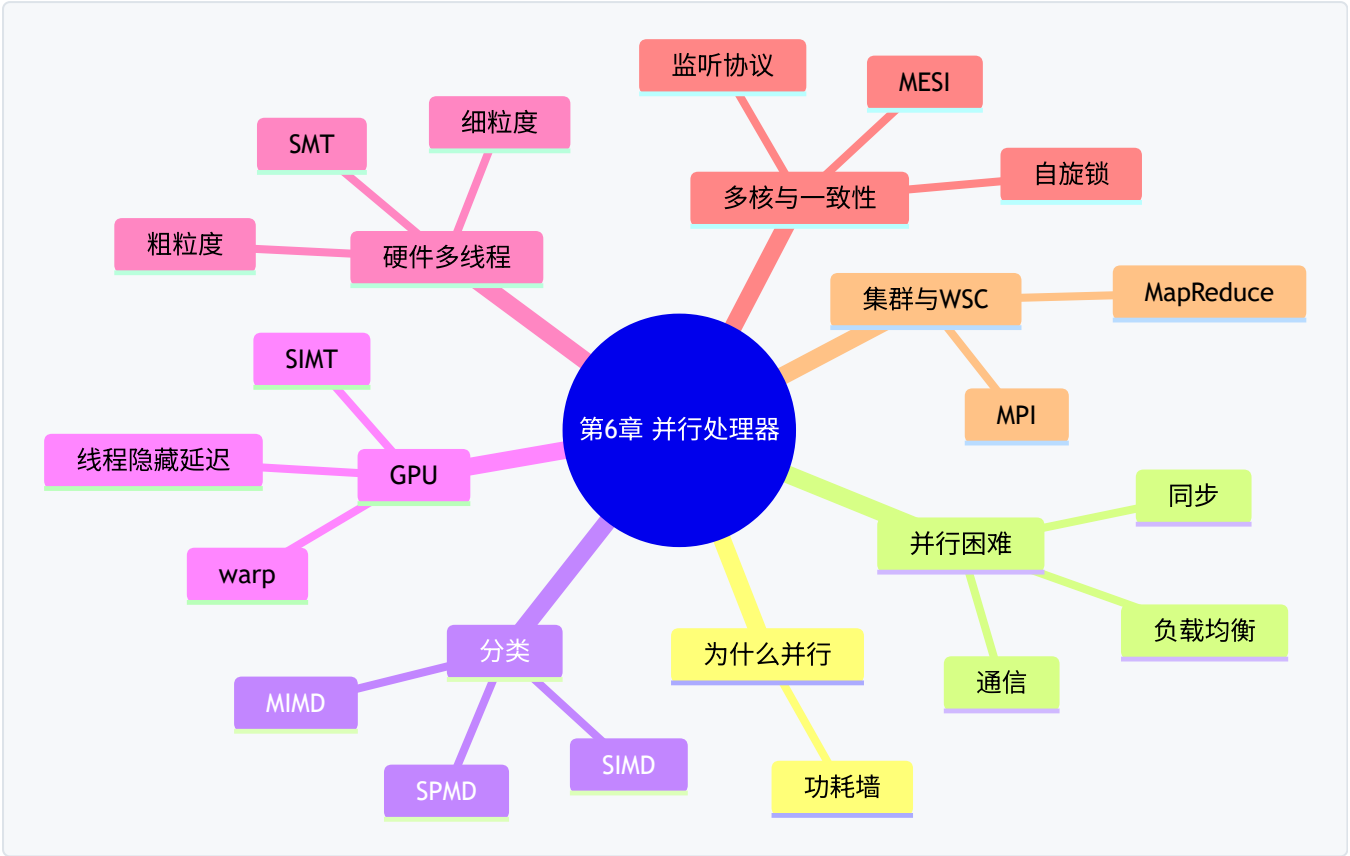
对应教材:第 6 章「从客户端到云的并行处理器(Parallel Processors from Client to Cloud)」

一、本章思维导图

第6章 并行处理器

- 6.1 为什么走向并行
 - 功耗墙(第1章)+ 单核 ILP 挖尽
 - 方向:显式并行(程序自己拆分)
- 6.2 Amdahl 定律 ★(并行的天花板)
 - 加速比 = $1 / [(1-P) + P/N]$
 - P = 可并行部分占比, N = 处理器数
 - 推论:串行部分决定极限,核再多也白搭
- 6.3 并行编程的三大困难
 - 同步(synchronization):步调要一致
 - 通信(communication):数据要共享
 - 负载均衡(load balancing):活要分得匀
- 6.4 并行硬件分类(Flynn)
 - SISD:单指令单数据(传统单核)
 - SIMD:单指令多数据(向量机、GPU)
 - MISD:罕见,基本不存在
 - MIMD:多指令多数据(多核)
 - SPMD:MIMD 上"同一程序、不同数据"
- 6.5 向量架构与 GPU
 - 向量机:一条指令算一整排数据
 - GPU:SIMT 模型(线程块、warp)
 - GPU 用"海量线程切换"隐藏访存延迟
 - GPU vs CPU:核多而简单 vs 核少而强
- 6.6 硬件多线程(在一个核里藏多个线程)
 - 细粒度:每条指令轮换线程
 - 粗粒度:长暂停(访存)时才换
 - 同时多线程 SMT:同周期多线程共占发射槽
- 6.7 多核处理器与缓存一致性 ★
 - 共享内存多处理器(UMA)
 - 一致性问题:两个核的 cache 各存一份
 - 监听协议(snooping):总线广播 + 写无效
 - MESI 状态:Modified/Exclusive/Shared/Invalid
 - 同步:原子操作(ll/sc)+ 自旋锁
- 6.8 集群与仓库级计算机(WSC)
 - 消息传递(message passing)、MPI
 - WSC:海量廉价服务器 + 网络
 - MapReduce:映射 + 归约
 - 关键指标:每瓦性能、PUE
- 6.9 谬误与陷阱
 - 忽略 Amdahl 定律的串行瓶颈
 - 峰值性能 ≠ 实际性能(通信开销!)
 - 并行提升的是吞吐率,不是单任务响应时间

Mermaid 版(可渲染)



二、学习目标

1. 解释为什么计算机行业从单核转向多核。
2. **会算 Amdahl 定律**:给定可并行比例和核数,求加速比与极限(必考)。
3. 说出并行编程的三大困难。
4. 解释 Flynn 分类(SISD/SIMD/MIMD)与 SPMD 的关系。
5. 解释 GPU 与 CPU 的架构差异,以及 GPU 如何隐藏访存延迟。
6. 对比三种硬件多线程(细粒度/粗粒度/SMT)。
7. 解释缓存一致性问题,描述监听协议和 MESI 状态。
8. 解释为什么需要原子操作和自旋锁。
9. 对比共享内存(多核)与消息传递(集群)两种并行方式。
10. 说出 MapReduce 的基本思想。

三、核心名词先混个脸熟

名词	一句话
显式并行	程序(人/编译器)自己拆分的并行

名词	一句话
Amdahl 定律	串行部分决定并行加速上限的定律
同步(synchronization)	让各线程步调一致(锁等)
负载均衡(load balance)	把工作量均分给各处理器
SIMD	一条指令同时算多个数据
MIMD	多个处理器各跑各自的指令流
SPMD	同一程序、不同数据(编程模型)
SIMT	GPU 的"单指令多线程"执行模型
warp	GPU 中一起执行的一组线程(32 个)
线程块(thread block)	GPU 调度的基本单元
硬件多线程	一个核同时保存多个线程的现场
SMT	同时多线程:同周期发射多线程的指令
共享内存多处理器	多核共享同一地址空间
缓存一致性(cache coherence)	保证各核看到一致的共享数据
监听协议(snooping)	通过总线监听维护一致性
写无效(write invalidate)	某核写块时让其他核的副本失效
MESI	缓存块的四种状态协议
原子操作(atomic)	不可被中途打断的操作(第2章 ll/sc)
自旋锁(spin lock)	反复检查锁变量直到拿到的锁
消息传递(message passing)	并行单元靠收发消息通信
MPI	消息传递编程接口标准
仓库级计算机(WSC)	数据中心规模的计算机
MapReduce	"映射+归约"的大数据编程模型
PUE	数据中心电源使用效率指标

四、分节精讲

6.1 为什么走向并行

- 第 1 章的**功耗墙**断了"提高主频"的路;第 4 章的**指令级并行(ILP)**已接近挖尽(单核内能并行的指令就那么多)。

- 于是行业转向:在一个芯片里放多个核(multicore),靠**显式并行**(程序把任务拆给多个核)提升性能。
- 代价:并行编程难得多——程序员/编译器要自己管理任务的拆分与协作。

6.2 Amdahl 定律 ★:并行的天花板

公式(必背):

加速比 = $1 / [(1 - P) + P/N]$
P = 可并行的部分所占比例
N = 处理器(核)数量

- **极限:** $N \rightarrow \infty$ 时,加速比 $\rightarrow 1/(1-P)$ 。串行部分 $1-P$ 是永远的天花板!
- 例(必会):程序 60% 可并行($P=0.6$):
 - 4 核: $1/(0.4 + 0.6/4) = 1/0.55 \approx 1.82$ 倍
 - 8 核: $1/(0.4 + 0.6/8) = 1/0.475 \approx 2.11$ 倍
 - 无穷核: $1/0.4 = 2.5$ 倍——100 核也只有 2.5 倍,其余全浪费!
- 💡 大白话:10 个人搬 100 箱货,其中 40 箱必须"一个人签收"(串行),剩下 60 箱随便分。哪怕雇 1000 个人,签收那 40 箱的时间省不掉,总时间最多缩短到 2.5 倍。
- ⚠ 易错点:P 是**可并行部分的时间占比**,不是"代码行数占比"。加速比按**时间**算。

6.3 并行编程的三大困难

1. **同步(synchronization):**线程间要"对齐节奏"(如生产消费、临界区互斥)→ 锁、屏障。同步开销大,且易死锁。
2. **通信(communication):**线程间要交换数据 → 通信延迟远大于计算,分布式系统里更是数量级之差。
3. **负载均衡(load balancing):**任务分不匀,快的核干等慢的核(木桶效应)。

6.4 并行硬件分类:Flynn 分类法

类别	全称	说明	例子
SISD	单指令流单数据流	传统串行单核	老式 CPU
SIMD	单指令流多数据流	一条指令同时算一排数据	向量机、GPU、SSE
MISD	多指令流单数据流	罕见,基本无实例	—
MIMD	多指令流多数据流	各处理器跑各自的指令	多核、集群

- **SPMD(Single Program Multiple Data):**一种**编程模型**——所有处理器跑**同一份程序**,但各自处理**不同的数据**(如各自处理数组的一段)。它通常跑在 MIMD 硬件上,却写出 SIMD 的效果。
- ⚠ 易错点:SIMD/MIMD 是**硬件分类**;SPMD 是**编程模型**,不是硬件类别。选择题常挖坑。

6.5 向量架构与 GPU

6.5.1 向量机(传统 SIMD)

- 用**向量寄存器**一次装一排数据(如 64 个双精度数),一条向量指令把整排一次算完。
- 优点:取指/译码开销摊薄到几十个数据上;访存模式规整(整块连续读)。
- 现代 CPU 的 **SSE/AVX** 就是"寄存器内的 SIMD"(呼应第 3 章子字并行)。

6.5.2 GPU:数据并行的王者 ★

- **用途**:图形渲染 = 海量像素的**同一种运算** → 天然数据并行。后扩展到通用计算(GPGPU,如 CUDA)。
- **执行模型 SIMT(单指令多线程)**:程序员写一个**核函数(kernel)**,交给成千上万个**线程**跑,每个线程处理不同数据。硬件把相邻线程打包成 **warp(32 个线程一组)**,一组一组地"同一条指令、不同数据"执行——像 SIMD,但程序员看到的是线程。
- **调度**:GPU 把计算组织成**线程块(thread block)**,块分到各个流多处理器上执行。
- **GPU 如何隐藏访存延迟?** GPU 核不做大 cache、不搞乱序执行,而是**靠海量线程**:一个 warp 等内存时,硬件瞬间切换到另一个 warp 继续算——用**"换线程"代替"等"**。
- 💡 大白话:CPU 像少数几个"高手厨子"(大 cache、乱序执行,擅长应对复杂多变的菜);GPU 像几千个"切菜工",每人只会一刀切,但架不住人多——一个工人去拿菜的工夫,立刻换下一个工人切。

6.5.3 GPU vs CPU(常考对比)

	CPU	GPU
核数	少(几个~几十)	多(几千)
单核能力	强(乱序、大 cache、高主频)	弱(简单、顺序)
擅长	复杂控制流、任务并行	海量数据并行
隐藏延迟	cache + 乱序执行	海量线程切换

6.6 硬件多线程:一个核里"分身"

一个核同时保存**多个线程的现场**(PC、寄存器),遇到暂停立刻换线程干:

方式	切换时机	特点
细粒度(fine-grained)	每条指令 之间轮换	能隐藏所有短暂停(如访存),但可能拖慢单线程
粗粒度(coarse-grained)	只在 长暂停 (如二级 cache 缺失)时切换	切换开销大(要排空流水线),但平时不打扰
同时多线程(SMT)	同一周期内 多个线程的指令混着发射	充分利用超标量的空槽,利用率最高

- 价值:**隐藏访存/分支的暂停**,把流水线的空槽填满——用"别人的活"填"我的等待"。
- 现实:Intel 的**超线程(Hyper-Threading)** 就是双线程 SMT。

6.7 多核处理器与缓存一致性 ★

6.7.1 共享内存多处理器

- 多个核共享**同一个物理地址空间**(共享内存,UMA:统一内存访问),通过读写共享变量协作。
- 每个核有自己的私有 cache → **缓存一致性问题(cache coherence)**:核 A 改了共享变量 x(在自己的 cache 里),核 B 的 cache 里还存着 x 的旧值 → 两个核看到的世界不一致!

6.7.2 监听协议(snooping)与写无效

- **监听协议**:所有核通过**总线/广播网络**互相监听对方的访存行为;某核要写一个块时,广播通知,其他核把该块的副本**作废(写无效, write invalidate)**;之后谁再读,发现无效就从"最新拥有者"那里取。
- **MESI 协议**(最经典,四个状态):

状态	含义
M(Modified,已修改)	本 cache 有 最新且独占 的副本,和主存不一致
E(Exclusive,独占)	本 cache 独占,但和主存一致
S(Shared,共享)	多个 cache 都有副本,都和主存一致
I(Invalid,无效)	本副本作废,不可用

- 💡 大白话:图书馆一本书(memory 里的数据)被借给一个学生,他做了批注(Modified)——其他学生再借,必须先拿他那份"带批注的"复印(从 Modified 者取),他手里的旧复印件就作废(Invalid)。

6.7.3 同步:原子操作与自旋锁

- 多核同步的基础是**原子操作**(第 2 章的 ll/sc):保证"读-改-写"不被打断。
- **自旋锁(spin lock)**:共享变量 lock=0 表示空闲。拿锁: ll 读 lock;是 0 就 sc 写 1,成功则进临界区;失败或被占就**原地打转(自旋)**重试。用完写回 0 释放。
- ⚠️ 陷阱:自旋锁在"争用激烈"时会浪费大量总线带宽;且锁设计不好会**死锁**。

6.8 集群与仓库级计算机(WSC):并行到云端

- **集群(cluster)**:通过网络把大量独立计算机(节点)连起来协同工作。节点间**没有共享内存**,靠**消息传递(message passing)**通信,编程标准是 **MPI**。
- **仓库级计算机(Warehouse Scale Computer)**:数据中心规模的"一台计算机"——数万台廉价服务器 + 交换机 + 存储,追求**每瓦性能**(能耗是钱!) and **PUE**(电源使用效率,总耗电/IT设备耗电,越接近 1 越好)。
- **MapReduce(大数据编程模型)**:把大规模数据处理拆成两步:
 1. **Map(映射)**:每台机器对本地数据做同一处理,产出"键-值"对;
 2. **Reduce(归约)**:把相同键的值汇总。
 - 例子:数单词——Map 阶段每台机器数自己文件的单词,Reduce 阶段把同一个单词的计数加总。
- 💡 大白话:MapReduce 像"全校统计运动服尺码":每个班主任(Map)统计本班的尺码表,教务处(Reduce)把各班同尺码的人数加起来。

6.9 谬误与陷阱

- 1. **陷阱:忽略 Amdahl 定律。** 只盯着"可并行部分"堆核数,串行 10% 的程序上限就是 10 倍,再多核都是摆设。
- 2. **谬误:峰值性能等于实际性能。** 通信、同步、负载不均让实际加速远低于"核数×单核峰值"。
- 3. **陷阱:用"每秒浮点运算峰值"比机器。** 达到峰值需要极端规整的访问模式,真实程序远达不到。
- 4. **陷阱:并行提升的是吞吐率,不是单任务的响应时间。** 100 个任务分给 100 核很快,但"一个任务"可能根本拆不开——单任务延迟不降反升(通信开销)。

五、名词解释汇总表

名词	英文	解释
Amdahl 定律	Amdahl's Law	加速比上限 = $1/(1-P)$
显式并行	explicit parallelism	由程序员/编译器拆分的并行
Flynn 分类	Flynn's taxonomy	按指令流/数据流把计算机分 SISD/SIMD/MISD/MIMD
SIMD	Single Instruction, Multiple Data	一条指令算多个数据
MIMD	Multiple Instruction, Multiple Data	各处理器独立执行
SPMD	Single Program, Multiple Data	同程序、异数据的编程模型
SIMT	Single Instruction, Multiple Threads	GPU 的执行模型
warp	—	GPU 中同步执行的一组线程(32)
线程块	thread block	GPU 调度单位
硬件多线程	hardware multithreading	一个核同时保存多线程现场
同时多线程	SMT	同周期发射多线程指令
共享内存多处理器	shared-memory multiprocessor	多核共享一个地址空间
缓存一致性	cache coherence	各核看到的共享数据保持一致
监听协议	snooping protocol	总线监听 + 写无效
MESI	—	Modified/Exclusive/Shared/Invalid 状态协议
原子操作	atomic operation	不可中断的读改写(ll/sc)
自旋锁	spin lock	反复尝试直到拿到锁
消息传递	message passing	靠收发消息协作(集群)

名词	英文	解释
MPI	Message Passing Interface	消息传递编程标准
仓库级计算机	WSC	数据中心级计算机
MapReduce	—	映射+归约的分布式编程模型
PUE	Power Usage Effectiveness	数据中心总耗电 ÷ IT 设备耗电

六、公式速查

公式	说明
加速比 = $1 / [(1-P) + P/N]$	Amdahl 定律
加速比极限 = $1 / (1-P)(N \rightarrow \infty)$	串行天花板
$P = \text{可并行部分时间} \div \text{总时间}$	P 的定义
实际性能 $\leq \min(\text{峰值}, \text{Amdahl 上限})$	综合约束

七、易混淆概念对比

对比	区别一句话
SIMD vs MIMD	硬件分类:一条指令算多数据 vs 多处理器各干各的
SPMD vs SIMD	编程模型(程序层面)vs 硬件类别
SIMT vs SIMD	GPU 的线程视角 vs 传统向量机视角
多核 vs GPU	核少而强(任务并行)vs 核多而弱(数据并行)
细粒度 vs 粗粒度多线程	每条指令轮换 vs 长暂停才换
SMT vs 粗粒度多线程	同周期并行发射 vs 暂停时整核切换
共享内存 vs 消息传递	多核(同一地址空间)vs 集群(收发消息)
缓存一致性 vs 缓存缺失	多核间副本一致 vs 单核内没命中
TLB vs MESI	地址翻译的 cache vs 一致性的状态协议
原子操作 vs 自旋锁	硬件提供的不可打断操作 vs 用原子操作实现的软件锁
加速比 vs 吞吐率	单任务提速倍数 vs 单位时间任务数
响应时间 vs 加速比	绝对时间 vs 相对倍数

八、自测题

1. (计算)程序 75% 可并行。求 4 核、16 核的加速比,以及核数无限时的极限。
2. (概念)Amdahl 定律的核心结论是什么?为什么它是"并行的天花板"?
3. (简答)并行编程有哪三大困难?各举一例。
4. (简答)用 Flynn 分类法说明 SISD、SIMD、MIMD,并说明 SPMD 属于哪一类。
5. (简答)GPU 和 CPU 的架构差异?GPU 靠什么隐藏访存延迟?
6. (简答)三种硬件多线程(细粒度/粗粒度/SMT)的切换时机有何不同?
7. (简答)什么是缓存一致性问题?监听协议如何解决?写无效是什么意思?
8. (简答)MESI 四个状态各表示什么?
9. (简答)什么是自旋锁?它依赖什么硬件特性?
10. (简答)多核(共享内存)与集群(消息传递)的本质区别?
11. (简答)MapReduce 的两个阶段各做什么?举一个例子。
12. (概念)为什么"峰值性能"不等于实际性能?

参考答案

1. 4 核: $1/(0.25 + 0.75/4) = 1/0.4375 \approx 2.29$ 倍;16 核: $1/(0.25 + 0.75/16) = 1/0.2969 \approx 3.37$ 倍;极限: $1/0.25 = 4$ 倍。
2. 加速比 $= 1/[(1-P)+P/N]$;串行部分 $1-P$ 无法被核数消除, $N \rightarrow \infty$ 时上限为 $1/(1-P)$ 。串行 1%,上限就是 100 倍——再堆核也突破不了。
3. 同步:线程要互相等(锁/屏障,易死锁);通信:共享/传输数据有延迟(分布式下更甚);负载均衡:任务分不匀,快的等慢的。
4. SISD:单指令单数据(传统串行);SIMD:单指令多数据(向量机/GPU);MIMD:多指令多数据(多核)。SPMD 是 **编程模型**(同程序异数据),不是硬件类别,通常跑在 MIMD 硬件上。
5. CPU:核少而强(乱序执行、大 cache、高主频),擅长复杂控制流;GPU:几千个简单核,擅长海量数据并行。GPU 靠**海量线程切换**隐藏访存延迟——一个 warp 等内存就换另一个 warp 上,不用大 cache 和乱序。
6. 细粒度:每条指令之间轮换(隐藏短暂停);粗粒度:仅在长暂停(如 L2 缺失)时切换(切换代价大);SMT:同一周期内多个线程的指令同时发射,填满超标量空槽。
7. 问题:各核私有 cache 存了同一共享数据的副本,一个核改了自己副本,其他核读旧值。监听协议:写时广播,其他核把该块的副本作废(写无效);之后谁读谁从最新处取。
8. M:本副本最新且独占(与主存不一致);E:独占且与主存一致;S:多副本、与主存一致;I:副本已作废。
9. 自旋锁:用共享变量标记锁状态,拿不到锁就原地循环重试。依赖**原子操作**(ll/sc 或等价物)保证"检查并置位"不被别的核插进来。
10. 多核共享同一物理内存(共享地址空间),通过共享变量协作,要解决一致性;集群节点各自有内存,靠网络消息传递(MPI)协作,天然无一致性问题但有通信延迟。
11. Map:各机器对本地数据做同样处理产出键值对(如各数各的文件里的单词);Reduce:按键汇总(如同单词计数相加)。

12. 峰值假设所有部件满负荷、数据规整;实际受 Amdahl 串行部分、通信/同步开销、负载不均、访存延迟限制,远达不到峰值。

九、本章小结

- **Amdahl 定律是并行的宪法**:先算串行部分,再决定值不值得并行。
- **谱系**:核内(ILP、SMT)→ 片内(多核、GPU)→ 机间(集群)→ 仓库级(WSC),并行尺度逐级放大,通信代价也逐级放大。
- **两个基本功**:多核的缓存一致性(MESI),和并行的三大困难(同步/通信/负载均衡)。

辅助学习 → [07-附录-逻辑设计基础.md](#):补上第 4 章需要的电路基础(0 基础必读)。

附录 B 逻辑设计基础(0 基础必读)

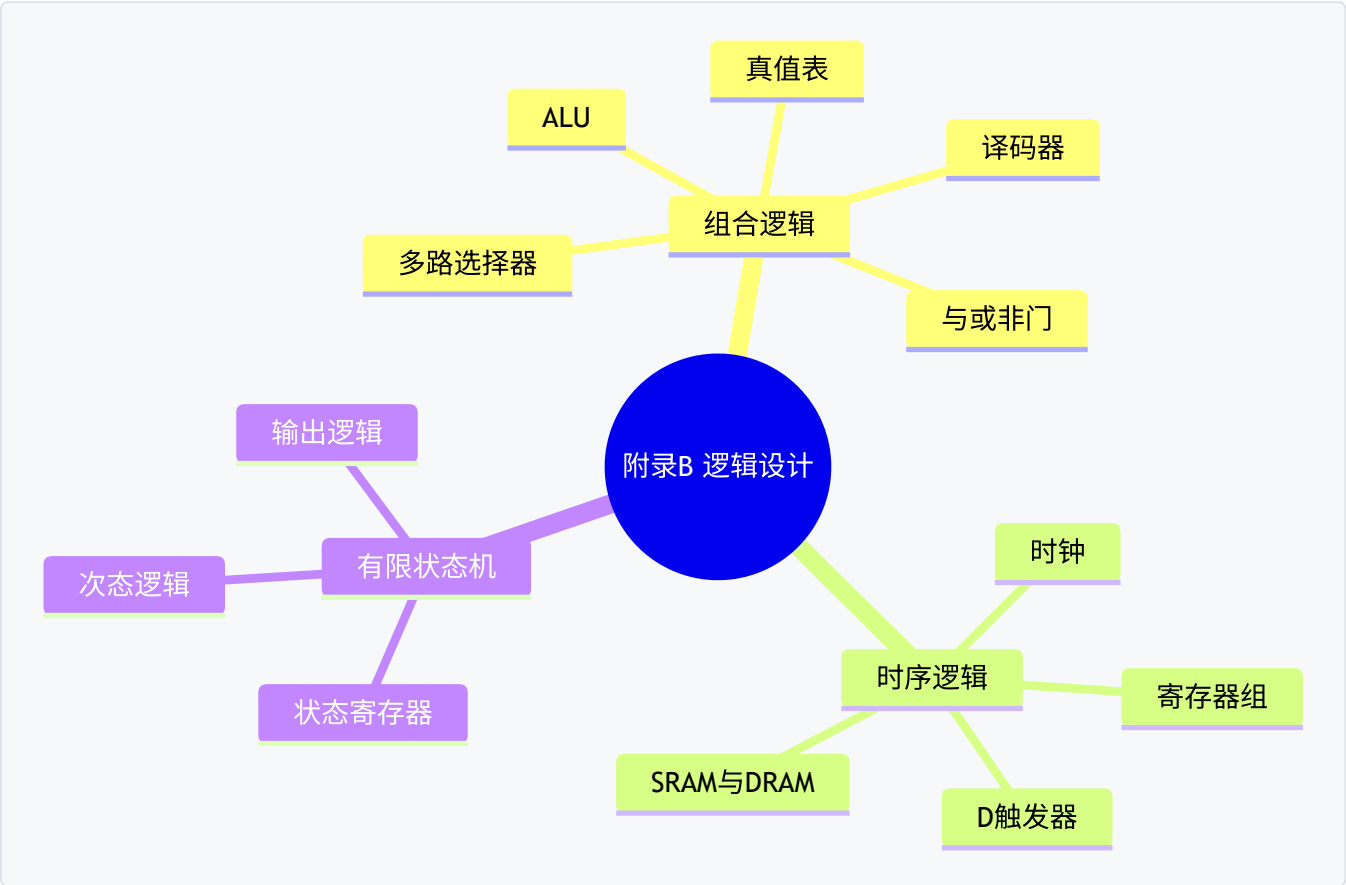
一句话导读:第 4 章的数据通路图里全是与门、触发器、多路选择器——这一篇用最少的篇幅把它们讲明白。**学第 4 章之前,先花半天读这篇**,数据通路就不再是天书。

对应教材:附录 B「逻辑设计基础(The Basics of Logic Design)」

一、本章思维导图

附录B 逻辑设计基础

- 组合逻辑(无记忆:输出只看当前输入)
 - 基本门:与 AND / 或 OR / 非 NOT(能组合出一切)
 - 真值表:穷举输入输出
 - 布尔代数:化简逻辑表达式
 - 译码器(decoder): n 位输入 $\rightarrow 2^n$ 路输出选1
 - 多路选择器(MUX):从多路输入选1路输出
 - ALU = 加法器 + 逻辑运算 + MUX 选择
- 时序逻辑(有记忆:输出还看历史状态)
 - 时钟(clock):节拍,边沿触发
 - D 触发器:时钟沿到来 $\rightarrow$ D 存入 Q
 - 寄存器 = 一排 D 触发器(PC 就是寄存器)
 - 寄存器组:读2写1 + 写使能(第4章主角)
 - SRAM:电平触发(小容量缓存)
 - DRAM:边沿触发(大容量主存)
- 有限状态机(FSM):时序逻辑的"大脑"
 - 三件套:状态寄存器 + 次态逻辑 + 输出逻辑
 - 工作方式:时钟沿 $\rightarrow$ 按当前状态+输入 更新状态
 - 用途:控制器(第4章)、cache 控制(附录D)
- 与全书的关系
 - 数据通路 = 组合逻辑 + 时序逻辑
 - 控制器 = 有限状态机



二、学习目标

- 1. 会写与/或/非门的真值表,会化简简单逻辑表达式。
- 2. 理解译码器(decoder)和多路选择器(MUX)的作用。
- 3. 理解时钟与边沿触发,会描述 D 触发器的行为。
- 4. 理解寄存器组的"读 2 写 1"接口(第 4 章数据通路的基础)。
- 5. 区分 SRAM(电平触发)与 DRAM(边沿触发)。
- 6. 说出有限状态机(FSM)的三件套和工作方式。
- 7. 理解"数据通路 = 组合 + 时序,控制器 = FSM"这个关系。

三、核心名词先混个脸熟

名词	一句话
组合逻辑	输出只取决于当前输入,没有记忆
时序逻辑	输出取决于输入和内部状态,有记忆
真值表	穷举所有输入组合及其输出

名词	一句话
译码器(decoder)	把 n 位输入翻译成 2^n 路中选 1 路
多路选择器(MUX)	用选择信号从多路输入中选 1 路
时钟(clock)	周期性跳变的信号,节拍器
边沿触发	只在时钟跳变瞬间采样
D 触发器	时钟沿到来时把 D 存入 Q
寄存器(register)	一组 D 触发器,存多位数据
写使能(write enable)	控制"写不写"的信号
寄存器组(register file)	32 个寄存器的阵列,读 2 写 1
SRAM	电平触发的存储(小容量、快)
DRAM	边沿触发的存储(大容量、需刷新)
有限状态机(FSM)	状态 + 输入 $\rightarrow$ 新状态 + 输出的控制模型
状态寄存器	FSM 中保存当前状态的部分

四、分节精讲

4.1 组合逻辑:没有记忆的电路

三种基本门(理论上能搭出任何数字电路):

门	符号	真值表	大白话
与 AND	$A \cdot B$	$1 \cdot 1 = 1$, 其余 = 0	两个开关都开,灯才亮
或 OR	$A + B$	$0 + 0 = 0$, 其余 = 1	任一开关开,灯就亮
非 NOT	$\bar{A}$	$0 \leftrightarrow 1$	取反

- **真值表**:把输入的所有组合和对应输出列成表,是描述和设计组合逻辑的基本工具。
 - **布尔代数**:用代数法则化简表达式(如 $A + A \cdot B = A$),让电路更小更快。
 - **译码器(decoder)**:输入 n 位二进制,输出 2^n 根线,恰好 1 根为 1——像"点名",点到谁谁响应。第 4 章里用来选寄存器。
 - **多路选择器(MUX, multiplexer)**:输入多路数据 + 1 个**选择信号**,输出被选中的那一路。**数据通路里到处都是它**(选择 ALU 的输入来源、选择写回数据等)。
 - **ALU**:把加法器、与门、或门等的结果全部算出来,再用一个 MUX 按控制信号**挑一个**输出——"全算好,再选择",简单粗暴但通用。
- 💡 大白话:组合逻辑像**自动售货机**:投币(输入)立刻掉饮料(输出),不记得你上次买了什么。

4.2 时序逻辑:有记忆的电路

时钟(clock):周期性在 0/1 之间跳变的信号。电路只在**跳变瞬间(边沿)**更新状态——所以整个系统"步调一致",这就是第 1 章"时钟周期"的物理来源。

D 触发器(D flip-flop):1 位的记忆单元。**时钟沿到来时,把输入 D 的值存入 Q**;其他时间 Q 保持不变。

D → |D触发器| → Q
 ↑时钟沿

- **寄存器** = 32 个 D 触发器排一排,共用一个时钟和写使能 → 一次存 32 位。**PC 就是一个寄存器。**
- **寄存器组(register file)**:第 4 章的核心部件,接口是:
 - 输入:**两个读寄存器号(各 5 位) + 一个写寄存器号 + 32 位写数据 + 写使能**;
 - 输出:**两个 32 位读数据**。
 - 可以**同时**读两个寄存器、写一个寄存器(这就是第 4 章数据通路"读 rs、rt,写 rd"的硬件支撑)。
- **SRAM**:以**电平**触发方式存储(类似锁存器),快、贵 → 小容量 cache 用它。
- **DRAM**:以**边沿**触发方式存储,慢、便宜、要刷新 → 主存用它(第 5 章)。

⚠ 易错点:组合逻辑"输入变输出立刻变";时序逻辑"输出只在时钟沿才变"。判断一个部件是哪类,就看它**要不要时钟**。

4.3 有限状态机(FSM):时序逻辑的"大脑"

三件套:

输入 → [次态逻辑] → [状态寄存器] → [输出逻辑] → 输出
 ↑_____ | (当前状态)

1. **状态寄存器**:存"当前状态"(如取指态、执行态);
2. **次态逻辑(组合)**:根据"当前状态 + 输入"算出**下一个状态**;
3. **输出逻辑(组合)**:根据"当前状态(+输入)"产生输出信号。

工作方式:每个时钟沿,状态寄存器更新为新状态 → 输出随之变化。

- **用途**:第 4 章的**控制器**本质上就是一个 FSM(多周期实现/异常处理都要它);附录 D 还用 FSM 控制 cache。
- 💡 大白话:FSM 像**电梯**:状态=楼层,输入=按键,次态逻辑=决定下一层,输出=开门/移动。它"记得自己在几楼"——这就是"有记忆"。

4.4 与全书的关系(一句话打通)

数据通路(datapath)= 组合逻辑(ALU、MUX、译码器)
 + 时序逻辑(寄存器、寄存器组、存储器)
控制器(control) = 有限状态机(FSM)
两者合起来 = CPU(第4章的全部内容)

五、名词解释汇总表

名词	英文	解释
组合逻辑	combinational logic	无记忆电路,输出只由当前输入决定
时序逻辑	sequential logic	有记忆电路,输出还与历史状态有关
真值表	truth table	穷举输入组合与输出的表
译码器	decoder	n 位输入 $\rightarrow 2^n$ 选 1 输出
多路选择器	multiplexer (MUX)	按选择信号从多路输入选一路
时钟	clock	系统节拍信号
边沿触发	edge-triggered	仅在时钟跳变沿采样输入
D 触发器	D flip-flop	1 位存储:沿到来时 $D \rightarrow Q$
寄存器	register	一组 D 触发器组成的多位存储
写使能	write enable	允许写入的控制信号
寄存器组	register file	多寄存器的读写阵列(读2写1)
SRAM	Static RAM	电平触发存储,快而贵
DRAM	Dynamic RAM	边沿触发存储,慢而便宜,需刷新
有限状态机	FSM	状态+输入驱动跳转的控制模型
次态逻辑	next-state logic	FSM 中计算下一状态的组合逻辑
输出逻辑	output logic	FSM 中产生输出的组合逻辑

六、易混淆概念对比

对比	区别一句话
组合逻辑 vs 时序逻辑	无记忆 vs 有记忆;不需要时钟 vs 需要时钟
译码器 vs 多路选择器	1 路激活"选地址" vs 多路输入"选数据"
锁存器(latch) vs 触发器(flip-flop)	电平敏感(整个高电平都透传)vs 边沿触发(只在沿上采样)
SRAM vs DRAM	电平触发快而贵 vs 边沿触发慢而便宜
状态寄存器 vs 普通寄存器	存 FSM 当前状态 vs 存普通数据(本质同是触发器)
次态逻辑 vs 输出逻辑	决定"下一个状态" vs 决定"当前输出"

七、自测题

1. (真值表)写出 2 输入与门、或门、异或门的真值表。
2. (化简)用布尔代数化简 $A + A \cdot B$ 。
3. (概念)3 位译码器有几根输出线?任意时刻有几根为 1?
4. (概念)4 选 1 MUX 需要几根选择信号线?
5. (概念)描述 D 触发器在时钟沿前后的行为。
6. (概念)寄存器组"读 2 写 1"接口:哪些输入、哪些输出?
7. (概念)FSM 由哪三部分组成?各起什么作用?
8. (判断)ALU 是组合逻辑还是时序逻辑?PC 呢?
9. (概念)为什么需要时钟?
10. (概念)第 4 章的控制单元可以看作什么?(提示:FSM)

参考答案

1. 与:00→0,01→0,10→0,11→1;或:00→0,其余→1;异或:00→0,01→1,10→1,11→0。
2. $A + A \cdot B = A \cdot (1 + B) = A$ (吸收律:只要 A 为 1,结果必为 1)。
3. 8 根输出线,恰好 1 根为 1。
4. $\log_2 4 = 2$ 根。
5. 时钟沿到来瞬间:D 的值被"锁存"进 Q;沿过后 Q 保持不变直到下一个沿(即使 D 再变也不影响 Q)。
6. 输入:读寄存器号 1、读寄存器号 2(各 5 位)、写寄存器号(5 位)、写数据(32 位)、写使能;输出:读数据 1、读数据 2(各 32 位)。可同时读 2 个、写 1 个。
7. 状态寄存器(存当前状态)+ 次态逻辑(按当前状态和输入算下一状态)+ 输出逻辑(按状态产生输出)。
8. ALU 是组合逻辑(输出即时跟随输入);PC 是时序逻辑(寄存器,时钟沿才更新)。
9. 让所有有状态部件在统一节拍下更新,避免信号先后到达造成混乱——系统按"心跳"步调一致地前进。
10. 有限状态机(FSM):当前状态=取指/译码/执行...,输入=指令的 opcode,输出=各控制信号。

八、本章小结

- **组合逻辑搭"运算",时序逻辑存"状态",FSM 做"指挥"**——这就是全部数字电路的三大件。
- 学完这篇,第 4 章的数据通路图应该能看懂每个部件了:寄存器组(读2写1)、ALU(组合逻辑+MUX)、PC(寄存器)、控制器(FSM)。

复习冲刺 → [08-全书公式与名词速查卡.md](#)

全书公式与名词速查卡(考前冲刺用)

把六章 + 附录的全部公式、指令表、寄存器表、名词索引浓缩成一张卡。 **考试前 30 分钟过一遍这里就够了。**

一、全部公式一览

第 1 章 性能与成本

公式	说明
时钟频率 = 1 / 时钟周期	互为倒数
CPU 执行时间 = 指令数 × CPI × 时钟周期	全书第一公式
性能X ÷ 性能Y = 执行时间Y ÷ 执行时间X	"n 倍快"
MIPS = 指令数 ÷ (执行时间 × 10 ⁶)	有缺陷的指标
芯片成本 = 晶圆成本 ÷ (每晶圆芯片数 × 成品率)	制造成本
动态功耗 ∝ 容性负载 × 电压 ² × 开关频率	功耗墙

第 3 章 算术

公式	说明
$a - b = a + (\sim b + 1)$	补码减法
溢出 ⇔ 正+正→负 或 负+负→正	符号判定
值 = $(-1)^S \times (1.Frac) \times 2^{(Exp - Bias)}$	规格化浮点数
值 = $(-1)^S \times (0.Frac) \times 2^{(1 - Bias)}$	非规格化数
Exp = 真指数 + Bias(单 127 / 双 1023)	偏阶编码
浮点乘法指数: $Exp = Exp_1 + Exp_2 - Bias$	去掉重复偏阶
行波进位延迟 ≈ 2n 级门延迟	加法器性能

第 4 章 处理器

公式	说明
流水线执行时间 ≈ (指令数 + 级数 - 1) × 周期	五级流水线

公式	说明
理想加速比 $\approx$ 级数	五级 $\approx$ 5 倍
理想 CPI = 1 / 发射宽度	多发射
单周期执行时间 = 指令数 $\times$ 最长指令时间	单周期基线

第 5 章 存储器

公式	说明
AMAT = 命中时间 + 缺失率 $\times$ 缺失代价	平均访存时间
多级:AMAT = $T_1 + \text{缺}_1 \times (T_2 + \text{缺}_2 \times T_3)$	两级 cache
块数 = cache 容量 $\div$ 块大小	直接映射
索引位数 = $\log_2(\text{块数} \div \text{路数})$	组相联
偏移位数 = $\log_2(\text{块大小})$	字节计
标记位数 = 地址位 - 索引位 - 偏移位	地址划分
优化收益 = 降低的缺失率 $\times$ 缺失代价	权衡

第 6 章 并行

公式	说明
加速比 = $1 / [(1-P) + P/N]$	Amdahl 定律
加速比极限 = $1 / (1-P)$	$N \rightarrow \infty$ 的天花板

二、MIPS 寄存器速查(必背)

名字	编号	用途	保存责任
\$zero	0	常数 0	—
\$at	1	汇编器保留	汇编器
\$v0-\$v1	2-3	返回值	调用者
\$a0-\$a3	4-7	参数	调用者
\$t0-\$t7	8-15	临时	调用者
\$s0-\$s7	16-23	保存	被调用者
\$t8-\$t9	24-25	临时	调用者

名字	编号	用途	保存责任
\$k0-\$k1	26-27	OS 保留	—
\$gp	28	全局指针	被调用者
\$sp	29	栈指针	被调用者
\$fp	30	帧指针	被调用者
\$ra	31	返回地址	被调用者(用则存)

三、MIPS 指令速查(完整版见第 2 章)

类别	指令	功能
算术	add / sub / addi / addu / subu / addiu / mul	加减(溢出检测与否)
乘除	mult / div → mfhi / mflo	积/商入 Hi/Lo
逻辑	and / or / nor / andi / ori	按位运算、掩码
移位	sll / srl / sra	左移 / 逻辑右移 / 算术右移
访存	lw / sw / lb / lbu / sb / lh / lhu / sh / lui	字/字节/半字、装载高 16 位
比较	slt / sltu / slti / sltiu	小于置位(有/无符号)
分支	beq / bne	相等/不等则跳(PC 相对)
跳转	j / jal / jr	无条件 / 调用 / 返回
同步	ll / sc	原子交换
浮点	lwc1 / swc1 / add.s / add.d / c.eq.s / bc1t	浮点访存、运算、比较分支
伪指令	move / li / la / blt / bgt / ble / bge	汇编器展开

指令格式速记:

- R 型: op(6) rs(5) rt(5) rd(5) shamt(5) funct(6)
- I 型: op(6) rs(5) rt(5) imm(16)
- J 型: op(6) addr(26)

关键编码:R 型 op=0;add funct=32、sub=34、and=36、or=37、nor=39、slt=42、jr=8、sll=0、srl=2、sra=3;j=2、jal=3、beq=4、bne=5、addi=8、andi=12、ori=13、slti=10、lw=35、sw=43。

四、单周期控制信号表(第 4 章必考)

信号	R型	lw	sw	beq
RegDst	1	0	×	×

信号	R型	lw	sw	beq
ALUSrc	0	1	1	0
MemtoReg	0	1	×	×
RegWrite	1	1	0	0
MemRead	0	1	0	0
MemWrite	0	0	1	0
Branch	0	0	0	1
ALUOp	10	00	00	01

ALUOp 含义:00=加(算地址)、01=减(比分支)、10=看 funct。

五、冒险解决速记(第 4 章必考)

冒险	解决	代价
ALU 结果被后 1~2 条用	转发(EX/MEM 或 MEM/WB)	0 周期
load 结果紧跟使用(load-use)	阻塞 1 拍 + 转发	1 周期
分支方向未知	ID 段提前判断 + 预测不跳 + 猜错冲刷	猜错 1 周期

六、IEEE 754 速记(第 3 章必考)

	单精度	双精度
总位 / 符号 / 指数 / 尾数	32 = 1+8+23	64 = 1+11+52
Bias	127	1023
有效精度(含隐藏位)	24 位	53 位
特殊值	Exp 全1 Frac=0→±∞;Exp 全1 Frac≠0→NaN;Exp 全0 Frac≠0→非规格化;全0→±0	

七、存储器层次四问框架(简答万能模板)

1. 块放哪?→ 直接映射 / 组相联 / 全相联
2. 怎么找?→ 索引定位 + 标记比对(虚拟存储:页表/TLB)
3. 替换谁?→ LRU / 随机
4. 写怎么办?→ 写直达 / 写回(配脏位)

八、全书核心名词索引(名词 → 章节)

名词	章	名词	章
ISA / 抽象 / 摩尔定律 / CPI / 吞吐率 / 功耗墙	1	补码 / 符号扩展 / 大小端 / 伪指令 / 栈帧	2
调用者保存 / 被调用者保存 / 寻址方式 / 重定位	2	溢出 / 先行进位 / Booth / IEEE754 / 偏阶 / 隐藏位	3
NaN / 非规格化数 / 保护位 / 粘贴位 / 子字并行	3	数据通路 / 流水线 / 转发 / 气泡 / 冲刷	4
冒险(三类) / EPC / 多发射 / 乱序 / 重命名 / 延迟槽	4	局部性 / 组相联 / 写回 / 脏位 / AMAT / TLB / 缺页	5
Amdahl / SIMD / MIMD / SPMD / warp / SMT / MESI / 自旋锁 / MapReduce	6	组合逻辑 / 时序逻辑 / MUX / 寄存器组 / FSM	附录 B

九、高频考点清单(自检用,逐条打勾)

第 1 章: □ 执行时间三因子计算 □ 相对性能计算 □ 功耗公式 □ 摩尔定律概念

第 2 章: □ 补码转换 □ 符号扩展 □ 机器码 ↔ 汇编互译 □ 过程调用模板默写 □ 保存约定 □ 五种寻址方式 □ 程序翻译流程

第 3 章: □ 溢出判定 □ 优化乘法器手算 □ 十进制 ↔ IEEE754 互转 □ 浮点加法手算 □ 特殊值含义

第 4 章: □ 控制信号表 □ ALUOp 两级译码 □ 流水线时空图 □ 找冒险 + 解决方案 □ load-use 阻塞 □ 控制冒险三招 □ EPC/Cause

第 5 章: □ 地址划分计算(标记/索引/偏移) □ AMAT 计算 □ 三种映射对比 □ 写策略对比 □ TLB 与缺页区别 □ 四问框架

第 6 章: □ Amdahl 计算 □ Flynn 分类 □ GPU vs CPU □ 三种多线程 □ MESI 状态 □ 共享内存 vs 消息传递

恭喜! 到这里,你已经完整走完了《计算机组成与设计》六章 + 附录的知识体系。最后建议:回到 [README.md](#) 的路线图,用每章的自测题再做一轮全真检验。